

Comprendre et construire un MiniGPT

Des mathématiques de terminale
au Transformer en PyTorch

100 pages A4 · 96 fiches avec exercices corrigés

Le fil conducteur

Un texte devient une suite d'entiers. Ces entiers désignent des vecteurs. L'attention combine les informations accessibles dans le passé. Le réseau attribue des scores aux prochains tokens. Une fonction de coût mesure l'erreur, puis la rétropropagation indique comment modifier ses paramètres. Enfin, la génération répète la prédiction, un token après l'autre.

Un livre fondé sur une implémentation réelle

Ce manuel accompagne les sept scripts Python, le notebook pédagogique et les tests du dépôt [hackolite/SHOKOBO](https://github.com/hackolite/SHOKOBO). Il distingue systématiquement les principes généraux des choix précis de cette implémentation : tokenisation caractère ou BPE, blocs post-normalisés, positions apprises ou RoPE, attention manuelle ou SDPA, et cache KV à fenêtre reconstruite.

Périmètre de lecture : code de référence `ae314c6`. Les exemples numériques du manuel sont didactiques, sauf mention explicite d'une valeur calculée à partir du code. Aucun résultat d'entraînement ni benchmark matériel n'est présenté comme une mesure effectuée.

Édition française — septembre 2026.

Mode d'emploi et conventions

À qui s'adresse ce manuel ?

Vous savez manipuler une fonction, lire une somme, calculer une dérivée simple et utiliser les probabilités du lycée. Vous n'avez pas besoin d'avoir étudié les matrices, les tenseurs, l'optimisation ou les réseaux de neurones : les premières fiches construisent ces notions. Une connaissance élémentaire de Python aide ; les objets du langage utiles au projet sont expliqués au fil du texte. Ce livre n'est toutefois pas un cours exhaustif du langage Python.

Comment travailler une fiche

Lisez son objectif, refaites les calculs au crayon, puis retrouvez le symbole indiqué sous le titre dans le fichier source. Cachez la correction avant de résoudre l'exercice. Les explications concernent le code exécuté, qui fait foi lorsqu'un schéma introductif simplifie certains détails. Une fiche occupe une page ; le sommaire donne à la fois son numéro et sa page physique.

Quatre parcours successifs

1. Fiches 1–24 : construire les outils mathématiques et comprendre comment une probabilité devient une erreur à minimiser.
2. Fiches 25–48 : suivre le texte jusqu'aux lots de fenêtres, sans contamination de la validation.
3. Fiches 49–72 : parcourir les embeddings, l'attention, le Transformer et leurs variantes, en contrôlant chaque dimension.
4. Fiches 73–96 : entraîner, évaluer, sauvegarder, générer et expérimenter sans confondre démonstration et performance industrielle.

Dictionnaire des dimensions

Symbole	Signification
B	Nombre de fenêtres dans un lot (batch)
T	Longueur de la séquence actuellement traitée
C	Longueur maximale du contexte
V	Taille du vocabulaire (ne pas confondre avec les valeurs d'attention)
D	Dimension des représentations (embedding)
H	Nombre de têtes ; $d_k = D/H$ est leur dimension
F	Dimension cachée du réseau feed-forward
L	Nombre de blocs Transformer

Les indices commencent généralement à zéro, comme en Python. Le logarithme est naturel : les pertes sont donc exprimées en nats. Une matrice de valeurs d'attention pourra être notée V_{att} lorsque le contexte ne suffit pas à la distinguer de la taille du vocabulaire.

Exécution et limites

Les chemins absolus correspondent au clone fourni ; adaptez leur racine sur votre ordinateur. Les commandes d'expérimentation utilisent des sorties séparées sous `/tmp` pour ne pas écraser vos checkpoints. Un modèle minuscule entraîné sur un petit texte n'est ni un assistant fiable ni une source factuelle. L'exhaustivité recherchée est celle du pipeline de ce dépôt, pas celle de tout l'apprentissage profond.

Table des matières

1	Prédire le suivant : la chaîne complète	5
2	Fonctions, arguments et indices Python	6
3	Sommes, moyennes et pondérations	7
4	Vecteurs : des listes de nombres organisées	8
5	Produit scalaire : comparer et combiner	9
6	Matrices : lignes, colonnes et tables de paramètres	10
7	Produit matriciel : un calcul complet	11
8	Transposée et contrôle des dimensions	12
9	Tenseurs : lire les axes B, T et D	13
10	Diffusion, view, transpose et contiguous	14
11	Fonctions affines et orientation de nn.Linear	15
12	Exponentielle et stabilité numérique	16
13	Logarithme naturel : transformer les produits en sommes	17
14	Probabilités conditionnelles : prévoir avec un contexte	18
15	Loi catégorielle et espérance	19
16	Chaîne probabiliste autorégressive	20
17	Softmax : définition et invariance par translation	21
18	Softmax chiffré : du tableau aux probabilités	22
19	Log-vraisemblance et entropie croisée	23
20	Dérivée : mesurer une variation locale	24
21	Règle de chaîne : dériver une composition	25
22	Gradient : une dérivée pour chaque paramètre	26
23	Rétropropagation : un petit réseau calculé entièrement	27
24	Descente de gradient et synthèse des fondements	28
25	Se repérer : scripts, imports et notebook	29
26	Texte Unicode, octets et vocabulaire	30
27	Construire le tokenizer caractère	31
28	Encoder, décoder et perdre une information	32
29	Sauvegarder l'état caractère sans réapprendre	33
30	BPE : apprendre des fusions, à la main	34
31	BPE byte-level : partir des 256 octets	35
32	Le budget d'apprentissage BPE et son biais	36
33	État BPE et frontières de tokenisation	37
34	Autorégression : une entrée, sa cible décalée	38
35	Compter les fenêtres du dataset en mémoire	39
36	Du dataset au batch : empiler les fenêtres	40
37	Éviter la fuite : séparer avant d'apprendre	41
38	Split contigu : ce que mesure la validation	42
39	Lire par blocs Unicode : les trois passages	43
40	Préparer les artefacts : protection et limites	44
41	Des IDs binaires de huit octets	45
42	Fenêtres memmap : formule et pas de déplacement	46
43	Memmap, copies et séparation des processus	47
44	Échantillonner avec remise n'est pas mélanger	48
45	Corpus trop court : identifier l'étape qui refuse	49
46	Lire les tests : preuves locales et angles morts	50
47	Atelier : préparer puis observer char et BPE	51
48	Auditer tout le pipeline avant le Transformer	52
49	Un identifiant choisit une ligne	53
50	Ajouter des positions apprises	54
51	Requêtes, clés et valeurs : trois projections	55
52	Construire la grille des scores	56
53	L'exemple minuscule, sans masque	57
54	Interdire le futur avant le softmax	58
55	Pourquoi diviser par la racine de la dimension ?	59
56	Les poids combinent les valeurs	60
57	Découper les coordonnées en têtes	61
58	Fusionner puis projeter : l'ordre exact	62

59	Auditer le parcours de l'attention	63
60	Le réseau feed-forward et GELU	64
61	La connexion résiduelle et le gradient	65
62	LayerNorm : normaliser un token	66
63	Le bloc est post-normalisé	67
64	Traverser MiniGPT de bout en bout	68
65	La tête produit des logits, pas des probabilités	69
66	Initialiser : écart-type et poids indépendants	70
67	Compter tous les paramètres	71
68	RoPE : tourner une paire de coordonnées	72
69	Fréquences, distance relative et décalage	73
70	SDPA : même calcul, contrat de masque différent	74
71	Le cache KV : un contrat d'inférence	75
72	Ce que vérifient réellement les tests	76
73	Choisir une configuration sans deviner les défauts	77
74	Passer des logits à la perte du lot	78
75	Respecter l'ordre réel de la rétropropagation	79
76	Comprendre les deux mémoires d'Adam	80
77	Lire AdamW sans lui ajouter des mécanismes absents	81
78	Séparer dropout, mode et enregistrement des gradients	82
79	Évaluer une moyenne réellement pondérée par token	83
80	Interpréter les courbes sans inventer un test indépendant	84
81	Atelier CPU : vérifier tout le parcours en quelques pas	85
82	Sauvegarder un modèle n'est pas reprendre un entraînement	86
83	Recharger les formats récents, anciens et le cas absent	87
84	Générer à partir du dernier logit seulement	88
85	Calculer l'effet de la température	89
86	Top-k : un seuil qui conserve aussi les égalités	90
87	Construire puis prolonger le cache des clés et valeurs	91
88	Au débordement, reconstruire toute la fenêtre	92
89	Estimer le coût du cache sans promettre une vitesse	93
90	Deux usages différents de bfloat16	94
91	Mesurer SDPA et le cache avec un protocole explicite	95
92	Comprendre le débit et les compteurs de mémoire	96
93	Ce que prouvent réellement les tests d'intégration	97
94	Comparer des expériences à unités constantes	98
95	Diagnostiquer une erreur avant de changer le modèle	99
96	Projet final : expliquer, vérifier et conclure avec prudence	100

Fiche 1 — Prédire le suivant : la chaîne complète

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/05_model.py

MiniGPT.forward

Objectif. Comprendre ce que calcule un modèle de langage avant d'étudier ses opérations.

Un texte arrive sous forme de caractères, non de nombres réels. Un *tokeniseur* le transforme en une suite d'identifiants entiers. Dans le mode caractère du dépôt, un token représente généralement un caractère ; dans le mode BPE, il peut représenter un fragment différent. Le vocabulaire est l'ensemble des tokens disponibles et sa taille se note V . Un identifiant est une étiquette : le token numéro 8 n'est pas deux fois plus important que le token numéro 4.

La tâche est de prédire le token immédiatement suivant à partir des tokens déjà visibles. Pour une suite fictive (2, 5, 1, 4), une fenêtre fournit l'entrée (2, 5, 1) et les cibles (5, 1, 4). À la première position, le modèle doit proposer 5 en ne connaissant que 2 ; à la deuxième, il dispose du préfixe (2, 5). Le mécanisme causal empêche de consulter les positions futures, même lorsqu'elles sont présentes dans le tableau.

À comprendre. La chaîne de calcul.

Dans `MiniGPT.forward`, `token_embedding` transforme chaque identifiant en une liste de D nombres, appelée représentation. Le mode positionnel appris ajoute `position_embedding`. Puis `blocks`, `final_norm` et `lm_head` produisent des scores, les *logits*. Le mode RoPE traite autrement les positions ; cela ne change pas l'objectif prédictif.

texte $\longrightarrow$ identifiants $\longrightarrow$ représentations $\longrightarrow$ logits $\longrightarrow$ probabilités.

Chaque position produit V scores, pas un identifiant directement. Une transformation appelée softmax donnera des probabilités positives dont la somme vaut un. À l'entraînement, on mesure la probabilité attribuée à la cible connue. En génération, on choisit un token, on l'ajoute à la suite, puis on recommence. Ces deux usages partagent le même prédictif.

Nous noterons B le nombre de fenêtres traitées ensemble, T leur longueur courante et C la capacité maximale de contexte. Sans cache, on doit avoir $1 \leq T \leq C$. Les L blocs transforment les représentations sans changer le nombre de positions. Aucun de ces nombres ne désigne une quantité de mots comprise par la machine.

Exercice. Avec $B = 2$, $T = 3$ et $V = 4$, combien de scores et combien de cibles un passage produit-il ? Quelle cible correspond au dernier token d'entrée de notre exemple ?

Correction détaillée. Chaque fenêtre fournit trois listes de quatre scores, donc $2 \times 3 \times 4 = 24$ scores pour six cibles. La dernière entrée vaut 1, mais sa cible vaut 4 : recopier l'entrée ne résoudrait pas la tâche. Les probabilités portent sur quatre possibilités à chacune des six positions, non sur les six positions elles-mêmes.

Fiche 2 — Fonctions, arguments et indices Python

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/02_dataset.py

NextTokenDataset

Objectif. Traduire les notations mathématiques en accès à une suite Python sans décalage accidentel.

Une fonction associe une sortie à une entrée autorisée. Au lycée, $f(x) = 2x + 1$ signifie qu’une même recette s’applique à chaque valeur de x . Un programme peut recevoir une liste plutôt qu’un seul nombre. `SimpleTokenizer.encode`, la méthode d’encodage du tokeniseur, reçoit un texte et renvoie une liste d’entiers. Son domaine n’est donc pas celui d’une fonction polynomiale, mais l’idée d’une transformation bien définie reste identique.

Une suite mathématique peut commencer à l’indice zéro : écrivons $s_0, s_1, \dots, s_{N-1}$ pour N tokens. Python adopte précisément cette convention. `s[0]` est le premier élément, tandis que `s[N-1]` est le dernier. Les crochets indiquent un accès, pas une multiplication. Un indice compte les positions ; la valeur rangée à cette position est une information distincte.

À comprendre. Les tranches excluent leur borne de droite.

L’expression `s[a:b]` prend les indices $a, a+1, \dots, b-1$. Si les bornes restent dans la liste et si $b \geq a$, elle contient $b-a$ éléments. Dans `NextTokenDataset`, la longueur de fenêtre est `context_length`, que nous notons C .

$$x_t = s_{i+t}, \quad y_t = s_{i+t+1}, \quad 0 \leq t < C.$$

Pour $s = (4, 7, 2, 9, 3)$, $i = 1$ et $C = 3$, l’entrée utilise les indices 1, 2, 3 : elle vaut $(7, 2, 9)$. La cible utilise les indices 2, 3, 4 : elle vaut $(2, 9, 3)$. Les deux listes ont la même longueur, mais leurs contenus sont décalés.

Lorsqu’on écrit une formule, annoncer le début des indices évite une ambiguïté fréquente : le « troisième élément » est ici celui d’indice deux, pas celui d’indice trois.

Il faut un token supplémentaire pour connaître la dernière cible. Avec un pas de déplacement égal à un, les débuts admissibles vont de zéro à $N - C - 1$, inclus : cela fait $N - C$ fenêtres. Ne confondons pas cette borne maximale avec le nombre de fenêtres. Le jeu de données sur disque ajoute un paramètre de pas ; son comptage tient aussi compte de ce paramètre.

Enfin, `logits[:, -1, :]` emploie un indice négatif pour sélectionner la dernière position. Les deux deux-points conservent les autres axes : ce n’est pas une sélection du dernier token du vocabulaire.

Exercice. Pour $s = (8, 1, 6, 2, 5, 7)$, $C = 2$ et $i = 2$, donner `s[i:i+C]`, la cible et le nombre total de fenêtres.

Correction détaillée. L’entrée vaut $(6, 2)$, la cible $(2, 5)$. Les indices de début possibles sont 0, 1, 2, 3, soit $6 - 2 = 4$ fenêtres. Le début 4 serait impossible : sa deuxième cible demanderait l’élément d’indice 6, absent de cette suite.

Fiche 3 — Sommes, moyennes et pondérations

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

evaluate

Objectif. Lire le symbole somme et comprendre pourquoi une moyenne de moyennes peut tromper.

Le signe $\sum$, lettre grecque sigma, abrège une addition répétée. L'indice écrit en dessous parcourt les valeurs indiquées ; chaque terme est calculé avant d'être ajouté. Par exemple, $\sum_{i=0}^3 a_i = a_0 + a_1 + a_2 + a_3$. Cet indice est local : remplacer partout i par j ne modifie pas le résultat. En revanche, changer les bornes change les termes inclus.

La moyenne arithmétique de $n > 0$ valeurs partage leur somme en n parts égales. Pour les erreurs 1, 2, 3, 6, la somme vaut 12 et la moyenne vaut 3. Elle a la même unité que les valeurs initiales. La somme augmente généralement quand on ajoute des observations ; la moyenne permet de comparer des ensembles de tailles différentes.

$$\bar{a} = \frac{1}{n} \sum_{i=0}^{n-1} a_i, \quad \sum_{i=0}^{n-1} (a_i - \bar{a}) = 0.$$

La deuxième identité se vérifie en développant : la somme des a_i vaut $n\bar{a}$, que l'on soustrait à elle-même. Elle ne signifie pas que les écarts individuels sont tous nuls.

À comprendre. Compter les observations plutôt que les lots.

Dans `evaluate`, `loss.item()` représente, avec le critère utilisé par l'entraînement, une erreur moyenne sur les tokens du lot. `y.numel()` compte ces tokens. Le programme multiplie les deux, additionne les résultats, puis divise par `total_tokens`. Un lot final plus petit reçoit ainsi un poids proportionnel à sa taille.

Supposons un premier lot de six cibles avec une erreur moyenne de 2, puis un lot de deux cibles avec une moyenne de 6. Les erreurs totales valent respectivement 12 et 12. La moyenne globale est donc $(12 + 12)/8 = 3$, et non $(2 + 6)/2 = 4$. Cette dernière opération donnerait autant d'importance à deux cibles qu'à six.

Plus généralement, une moyenne pondérée utilise des coefficients positifs dont la somme vaut un. Ici, les poids sont $6/8$ et $2/8$. Nous retrouverons cette idée avec les probabilités et, plus tard, avec les mélanges de représentations. Les poids n'ont pas besoin d'être égaux pour former une moyenne.

Exercice. Trois lots contiennent respectivement 4, 4 et 2 tokens. Leurs pertes moyennes valent 1, 3 et 5. Calculer la perte globale.

Correction détaillée. La somme des pertes vaut $4 \times 1 + 4 \times 3 + 2 \times 5 = 26$. Le nombre de tokens vaut 10, donc la moyenne correcte est 2,6. La moyenne non pondérée des trois valeurs serait 3 : elle surestime ici l'importance du dernier petit lot.

Fiche 4 — Vecteurs : des listes de nombres organisées

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/05_model.py

MiniGPT.forward

Objectif. Manipuler une représentation de token sans supposer de connaissances d’algèbre linéaire.

Un vecteur de dimension D est ici une liste ordonnée de D nombres réels. Dire $u \in \mathbb{R}^D$ signifie simplement $u = (u_0, \dots, u_{D-1})$, avec une valeur réelle à chaque place. L’ordre compte : $(1, 2)$ et $(2, 1)$ ne sont pas le même vecteur. Le mot dimension compte les coordonnées, non les tokens ou les couches.

Dans MiniGPT, `token_embedding` contient une représentation apprise pour chaque identifiant. Si $D = 3$, un token pourrait recevoir $(0, 2, -0, 1, 0, 7)$. Ces coordonnées ne sont pas imposées comme « sujet », « verbe » et « adjectif ». Elles sont ajustées ensemble par l’apprentissage ; leur interprétation ne se lit pas directement dans leurs indices.

À comprendre. Deux opérations élémentaires.

Pour additionner deux vecteurs de même dimension, on additionne leurs coordonnées correspondantes. Pour multiplier un vecteur par un nombre, on multiplie toutes ses coordonnées par ce nombre, appelé scalaire.

$$(1, -2, 3) + (4, 5, -1) = (5, 3, 2), \quad 2(1, -2, 3) = (2, -4, 6).$$

Une addition ne concatène donc pas les listes : sa dimension reste 3. Au contraire, les placer bout à bout donnerait six coordonnées et constituerait une autre opération.

Soustraire deux vecteurs fonctionne de la même manière. Le vecteur différence décrit un déplacement coordonnée par coordonnée ; sa norme mesure la distance entre les deux représentations.

Dans le mode positionnel appris, `tok_emb + pos_emb` ajoute un vecteur lié au token et un vecteur lié à sa position. Pour un token représenté par $(1, 0, -1)$ et une position représentée par $(0, 1, 0, 2, 0, 3)$, le résultat vaut $(1, 1, 0, 2, -0, 7)$. Le même token placé ailleurs garde sa représentation initiale, mais reçoit une autre contribution positionnelle.

La longueur géométrique, ou norme euclidienne, généralise Pythagore : $\|u\| = \sqrt{u_0^2 + \dots + u_{D-1}^2}$. Ainsi $\|(3, 4)\| = 5$. Cette longueur mesure une amplitude, pas une probabilité. Des coordonnées négatives et des normes supérieures à un sont parfaitement autorisées pour les représentations du modèle.

Il est utile de séparer trois notions : l’identifiant entier choisit une ligne de table ; le vecteur représente ce token ; sa norme résume une seule propriété de ce vecteur. Deux vecteurs de même norme peuvent contenir des informations très différentes.

Exercice. Un token a pour représentation $u = (2, -1)$, sa position $p = (-1, 3)$. Calculer $h = u + p$, $h/2$, puis $\|h\|$.

Correction détaillée. On obtient $h = (1, 2)$, puis $h/2 = (0, 5, 1)$. Sa norme vaut $\sqrt{1^2 + 2^2} = \sqrt{5}$. Le vecteur h possède toujours deux coordonnées : l’addition positionnelle n’a ni ajouté un token ni doublé la dimension.

Fiche 5 — Produit scalaire : comparer et combiner

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

scaled_dot_product_attention

Objectif. Calculer un score à partir de deux vecteurs et distinguer ce score d’une probabilité.

Le produit scalaire de deux vecteurs de même dimension consiste à multiplier les coordonnées correspondantes, puis à additionner les produits. Son résultat est un seul nombre, d’où son nom. Avec $u = (1, 2, -1)$ et $v = (3, 0, 4)$, les trois contributions sont 3, 0, -4, donc le score total vaut -1.

$$u \cdot v = \sum_{j=0}^{D-1} u_j v_j, \quad (1, 2, -1) \cdot (3, 0, 4) = 3 + 0 - 4 = -1.$$

La dimension doit être identique pour savoir quelles coordonnées apparier. Le produit scalaire diffère du produit coordonnée par coordonnée : ce dernier conserverait la liste $(3, 0, -4)$, alors que le produit scalaire la réduit à sa somme.

L’ordre des deux vecteurs ne change pas ce nombre, puisque chaque produit de réels peut être inversé. En revanche, permuter les coordonnées d’un seul vecteur modifie généralement les associations et donc le score.

À comprendre. Une lecture comme combinaison pondérée.

Si u contient des observations et v des coefficients, le score $u \cdot v$ mesure leur combinaison. Un coefficient nul ignore une observation ; un coefficient négatif inverse son effet. Le modèle apprend précisément des coefficients de ce type. Tous les coefficients ne sont pas nécessairement positifs.

Une deuxième lecture est géométrique. Pour des vecteurs non nuls, $u \cdot v = \|u\| \|v\| \cos \theta$, où θ est l’angle entre leurs directions. Un angle droit donne zéro. Mais zéro peut aussi venir de contributions qui s’annulent : $(1, 1) \cdot (1, -1) = 1 - 1 = 0$. Il n’implique donc pas que toutes les coordonnées soient nulles.

Les exemples du script appellent la fonction `scaled_dot_product_attention`. La ligne `q @ k.transpose(-2, -1)` calcule de nombreux produits scalaires à la fois. Nous étudierons leur organisation plus tard ; pour l’instant, chaque case obtenue est simplement un score entre deux listes de même longueur.

Un grand score n’est pas une preuve absolue de ressemblance. Multiplier u par 10 multiplie $u \cdot v$ par 10, même si sa direction reste inchangée. C’est pourquoi l’amplitude des vecteurs intervient dans ces scores. La division ultérieure par $\sqrt{d_k}$, avec $d_k = D/H$ pour H têtes, sera expliquée dans les fiches consacrées à l’attention.

Exercice. Calculer $a \cdot b$, puis $(2a) \cdot b$, pour $a = (2, -1, 3)$ et $b = (1, 4, 0)$. Le premier résultat peut-il être utilisé directement comme probabilité ?

Correction détaillée. Le premier score vaut $2 \times 1 + (-1) \times 4 + 3 \times 0 = -2$. Le second vaut $4 - 8 + 0 = -4$, soit deux fois le premier. Une probabilité ne peut pas être négative : ces scores nécessitent une transformation supplémentaire avant une interprétation probabiliste.

Fiche 6 — Matrices : lignes, colonnes et tables de paramètres

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/05_model.py

MiniGPT

Objectif. Lire un tableau rectangulaire et relier ses dimensions à une table de représentations.

Une matrice est un tableau rectangulaire de nombres. Sa forme indique d'abord le nombre de lignes, puis le nombre de colonnes. Une matrice A de forme 2×3 contient donc six nombres. Le coefficient A_{ij} se trouve à la ligne i , colonne j . Nous conservons les indices commençant à zéro pour faciliter la correspondance avec Python.

$$A = \begin{pmatrix} 1 & -2 & 0 \\ 3 & 4 & 5 \end{pmatrix}, \quad A_{0,1} = -2, \quad A_{1,2} = 5.$$

Une ligne de cette matrice est un vecteur de dimension 3. Une colonne possède deux coordonnées. Dire seulement « une matrice de six éléments » perd une information essentielle : les formes 1×6 , 2×3 et 3×2 n'organisent pas les données de la même manière.

À comprendre. Une matrice peut servir de table.

Dans MiniGPT, le poids de `token_embedding` a pour forme $[V, D]$. Chaque ligne correspond à un identifiant, chaque colonne à une coordonnée de représentation. Chercher le token d'identifiant 2 revient à sélectionner la ligne 2, et non à multiplier toutes les valeurs du tableau par 2.

Avec $V = 4$ et $D = 3$, cette table contient $4 \times 3 = 12$ paramètres. Une entrée $(2, 0, 2)$ consulte trois lignes, dont deux fois la même. Les deux occurrences du token 2 reçoivent initialement le même vecteur. Leur position et leur contexte pourront ensuite différencier leurs représentations.

L'addition de matrices exige des formes identiques dans sa définition élémentaire : chaque case s'ajoute à la case correspondante. Multiplier une matrice par un scalaire multiplie toutes ses cases. Ces opérations ne mélangent pas les colonnes ; le produit matriciel, étudié ensuite, réalise précisément un mélange plus structuré.

Les tables ne sont pas toutes carrées. La table positionnelle apprise a la forme $[C, D]$, alors que la table des tokens a la forme $[V, D]$. Elles partagent la dimension des représentations mais n'indexent pas les mêmes objets. C compte les positions admissibles, tandis que V compte les tokens distincts.

Exercice. Une table contient les lignes $(1, 0)$, $(0, 2)$ et $(-1, 3)$. Donner sa forme, son nombre de paramètres et le tableau obtenu en consultant les identifiants $(2, 1)$.

Correction détaillée. La forme est 3×2 , donc la table contient six paramètres. Les lignes sélectionnées sont $(-1, 3)$, puis $(0, 2)$. Le résultat est une matrice 2×2 . Le nombre de lignes de sortie dépend du nombre de tokens consultés, pas de la taille totale du vocabulaire.

Fiche 7 — Produit matriciel : un calcul complet

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

pytorch_attention_example

Objectif. Effectuer chaque case d'un produit et comprendre le rôle de la dimension commune.

Multiplier deux matrices n'est pas multiplier simplement leurs cases. Une ligne de la première rencontre une colonne de la seconde : leur produit scalaire donne une case du résultat. Ainsi une matrice A de forme $m \times n$ peut multiplier une matrice R de forme $n \times p$. Le nombre n doit être commun ; le résultat a la forme $m \times p$.

$$A = \begin{pmatrix} 1 & 2 & -1 \\ 0 & 3 & 2 \end{pmatrix}, \quad R = \begin{pmatrix} 2 & 1 \\ -1 & 0 \\ 4 & 2 \end{pmatrix}.$$

Ici, deux lignes de longueur trois rencontrent deux colonnes de longueur trois. La première case vaut $1 \times 2 + 2 \times (-1) + (-1) \times 4 = -4$. La deuxième vaut $1 \times 1 + 2 \times 0 + (-1) \times 2 = -1$. La troisième vaut $0 \times 2 + 3 \times (-1) + 2 \times 4 = 5$. La dernière vaut $0 \times 1 + 3 \times 0 + 2 \times 2 = 4$.

$$AR = \begin{pmatrix} -4 & -1 \\ 5 & 4 \end{pmatrix}, \quad (AR)_{ij} = \sum_{r=0}^{n-1} A_{ir} R_{rj}.$$

Les indices libres i, j désignent la case conservée. L'indice r , parcouru dans la somme, disparaît du résultat. Cette différence explique pourquoi la dimension commune ne figure plus dans la forme finale.

À comprendre. Le code applique cette recette en parallèle.

L'opérateur Python `@` réalise le produit matriciel pour des tableaux à deux dimensions. Dans le script, l'exemple `pytorch_attention_example` construit notamment `q = x @ wq`. La matrice `wq y` est une matrice explicite de coefficients ; ne lui attribuons pas automatiquement l'orientation interne d'un module `nn.Linear`.

Chaque ligne d'entrée est transformée par les mêmes colonnes de coefficients. On peut donc traiter plusieurs positions sans écrire une boucle Python pour chacune. L'opération ne mélange pas ici les lignes d'entrée entre elles : chaque ligne produit sa propre ligne de sortie.

Dans l'exemple chiffré, chaque case demande trois multiplications et deux additions. Quatre cases nécessitent donc douze multiplications et huit additions. Ce comptage élémentaire permet déjà d'anticiper le travail demandé lorsque la longueur des lignes ou le nombre de sorties augmente.

L'ordre des facteurs importe. Dans notre exemple, AR a la forme 2×2 , mais RA aurait la forme 3×3 . Ils ne peuvent même pas être comparés case par case. Avec des matrices carrées, des formes compatibles ne suffisent toujours pas à garantir l'égalité des deux produits.

Exercice. Pour $X = \begin{pmatrix} 2 & 1 \\ -1 & 3 \end{pmatrix}$ et $w = \begin{pmatrix} 4 \\ -2 \end{pmatrix}$, calculer Xw . Quelle opération donnerait plutôt un tableau 2×2 de produits individuels ?

Correction détaillée. La première sortie vaut $2 \times 4 + 1 \times (-2) = 6$. La seconde vaut $(-1) \times 4 + 3 \times (-2) = -10$. Ainsi $Xw = (6, -10)^T$, de forme 2×1 . Un produit élément par élément, éventuellement avec diffusion, conserverait deux colonnes ; il n'effectuerait pas ces sommes.

Fiche 8 — Transposée et contrôle des dimensions

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

MultiHeadSelfAttention.forward

Objectif. Permuter des axes volontairement plutôt que corriger les formes au hasard.

La transposée d'une matrice échange lignes et colonnes. Si A a la forme $m \times n$, alors A^T a la forme $n \times m$, et sa case (j, i) contient A_{ij} . Les nombres ne changent pas, seule leur organisation change. Deux transpositions successives rendent la matrice initiale. Le nombre total de coefficients est conservé, même si le tableau devient plus haut ou plus large.

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}.$$

Une ligne devient ainsi une colonne. Cette opération permet de présenter des vecteurs dans l'orientation nécessaire à un produit matriciel ; ce n'est ni une inversion ni une division. Même lorsqu'une matrice inverse existe, elle répond à une autre question.

Supposons que Q et K contiennent chacun T lignes de d_k coordonnées. Pour comparer chaque ligne de Q à chaque ligne de K , il faut présenter ces dernières comme des colonnes. Le calcul des formes s'écrit alors :

$$\underbrace{Q}_{T \times d_k} \underbrace{K^T}_{d_k \times T} = \underbrace{S}_{T \times T}, \quad S_{ij} = \sum_{r=0}^{d_k-1} Q_{ir} K_{jr}.$$

La case S_{ij} correspond à la ligne i de Q et à la ligne j de K . En général S_{ij} n'égale pas S_{ji} , car Q et K sont deux tableaux différents.

À comprendre. Les axes négatifs se comptent depuis la droite.

Dans `scaled_dot_product_attention`, appelée par `MultiHeadSelfAttention.forward`, `k.transpose(-2, -1)` échange les deux derniers axes. Les axes précédents, dédiés notamment aux lots et aux têtes, restent en place. Échanger tous les axes d'un tableau plus grand serait donc une opération différente.

Une autre règle utile est que la transposée d'un produit inverse l'ordre des facteurs : $(AR)^T = R^T A^T$. On peut le vérifier sur chaque case en développant la somme. Cette inversion d'ordre est indispensable : garder le même ordre donnerait souvent des dimensions incompatibles, avant même de considérer les valeurs.

Un contrôle de dimensions doit précéder les valeurs numériques. Avec $T = 5$ et $d_k = 2$, le produit attendu est $(5 \times 2)(2 \times 5)$, donc 5×5 . Tenter $(5 \times 2)(5 \times 2)$ échoue parce que 2 et 5 ne coïncident pas. Si ces nombres étaient accidentellement égaux, l'absence d'erreur informatique ne prouverait pas la justesse du sens.

Exercice. Soient $Q = ((1, 0), (0, 2))$ et $K = ((3, 1), (4, -1))$, les couples représentant les lignes. Calculer QK^T et expliquer sa case $(1, 0)$.

Correction détaillée. Les produits scalaires donnent les lignes $(3, 4)$ et $(2, -2)$. La case $(1, 0)$ vaut $0 \times 3 + 2 \times 1 = 2$: elle compare la deuxième ligne de Q à la première ligne de K , conformément à notre convention d'indices commençant à zéro.

Fiche 9 — Tenseurs : lire les axes B, T et D

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/05_model.py

MiniGPT.forward

Objectif. Suivre les tableaux du modèle grâce à la signification de leurs axes.

Dans ce manuel, un tenseur est un tableau de nombres muni de zéro, un ou plusieurs axes. Un scalaire n'a aucun axe ; un vecteur en a un ; une matrice en a deux. Cette définition pratique suffit pour lire PyTorch, sans introduire la théorie géométrique générale des tenseurs. Le nombre d'axes et la taille de chaque axe sont deux informations différentes.

L'entrée `x` de `MiniGPT.forward` a la forme $[B, T]$. B compte les exemples réunis dans le lot ; T compte les tokens de chaque exemple. Ainsi `x[1, 2]` est l'identifiant du troisième token du deuxième exemple. Après consultation de la table, `tok_emb` a la forme $[B, T, D]$, car chaque identifiant a été remplacé par D coordonnées.

$$[B, T] \longrightarrow [B, T, D] \longrightarrow [B, T, V].$$

La dernière forme appartient aux logits : chaque position reçoit un score pour chacun des V tokens du vocabulaire. L'axe des caractéristiques D a été remplacé par l'axe des classes V , mais les axes des exemples et des positions restent présents.

À comprendre. Lire une case avant de compter toutes les cases.

Dans un tenseur h , $h_{b,t,j}$ signifie « coordonnée j de la position t dans l'exemple b ». Cette phrase distingue les rôles des indices. Pour $B = 2, T = 3, D = 4$, le tenseur contient $2 \times 3 \times 4 = 24$ nombres. Il possède trois axes, pas vingt-quatre dimensions au sens du nombre d'axes.

La dimension cachée D ne doit pas être confondue avec le contexte maximal C . On peut soumettre une séquence plus courte que C sans changer D . De même, ajouter des couches L augmente le nombre de transformations, pas le nombre de positions. Le réseau intermédiaire emploie une largeur F , puis revient à D .

Au moment du calcul de la perte, `reshape(-1, logits.size(-1))` réunit les axes B, T en un axe de BT prédictions. Les cibles sont aplaties dans le même ordre. Il faut préserver cet alignement : avoir autant de lignes que de cibles ne suffit pas si elles ont été permutées différemment.

Exercice. Avec $B = 3, T = 5, D = 8, V = 11$, donner les formes et nombres d'éléments des représentations et des logits. Quelle est la forme des logits aplaties pour la perte ?

Correction détaillée. Les représentations ont la forme $[3, 5, 8]$, soit 120 nombres. Les logits ont la forme $[3, 5, 11]$, soit 165 nombres. L'aplatissement produit $[15, 11]$, accompagné de 15 cibles : on conserve onze choix pour chacune des quinze prédictions.

Fiche 10 — Diffusion, view, transpose et contiguous

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

MultiHeadSelfAttention.forward

Objectif. Distinguer une opération sur les valeurs d’une réorganisation des axes et de la mémoire.

La *diffusion*, ou broadcasting, rend certaines opérations possibles entre formes différentes. PyTorch aligne les axes depuis la droite. Sur chaque axe, les tailles doivent être égales, ou l’une doit valoir un ; les axes absents se comportent comme des axes de taille un. La valeur singleton est réutilisée aux positions nécessaires.

Dans `MiniGPT.forward`, `tok_emb` a la forme $[B, T, D]$ et `pos_emb` la forme $[1, T, D]$. Leur addition réutilise les mêmes représentations positionnelles pour chaque exemple. Le résultat est $[B, T, D]$, sans qu’il soit nécessaire d’écrire B copies de la table positionnelle.

À comprendre. Changer de forme ne signifie pas permuter.

`view` réinterprète une organisation compatible des mêmes éléments. Il conserve leur nombre et exige des contraintes sur leur disposition en mémoire. Une suite de 12 valeurs peut devenir un tableau $[3, 4]$, mais pas $[3, 5]$. La valeur -1 demande à PyTorch de déduire une taille ; une seule dimension peut être ainsi laissée indéterminée.

$$[B, T, D] \xrightarrow{\text{view}} [B, T, H, d_k] \xrightarrow{\text{transpose}(1,2)} [B, H, T, d_k], \quad D = Hd_k.$$

Ce sont les opérations utilisées pour `q`, `k` et `v` dans `MultiHeadSelfAttention.forward`. La première sépare les coordonnées en H groupes ; la seconde échange les axes des positions et des groupes. Le détail du calcul des têtes appartient aux fiches sur l’attention.

`transpose` peut produire une vue dont les éléments voisins logiquement ne sont plus voisins dans le stockage. Les pas mémoire, appelés *strides*, décrivent alors comment retrouver chaque élément. Une telle vue n’est pas nécessairement contiguë.

À la sortie, le code utilise `transpose(1, 2).contiguous().view(...)`. `contiguous` garantit une disposition contiguë, en effectuant une copie si nécessaire, avant la réunion des groupes. `reshape` peut choisir une vue ou une copie ; il ne remplace cependant jamais une permutation d’axes manquante.

Exercice. Une matrice contient les lignes $(0, 1, 2)$ et $(3, 4, 5)$. Comparer sa transposée et sa réorganisation contiguë en forme $[3, 2]$. Pourquoi le même nombre d’éléments ne garantit-il pas le même résultat ?

Correction détaillée. La transposée contient les lignes $(0, 3), (1, 4), (2, 5)$. Un `view(3, 2)` sur la matrice initiale contiguë donne $(0, 1), (2, 3), (4, 5)$. Les deux résultats contiennent six nombres, mais ils n’associent pas les mêmes valeurs à chaque indice. Le premier échange les axes ; le second redécoupe l’ordre existant.

Fiche 11 — Fonctions affines et orientation de `nn.Linear`

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/04_transformer.py

FeedForward

Objectif. Passer de $ax + b$ à plusieurs entrées et sorties sans inverser les poids PyTorch.

Au lycée, une fonction affine s'écrit $f(x) = ax + b$. Le coefficient a règle la variation ; b fixe la valeur en zéro. Pour plusieurs entrées, on remplace ax par une somme pondérée. Pour plusieurs sorties, on apprend une somme différente pour chacune.

Un module `nn.Linear(D, F)` reçoit D coordonnées et produit F coordonnées. Malgré son nom, il réalise par défaut une application *affine*, car il contient un biais. La transformation devient linéaire au sens mathématique si ce biais est nul ou désactivé.

$$y_j = \sum_{i=0}^{D-1} W_{ji}x_i + b_j, \quad W \in \mathbb{R}^{F \times D}, \quad b \in \mathbb{R}^F.$$

Attention à l'ordre : `weight` est stocké sous la forme `[out_features, in_features]`, donc $[F, D]$. Si les entrées sont les lignes d'une matrice X , le calcul s'écrit $Y = XW^T + b$, avec diffusion du biais sur les lignes. Écrire XW utiliserait une autre convention.

À comprendre. Un exemple à deux sorties.

Prenons $x = (2, -1)$, les lignes de poids $(3, 4)$ et $(-2, 1)$, puis $b = (1, -3)$. La première sortie vaut $3 \times 2 + 4 \times (-1) + 1 = 3$. La seconde vaut $(-2) \times 2 + 1 \times (-1) - 3 = -8$. Le résultat $(3, -8)$ contient deux scores sans contrainte de positivité.

Dans `FeedForward.net`, la première couche passe de D à F , une fonction GELU transforme les valeurs, puis une seconde couche revient de F à D . Les mêmes paramètres s'appliquent à chaque position de chaque exemple. La forme $[B, T, D]$ devient donc $[B, T, F]$, puis $[B, T, D]$.

Sans transformation non affine entre deux couches affines, leur composition serait encore affine : développer $a_2(a_1x + b_1) + b_2$ donne $(a_2a_1)x + (a_2b_1 + b_2)$. La fonction intermédiaire apporte donc une capacité que la simple succession de sommes pondérées n'aurait pas.

Le biais agit indépendamment de l'entrée : si toutes les coordonnées reçues sont nulles, la sortie est exactement le vecteur des biais. C'est une manière simple de vérifier son rôle et son orientation. Une valeur de biais existe par sortie, pas par entrée ; elle est partagée entre les positions comme les coefficients de la matrice.

Exercice. Combien de paramètres possède `nn.Linear(3, 2)` avec biais ? Pour une entrée $[4, 5, 3]$, quelle forme sort du module ?

Correction détaillée. La matrice contient $2 \times 3 = 6$ coefficients et le biais contient deux nombres, soit huit paramètres. La sortie a la forme $[4, 5, 2]$. Les vingt positions partagent ces huit paramètres : leur nombre ne se multiplie pas par le nombre d'exemples ou de tokens traités.

Fiche 12 — Exponentielle et stabilité numérique

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

softmax_manual

Objectif. Comprendre pourquoi une formule exacte peut échouer avec des nombres flottants.

La fonction exponentielle, notée $\exp(x) = e^x$, associe un nombre strictement positif à tout réel fini x . Elle vaut un en zéro, augmente avec x , et transforme les additions en multiplications : $e^{a+b} = e^a e^b$. Pour un nombre négatif, $e^{-a} = 1/e^a$. Ainsi une entrée négative ne produit jamais une exponentielle négative.

Quelques repères suffisent : $e \approx 2,718$, $e^2 \approx 7,389$ et $e^{-2} \approx 0,1353$. Une différence additive de un dans l'entrée multiplie la sortie par environ 2,718. Cette croissance rapide rend l'exponentielle utile pour comparer des scores, mais délicate numériquement.

À comprendre. La machine n'utilise pas tous les réels.

Un format flottant dispose d'une précision et d'une plage finies. Une valeur trop grande peut devenir $+\infty$: c'est un débordement. Une valeur positive trop petite peut être arrondie à zéro. En float32, e^{1000} déborde largement, bien que ce nombre existe parfaitement en mathématiques.

La fonction `softmax_manual`, définie dans le script d'attention, commence par soustraire le maximum de chaque ligne avant `torch.exp`. Pour les scores (1000, 999), elle exponentie donc (0, -1).

$$\frac{e^{1000}}{e^{1000} + e^{999}} = \frac{1}{1 + e^{-1}} \approx 0,7311.$$

Le quotient initial pourrait devenir une opération indéfinie entre infinis dans le programme naïf. Le quotient réécrit utilise des nombres modestes et donne la même valeur mathématique. Nous démontrerons l'invariance générale du softmax plus loin.

Après soustraction d'un maximum fini, toutes les exponentielles sont au plus égales à un et au moins l'une vaut exactement un. Leur somme ne peut donc pas s'annuler. Des termes très petits peuvent encore être arrondis à zéro, mais ils ne font plus déborder le calcul des exponentielles.

La précision limitée affecte aussi les additions : ajouter une très petite quantité à une très grande peut laisser le résultat inchangé après arrondi. Réécrire une formule n'est donc pas seulement une question de présentation.

Point de vigilance. La stabilisation suppose une ligne contenant un score fini et aucun $+\infty$. Si tous les scores valent $-\infty$, soustraire leur maximum produit des valeurs indéfinies. Masquer toutes les possibilités n'est pas une distribution de probabilités valide.

Exercice. Stabiliser les scores (100, 102, 101), puis donner les trois exponentielles obtenues et la plus grande probabilité.

Correction détaillée. On soustrait 102, d'où (-2, 0, -1). Les exponentielles valent environ (0,1353, 1, 0,3679). Leur somme vaut 1,5032, donc la probabilité maximale vaut environ $1/1,5032 = 0,6652$, à l'indice 1. Le décalage n'a pas changé l'ordre des scores.

Fiche 13 — Logarithme naturel : transformer les produits en sommes

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/05_model.py

MiniGPT.forward

Objectif. Donner un sens au logarithme d'une probabilité et à son opposé.

Le logarithme naturel $\ln$ est la fonction réciproque de l'exponentielle. Pour $x > 0$, $\ln(x)$ est l'unique réel dont l'exponentielle vaut x . Ainsi $\ln(1) = 0$, $\ln(e) = 1$ et $\ln(e^{-2}) = -2$. Son domaine réel exclut zéro et les nombres négatifs.

Cette fonction est croissante : une probabilité plus grande possède un logarithme plus grand. Pourtant, entre zéro et un, le logarithme est négatif. Par exemple, $\ln(0,5) \approx -0,6931$ et $\ln(0,1) \approx -2,3026$. Le signe négatif ne signale pas une erreur ; il est la conséquence normale d'une entrée inférieure à un.

$$\ln(ab) = \ln(a) + \ln(b), \quad \ln(a/b) = \ln(a) - \ln(b), \quad a, b > 0.$$

Ces identités se comprennent en écrivant $a = e^u$, $b = e^v$: leur produit vaut e^{u+v} , donc son logarithme vaut $u + v$. Attention, aucune identité analogue ne donne $\ln(a + b) = \ln(a) + \ln(b)$.

À comprendre. Mesurer la surprise d'une observation.

Pour une cible observée de probabilité p , la quantité $-\ln p$ est positive ou nulle. Elle vaut zéro si $p = 1$ et augmente fortement quand p approche zéro. Cette mesure pénalise donc davantage une cible jugée très improbable. Elle est exprimée en *nats* lorsqu'on utilise le logarithme naturel, plutôt qu'en bits.

Dans `MiniGPT.forward`, `F.cross_entropy` calcule une perte liée à ce logarithme, directement à partir des logits. Le code évite de demander explicitement le logarithme de probabilités déjà arrondies à zéro. Les formules détaillées de cette perte apparaîtront après l'étude du softmax.

Pour une suite de prédictions, multiplier beaucoup de probabilités peut donner un nombre extrêmement petit. Dix probabilités égales à 0,1 donnent 10^{-10} ; leurs logarithmes s'additionnent et donnent $10 \ln(0,1) \approx -23,026$. La représentation logarithmique garde ainsi une échelle plus maniable sans modifier l'ordre de préférence entre probabilités positives.

Le logarithme mesure aussi des améliorations relatives. Passer d'une probabilité à une probabilité deux fois plus grande diminue la surprise de $\ln 2$, quelle que soit la valeur initiale, tant que la nouvelle probabilité reste admissible. Ce comportement diffère d'une erreur mesurée par simple soustraction des probabilités : le gain dépend d'un rapport, pas seulement d'un écart absolu.

Exercice. Deux cibles successives reçoivent les probabilités $1/2$ et $1/4$. Calculer le produit, son logarithme et la somme des surprises.

Correction détaillée. Le produit vaut $1/8$. Comme $8 = 2^3$, $\ln(1/8) = -3 \ln 2 \approx -2,0794$. La somme des surprises vaut $\ln 2 + \ln 4 = 3 \ln 2 \approx 2,0794$. On retrouve exactement l'opposé du logarithme du produit : les contributions locales s'additionnent.

Fiche 14 — Probabilités conditionnelles : prévoir avec un contexte

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/07_generate.py

generate

Objectif. Interpréter la barre verticale d'une probabilité sans supposer l'indépendance des tokens.

Une probabilité mesure la plausibilité d'un événement dans une situation donnée. Pour conditionner sur un événement E de probabilité non nulle, on restreint l'univers aux cas où E se produit. La probabilité conditionnelle d'un événement A s'écrit $P(A | E)$, que l'on lit « probabilité de A sachant E ».

$$P(A | E) = \frac{P(A \cap E)}{P(E)}, \quad P(A \cap E) = P(E)P(A | E).$$

Le symbole $\cap$ signifie que les deux événements se produisent. La formule ne dit pas que A et E sont indépendants. L'indépendance serait le cas particulier où connaître E ne change pas la probabilité de A .

Imaginons vingt occurrences d'un préfixe dans un petit corpus. Douze sont suivies du caractère « a », cinq du caractère « e » et trois d'un espace. Les fréquences conditionnelles observées sont $12/20 = 0,6$, $5/20 = 0,25$ et $3/20 = 0,15$. Elles totalisent un parce que ces trois issues couvrent ici toutes les observations après ce préfixe.

À comprendre. Le contexte fait partie de la question.

Le modèle estime une distribution $P_\theta(s_{t+1} = v | s_0, \dots, s_t)$, où θ désigne ses paramètres appris et v un identifiant candidat. Changer le préfixe change potentiellement toute la distribution. Les paramètres partagés permettent de construire des estimations même pour des préfixes jamais vus exactement.

Dans `generate`, `idx_cond` garde les derniers tokens autorisés par `context_length`. Les logits de la dernière position servent à prévoir le prochain. Si le texte dépasse la capacité C , ce sont les derniers C tokens, et non l'histoire entière, qui sont fournis au modèle sans cache.

Une probabilité prédite n'est pas automatiquement une fréquence fiable dans le monde réel. Elle dépend du corpus, du modèle et de l'apprentissage. De même, conditionner sur un texte ne prouve aucune causalité entre événements du monde : il s'agit d'une relation de prédiction entre tokens.

Pour chaque contexte fixé, les probabilités de tous les tokens possibles doivent totaliser un. On ne somme pas les probabilités d'un même token sur des contextes différents pour effectuer cette vérification.

Exercice. Dans cent observations, quarante possèdent un préfixe E , et dix de ces quarante sont suivies du token A . Calculer $P(E)$, $P(A \cap E)$ et $P(A | E)$ à partir des fréquences. Peut-on en déduire $P(A)$?

Correction détaillée. On obtient $P(E) = 0,4$, $P(A \cap E) = 0,1$, puis $P(A | E) = 0,1/0,4 = 0,25$. On ne connaît pas les occurrences de A parmi les soixante autres observations ; sa probabilité globale reste donc indéterminée.

Fiche 15 — Loi catégorielle et espérance

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/07_generate.py

generate

Objectif. Distinguer un tirage de token, le choix du maximum et une moyenne probabiliste.

Le prochain token appartient à un ensemble fini de V catégories. Une loi catégorielle lui attribue les probabilités $p_0, \dots, p_{V-1}$, toutes positives ou nulles, dont la somme vaut un. Une réalisation de cette loi est un seul identifiant ; ce n'est pas le vecteur des probabilités lui-même.

Supposons trois tokens notés « a », « b » et espace, avec les probabilités $(0,5, 0,3, 0,2)$. On peut découper l'intervalle $[0, 1[$ en trois morceaux de longueurs correspondantes. Un nombre uniforme tombant dans le premier morceau choisit « a », dans le deuxième « b », et dans le troisième l'espace. C'est une façon intuitive de comprendre un tirage catégoriel.

Dans `generate`, `torch.multinomial(probs, num_samples=1)` réalise ce type de sélection pour chaque ligne. Malgré le nom de la fonction, avec un seul tirage par ligne on obtient ici un échantillon catégoriel. Au contraire, `torch.argmax` choisit systématiquement une catégorie de probabilité maximale.

À comprendre. L'espérance concerne une quantité numérique associée au résultat.

Si chaque token v reçoit une valeur $g(v)$, sa moyenne théorique sous la loi est :

$$\mathbb{E}[g(S)] = \sum_{v=0}^{V-1} p_v g(v).$$

Pour $g(a) = 1$, $g(b) = 2$ et $g(\text{espace}) = 0$, elle vaut $0,5 \times 1 + 0,3 \times 2 + 0,2 \times 0 = 1,1$. Cette valeur n'est pas un token ; elle résume les résultats de nombreux tirages.

Faire la moyenne brute des identifiants aurait rarement un sens linguistique : renuméroter le vocabulaire changerait cette moyenne sans changer le texte. En revanche, l'indicatrice d'un événement est utile : elle vaut un si l'événement arrive, zéro sinon. Son espérance est exactement la probabilité de l'événement.

Sur n répétitions indépendantes avec la même distribution, le nombre attendu de « a » vaut $0,5n$. Ce n'est pas une garantie de résultat exact. Pendant une génération réelle, la distribution change avec les nouveaux tokens ; on ne peut donc pas appliquer ce comptage comme si tout le texte provenait d'une unique loi fixe.

Exercice. Avec les probabilités $(0,5, 0,3, 0,2)$, quelle est la probabilité d'obtenir deux fois « b » en deux tirages indépendants à contexte fixé ? Quel résultat donnerait le choix du maximum ?

Correction détaillée. La probabilité vaut $0,3 \times 0,3 = 0,09$. Le choix du maximum donnerait deux fois « a » sans tirage. Confondre ces méthodes éliminerait la diversité que le paramètre `sample` permet précisément de conserver.

Fiche 16 — Chaîne probabiliste autorégressive

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/07_generate.py

generate

Objectif. Construire la probabilité d’une continuation à partir de prédictions locales.

Une génération autorégressive produit les tokens un par un. Après chaque choix, le token obtenu devient une partie du contexte du choix suivant. Le préfixe s’allonge donc au cours du calcul. Cela ne signifie pas que les nouveaux tokens sont indépendants ; leur dépendance au passé constitue au contraire le principe du modèle.

Pour un préfixe fixé c et une continuation de n tokens, la règle des probabilités conditionnelles donne :

$$P_{\theta}(s_1, \dots, s_n \mid c) = \prod_{t=1}^n P_{\theta}(s_t \mid c, s_1, \dots, s_{t-1}).$$

Le signe $\prod$, lettre grecque pi majuscule, abrège une multiplication répétée. Il joue pour le produit le rôle du sigma pour la somme. Au premier facteur, aucun token nouveau n’a encore été produit : on conditionne seulement sur c .

Supposons que la continuation « a », puis « b » ait les probabilités successives 0,6 et 0,25. Sa probabilité vaut $0,6 \times 0,25 = 0,15$. Le deuxième nombre doit être évalué après avoir ajouté « a ». Utiliser la probabilité de « b » avant cet ajout calculerait une autre quantité.

À comprendre. Choix local et meilleure suite sont deux problèmes distincts.

Imaginons deux premiers choix : « a » reçoit 0,6, « b » reçoit 0,4. Après « a », le meilleur second choix ne reçoit que 0,5, tandis qu’après « b », un second choix reçoit 0,9. Le meilleur prolongement de la branche « a » vaut 0,3, celui de « b » vaut 0,36. Choisir le maximum à chaque étape peut donc manquer la continuation globalement la plus probable.

Dans `generate`, `torch.cat([idx, next_token], dim=1)` réalise l’ajout au préfixe. La boucle recalcule ensuite une prédiction. Le contexte effectif reste borné par C . Le produit décrit toujours la loi de la procédure, mais ses facteurs utilisent alors les suffixes effectivement visibles.

Pour identifier cette loi à celle des logits bruts du modèle, on considère une température égale à un, sans filtrage. Modifier la température ou supprimer des candidats modifie les probabilités utilisées par la procédure de génération. Ces options seront détaillées après les fondements mathématiques.

Exercice. Trois probabilités conditionnelles d’une continuation valent $1/2$, $1/3$ et $3/4$. Calculer sa probabilité et sa surprise totale.

Correction détaillée. Le produit vaut $(1/2)(1/3)(3/4) = 1/8$. Sa surprise vaut $-\ln(1/8) = \ln 8 = 3 \ln 2 \approx 2,0794$. Les facteurs dépendent de préfixes successifs différents ; leur multiplication n’exige aucune hypothèse d’indépendance.

Fiche 17 — Softmax : définition et invariance par translation

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

softmax_manual

Objectif. Transformer des scores arbitraires en une distribution, puis démontrer une propriété utile.

Une liste de logits $z = (z_0, \dots, z_{V-1})$ peut contenir des valeurs négatives, positives ou supérieures à un. Pour obtenir une loi catégorielle, le softmax exponentie chaque score puis divise par la somme de toutes les exponentielles.

$$p_i = \frac{e^{z_i}}{\sum_{j=0}^{V-1} e^{z_j}}, \quad 0 \leq i < V.$$

Pour des scores finis, chaque numérateur est strictement positif et le dénominateur l'est aussi. En additionnant les p_i , on retrouve le dénominateur divisé par lui-même, donc un. La fonction est croissante dans l'ordre des scores : le plus grand logit reçoit la plus grande probabilité.

À comprendre. Seules les différences entre scores comptent.

Ajoutons le même réel c à tous les logits. La propriété de l'exponentielle permet de factoriser e^c dans chaque terme du dénominateur :

$$\frac{e^{z_i+c}}{\sum_j e^{z_j+c}} = \frac{e^c e^{z_i}}{e^c \sum_j e^{z_j}} = p_i.$$

Le facteur commun disparaît. Les scores $(1, 2, 3)$ et $(-2, -1, 0)$ définissent donc exactement la même distribution. Soustraire le maximum, comme le fait `softmax_manual`, est un cas particulier avec $c = -\max_j z_j$.

Le rapport de deux probabilités vaut $p_i/p_j = e^{z_i-z_j}$. Une différence de $\ln 2$ correspond donc à un rapport de probabilités égal à deux. Cela fournit une interprétation des écarts entre logits, mais pas de leur niveau absolu.

Ne confondons pas addition et multiplication : multiplier tous les logits par deux double leurs écarts et change généralement la distribution. Cette distinction expliquera l'effet de la température en génération. De même, augmenter un seul logit n'est pas une translation commune : sa probabilité augmente et celles des autres candidats diminuent.

Dans le dépôt, `dim=-1` précise l'axe de normalisation. Pour des logits $[B, T, V]$, le dernier axe est le vocabulaire. Dans les scores d'attention, il correspond aux positions consultées. La recette mathématique reste la même, mais les événements auxquels elle attribue des probabilités ne sont pas les mêmes.

Le dénominateur commun couple les candidats : modifier un score change également la normalisation des autres. Le softmax n'applique donc pas une conversion indépendante à chaque case.

Exercice. Les logits $(0, \ln 2)$ donnent quelles probabilités ? Que deviennent-elles après ajout de 7, puis après multiplication des logits initiaux par deux ?

Correction détaillée. Les exponentielles initiales valent $(1, 2)$, donc les probabilités sont $(1/3, 2/3)$. Ajouter 7 ne change rien. Multiplier les logits initiaux par deux donne les exponentielles $(1, 4)$, donc $(1/5, 4/5)$: le choix déjà favorisé devient plus dominant.

Fiche 18 — Softmax chiffré : du tableau aux probabilités

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/07_generate.py

generate

Objectif. Effectuer un softmax exact, contrôler sa somme et interpréter une sélection.

Prenons un vocabulaire fictif de trois tokens d'identifiants 0, 1 et 2. À la dernière position d'un préfixe, supposons les logits $z = (0, \ln 2, \ln 4)$. Ce choix permet un calcul exact, car les exponentielles valent des entiers :

$$e^z = (1, 2, 4), \quad \sum_i e^{z_i} = 7, \quad p = \left(\frac{1}{7}, \frac{2}{7}, \frac{4}{7}\right).$$

Les probabilités approchées sont $(0,142857, 0,285714, 0,571429)$. La somme exacte vaut un. Les valeurs décimales arrondies peuvent présenter un léger écart à un ; il faut distinguer cet effet d'affichage d'une erreur dans la normalisation.

Refaisons le calcul stabilisé en soustrayant $\ln 4$, le plus grand logit. Les scores deviennent $(-\ln 4, -\ln 2, 0)$, leurs exponentielles $(1/4, 1/2, 1)$, et leur somme $7/4$. Diviser chaque terme par $7/4$ redonne les mêmes fractions : $(1/4)/(7/4) = 1/7$, puis $2/7$ et $4/7$.

À comprendre. Une probabilité maximale n'est pas une certitude.

Le token 2 est le candidat le plus probable, mais sa probabilité reste seulement $4/7$. Le tirage peut donc retourner 0 ou 1. Le choix par maximum retourne ici 2 sans hasard. Le softmax calcule une distribution ; il ne choisit pas lui-même le token suivant.

Dans `generate`, `next_token_logits` est obtenu par `logits[:, -1, :]`. Si la forme initiale est `[1, 5, 3]`, cette sélection donne `[1, 3]` : une seule distribution sur trois candidats, celle qui suit les cinq tokens déjà fournis. Les distributions des autres positions ne sont pas additionnées à celle-ci.

La variable `probs` résulte ensuite de `torch.softmax`. Pour retrouver exactement notre exemple, on suppose une température égale à un et aucun filtrage. Si l'on supprimait le troisième candidat avant normalisation, les deux probabilités restantes deviendraient $1/3$ et $2/3$, et non $1/7$ et $2/7$. Une distribution doit répartir toute la masse entre les possibilités encore autorisées.

Pour contrôler un calcul, trois questions suffisent ici : les valeurs sont-elles non négatives, leur somme vaut-elle un, et le classement respecte-t-il celui des logits non masqués ? Ces contrôles ne remplacent pas le calcul exact.

Exercice. Le même calcul est effectué sur les logits $(\ln 3, \ln 3, 0)$. Donner les probabilités exactes, la somme des probabilités des deux premiers tokens et le résultat après soustraction du maximum.

Correction détaillée. Les exponentielles valent $(3, 3, 1)$, de somme 7 : on obtient $(3/7, 3/7, 1/7)$. Les deux premiers tokens réunis ont la probabilité $6/7$. Après soustraction de $\ln 3$, les exponentielles deviennent $(1, 1, 1/3)$, de somme $7/3$. Le quotient final ne change pas ; les deux premiers candidats restent ex æquo.

Fiche 19 — Log-vraisemblance et entropie croisée

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/05_model.py

MiniGPT.forward

Objectif. Comprendre la quantité scalaire que l'apprentissage cherche à diminuer.

Une cible connue est un identifiant y . Si le modèle lui attribue la probabilité p_y , sa vraisemblance pour cette observation est p_y . Maximiser cette valeur revient à maximiser $\ln p_y$, car le logarithme est croissant. Pour formuler un problème de minimisation, on prend l'opposé : la perte vaut $\ell = -\ln p_y$.

$$\ell = -z_y + \ln \left(\sum_{j=0}^{V-1} e^{z_j} \right), \quad \mathcal{J} = \frac{1}{BT} \sum_{b=0}^{B-1} \sum_{t=0}^{T-1} \ell_{b,t}.$$

La première formule remplace p_y par son expression softmax. Elle montre qu'on peut calculer directement la perte à partir des logits. Le logarithme de la somme d'exponentielles se calcule de façon stable en retirant leur maximum avant exponentiation.

À comprendre. Pourquoi parler d'entropie croisée ?

Représentons la cible par une distribution q qui vaut un pour le bon identifiant et zéro ailleurs. L'entropie croisée est $-\sum_j q_j \ln p_j$. Tous les termes s'annulent sauf celui de la cible : on retrouve $-\ln p_y$. On n'a pas besoin de fabriquer explicitement ce tableau de zéros et de un ; les cibles entières suffisent à PyTorch.

Plus généralement, une distribution q peut répartir la masse entre plusieurs issues. Son entropie propre est $-\sum_j q_j \ln q_j$, tandis que son entropie croisée avec la prédiction utilise $\ln p_j$. Les deux expressions coïncident lorsque $p = q$. Nous conservons ici les cibles entières effectivement utilisées dans le dépôt.

Pour $p = (1/7, 2/7, 4/7)$, une cible 2 entraîne $\ell = \ln(7/4) \approx 0,5596$. Une cible 0 entraîne $\ell = \ln 7 \approx 1,9459$. Le même tableau de scores est donc bon ou mauvais relativement à la réponse observée.

Dans `MiniGPT.forward`, `F.cross_entropy` reçoit les logits de forme $[BT, V]$ et les cibles de forme $[BT]$. Il ne faut pas appliquer un softmax au préalable : la fonction attend des scores bruts. La moyenne décrit les positions évaluées, qui ne sont pas nécessairement des observations indépendantes.

Les cibles proviennent du texte lui-même, grâce au décalage d'une position. Il n'est donc pas nécessaire qu'une personne annote chaque prochain caractère. Cette construction explique le terme d'apprentissage auto-supervisé : le corpus fournit à la fois les entrées et les réponses attendues.

Exercice. Deux cibles reçoivent les probabilités $1/2$ et $1/8$. Calculer la perte moyenne et la comparer à celle d'un prédicteur uniforme sur quatre tokens.

Correction détaillée. La perte moyenne vaut $(\ln 2 + \ln 8)/2 = 2 \ln 2 = \ln 4 \approx 1,3863$. Le prédicteur uniforme attribue $1/4$ à chaque cible et obtient aussi $\ln 4$. Des erreurs individuelles différentes peuvent donc conduire à la même perte moyenne.

Fiche 20 — Dérivée : mesurer une variation locale

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

train

Objectif. Retrouver l'idée de pente et comprendre ce que signifie un signal d'amélioration.

Pour une fonction réelle f , le quotient $(f(a+h) - f(a))/h$ mesure la variation moyenne entre deux entrées séparées par $h \neq 0$. Si ce quotient tend vers une valeur finie lorsque h tend vers zéro, cette limite est la dérivée $f'(a)$. Elle représente la pente de la tangente au point d'abscisse a .

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}, \quad f(a+h) \approx f(a) + hf'(a).$$

La deuxième relation est une approximation locale, pas une égalité pour n'importe quel déplacement. Si $f'(a) > 0$, un petit déplacement positif augmente la fonction ; un petit déplacement négatif la diminue. Une dérivée négative inverse ces indications.

À comprendre. Un calcul issu de la définition.

Pour $f(x) = x^2$, développer $(a+h)^2$ donne $a^2 + 2ah + h^2$. Le quotient de variation vaut donc $2a + h$, et sa limite vaut $2a$. Au point $a = 3$, la dérivée vaut 6. Un déplacement $h = 0,01$ prédit une hausse d'environ 0,06 ; la hausse exacte vaut $3,01^2 - 9 = 0,0601$.

Les règles utiles pour la suite sont : la dérivée d'une constante est zéro, celle de $ax + b$ est a , celle de e^x est e^x , et celle de $\ln x$, pour $x > 0$, est $1/x$. La dérivée d'une somme est la somme des dérivées. Ces quelques règles couvrent une grande partie des opérations de nos petits exemples.

Dans `train`, `loss.backward()` calcule des dérivées de la perte par rapport aux paramètres. Il ne remplace pas chaque poids par une infinité de petites perturbations : il utilise les règles de dérivation des opérations composées. Nous verrons ensuite comment cela se fait.

Une pente nulle ne prouve pas qu'un point est un minimum. Pour $f(x) = -x^2$, la dérivée en zéro est nulle, mais zéro est un maximum. L'information de premier ordre est locale et doit être interprétée avec prudence.

On peut estimer une dérivée en comparant deux valeurs proches, mais choisir un écart minuscule n'est pas toujours plus fiable sur ordinateur. Les arrondis peuvent masquer la différence entre les deux résultats. La dérivation des opérations évite cette soustraction fragile et fournit directement les sensibilités des calculs effectivement composés.

Exercice. Pour $f(w) = (w - 3)^2$, calculer $f'(1)$. Prévoir l'effet d'un déplacement de 1 vers 1,1, puis comparer avec les valeurs exactes.

Correction détaillée. On développe $f(w) = w^2 - 6w + 9$, donc $f'(w) = 2w - 6$ et $f'(1) = -4$. La variation prévue est $0,1 \times (-4) = -0,4$. La perte passe exactement de 4 à 3,61, soit une variation de $-0,39$: l'approximation indique correctement le sens de l'amélioration.

Fiche 21 — Règle de chaîne : dériver une composition

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/04_transformer.py

FeedForward.forward

Objectif. Propager une petite variation à travers plusieurs calculs successifs.

Une composition applique une fonction au résultat d'une autre. Si $u = g(x)$ puis $y = f(u)$, on écrit $y = f(g(x))$. Une petite variation de x modifie d'abord u , puis cette modification se répercute sur y . La règle de chaîne multiplie ces deux sensibilités locales :

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}, \quad (f \circ g)'(x) = f'(g(x))g'(x).$$

La notation ressemble à une simplification de fractions, mais elle exprime une règle de dérivation. Le point auquel chaque dérivée est évaluée compte : f' s'évalue à la valeur intermédiaire $g(x)$, pas automatiquement à x . Cette règle suppose que les fonctions rencontrées soient dérivables aux points utilisés pendant le calcul.

Considérons $u = 3x - 1$, puis $y = u^2$. On a $du/dx = 3$ et $dy/du = 2u$, donc $dy/dx = 6(3x - 1)$. À $x = 2$, l'intermédiaire vaut 5 et la dérivée finale vaut 30. Développer $y = 9x^2 - 6x + 1$ donne également $18x - 6 = 30$, ce qui vérifie le calcul par une autre voie.

À comprendre. Les variables intermédiaires rendent les calculs lisibles.

Au lieu de dériver une expression immense, on conserve les résultats de chaque opération. Pour $a = wx + b$, $h = \phi(a)$, $z = vh + c$, la sensibilité de z à w , les autres valeurs étant fixées, est $v\phi'(a)x$. On retrouve les facteurs du trajet $w \rightarrow a \rightarrow h \rightarrow z$.

`FeedForward.forward` applique une séquence : couche affine, GELU, couche affine, puis dropout. Les fonctions réellement utilisées ont leurs règles de dérivation. Notre carré u^2 est seulement un exemple facile à calculer, pas la définition de GELU.

Si plusieurs trajets relient une variable à la sortie, leurs contributions s'ajoutent. Pour $y = x^2 + x$, la branche carrée apporte $2x$ et la branche directe apporte 1. La dérivée totale est $2x + 1$. Cette idée éclairera plus tard les additions résiduelles du Transformer.

La variable par rapport à laquelle on dérive doit toujours être annoncée. Dans $a = wx + b$, dériver par rapport à x donne w , tandis que dériver par rapport à w donne x . Le biais est constant dans ces deux calculs, mais sa dérivée vaut un lorsqu'on dérive par rapport à b . Une même expression possède donc plusieurs sensibilités, selon la question posée.

Des produits répétés de petites sensibilités peuvent atténuer fortement une variation ; de grandes sensibilités peuvent l'amplifier. Ce constat arithmétique prépare l'étude des réseaux profonds, sans remplacer l'analyse de leurs valeurs réelles.

Exercice. Dériver $f(x) = \ln(1 + e^x)$ en nommant ses intermédiaires. Quelle est la dérivée en zéro ?

Correction détaillée. Posons $u = e^x$, $v = 1 + u$, puis $f = \ln v$. Les sensibilités sont $du/dx = e^x$, $dv/du = 1$ et $df/dv = 1/v$. Leur produit vaut $f'(x) = e^x/(1 + e^x)$. En zéro, $e^0 = 1$, donc $f'(0) = 1/2$.

Fiche 22 — Gradient : une dérivée pour chaque paramètre

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

train

Objectif. Étendre la pente d’une courbe à une perte dépendant de plusieurs nombres.

Un réseau possède de nombreux paramètres, regroupés symboliquement dans $\theta = (\theta_0, \dots, \theta_{n-1})$. Sa perte $\mathcal{J}(\theta)$ reste un seul nombre. La dérivée partielle $\partial\mathcal{J}/\partial\theta_i$ mesure l’effet local du paramètre i lorsque les autres sont maintenus fixes. Le symbole ∂ rappelle cette convention.

Le gradient rassemble toutes ces dérivées dans le même ordre que les paramètres. Il indique comment une petite modification simultanée se répercute au premier ordre :

$$\nabla\mathcal{J} = \left(\frac{\partial\mathcal{J}}{\partial\theta_0}, \dots, \frac{\partial\mathcal{J}}{\partial\theta_{n-1}} \right), \quad \Delta\mathcal{J} \approx \nabla\mathcal{J} \cdot \Delta\theta.$$

Nous retrouvons le produit scalaire : chaque variation est pondérée par sa sensibilité, puis toutes les contributions s’additionnent.

Pour $\mathcal{J}(a, b) = (a + 2b - 3)^2$, la règle de chaîne donne les dérivées $2(a + 2b - 3)$ et $4(a + 2b - 3)$. Au point $(0, 0)$, le gradient est $(-6, -12)$. Une perturbation $(0, 0, 1, 0)$ prévoit une baisse de 0,06 ; la perturbation $(0, 0, 0, 1)$ prévoit une baisse de 0,12. Les deux paramètres n’ont pas la même influence locale.

À comprendre. Le gradient des logits possède une forme particulièrement simple.

Pour la perte d’une cible y étudiée précédemment, la dérivation du logarithme de la somme donne :

$$\frac{\partial\ell}{\partial z_i} = p_i - \mathbf{1}_{i=y}.$$

L’indicatrice $\mathbf{1}_{i=y}$ vaut un pour la cible, zéro sinon. Le terme p_i vient de $e^{z_i} / \sum_j e^{z_j}$, et la dérivée de $-z_y$ fournit le terme négatif. Augmenter le logit correct diminue ainsi localement la perte, tandis qu’augmenter un concurrent l’augmente.

Dans `train`, `loss.backward()` propage ces sensibilités jusqu’aux paramètres. Leur attribut `grad` possède leur forme : une matrice de poids reçoit une matrice de dérivées. Pour une perte moyenne sur plusieurs positions, les contributions sont additionnées puis divisées par leur nombre. Les identifiants entiers ne sont pas les paramètres à ajuster.

Un poids partagé intervient à plusieurs positions. Sa dérivée totale additionne les contributions de toutes ces utilisations : on n’apprend pas une copie indépendante du poids pour chaque token. Cette accumulation explique aussi pourquoi les anciennes valeurs de `grad` doivent être effacées lorsqu’on commence une nouvelle mise à jour sans accumulation volontaire.

Exercice. Avec $p = (0, 2, 0, 5, 0, 3)$ et une cible d’identifiant 2, calculer le gradient par rapport aux logits, puis la somme de ses coordonnées.

Correction détaillée. Le gradient vaut $(0, 2, 0, 5, -0, 7)$, de somme zéro. Une translation commune de tous les logits a donc une variation de perte nulle au premier ordre. Cela concorde avec l’invariance exacte du softmax démontrée précédemment.

Fiche 23 — Rétropropagation : un petit réseau calculé entièrement

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

train

Objectif. Effectuer un passage avant et un passage arrière sans traiter la dérivation automatique comme une magie.

Construisons un réseau scalaire pédagogique. Son entrée est x , ses paramètres sont w, b, v, c , et ses deux logits sont $(z, 0)$. La cible est le premier token, d'identifiant zéro. La fonction carrée remplace volontairement GELU pour permettre un calcul intégral à la main ; l'entropie croisée reste celle étudiée pour le modèle.

$$a = wx + b, \quad h = a^2, \quad z = vh + c, \quad \ell = \ln(1 + e^{-z}).$$

La dernière formule vient de $-\ln(e^z/(e^z + 1))$. Le logit du deuxième token reste fixé à zéro : ce n'est pas un paramètre supplémentaire. Nous choisissons $x = 2, w = 1, b = -1, v = 2, c = -2$.

À comprendre. Passage avant : mémoriser les intermédiaires.

On calcule successivement $a = 1 \times 2 - 1 = 1$, $h = 1^2 = 1$, puis $z = 2 \times 1 - 2 = 0$. Les deux logits sont donc nuls, les probabilités valent $(1/2, 1/2)$ et la perte vaut $\ln 2 \approx 0,6931$. Ces valeurs intermédiaires serviront dans les dérivées.

Le passage arrière commence à la perte. La dérivée par rapport à z vaut $-1/(1 + e^z)$, donc $-1/2$ ici. Pour l'opération $z = vh + c$, on multiplie ce signal par la dérivée locale correspondant à chaque entrée : $\partial\ell/\partial v = (-1/2)h = -1/2$, $\partial\ell/\partial c = -1/2$, et $\partial\ell/\partial h = (-1/2)v = -1$.

À travers le carré, on obtient $\partial\ell/\partial a = (-1) \times 2a = -2$. La première couche affine reçoit alors ce signal :

$$\frac{\partial\ell}{\partial w} = -2x = -4, \quad \frac{\partial\ell}{\partial b} = -2, \quad \frac{\partial\ell}{\partial x} = -2w = -2.$$

Le gradient des quatre paramètres, dans l'ordre (w, b, v, c) , est donc $(-4, -2, -1/2, -1/2)$. La dérivée de l'entrée a été calculée pour suivre toute la chaîne, mais l'entrée observée ne doit pas être modifiée pour apprendre les paramètres.

Dans `train`, `loss.backward()` automatise ce parcours inverse pour le vrai graphe du MiniGPT. Chaque opération transmet une sensibilité locale ; les chemins partagés additionnent leurs contributions. Le passage arrière ne change pas encore les paramètres : cette responsabilité appartient à l'optimiseur.

La valeur initiale du signal arrière est un : la dérivée de la perte par rapport à elle-même vaut un. Chaque étape conserve ensuite les paramètres aux valeurs utilisées pendant le passage avant. Si l'on changeait un poids au milieu de ce parcours, les dérivées locales ne décriraient plus toutes le même réseau et le gradient obtenu perdrait sa cohérence.

Exercice. En ne changeant que c , appliquer un déplacement opposé au gradient de taille 0,1 fois sa valeur. Recalculer le logit et la perte.

Correction détaillée. Le nouveau biais vaut $-2 - 0,1(-1/2) = -1,95$. Le logit devient $z = 2 - 1,95 = 0,05$. La perte vaut $\ln(1 + e^{-0,05}) \approx 0,6685$, inférieure à $\ln 2$, conformément au signe du gradient.

Fiche 24 — Descente de gradient et synthèse des fondements

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

train

Objectif. Relier la prédiction, la perte et les dérivées à une règle élémentaire d'apprentissage.

Une fois le gradient connu, comment modifier les paramètres ? La descente de gradient élémentaire avance dans la direction opposée au gradient. Le taux d'apprentissage $\eta > 0$ règle la taille du déplacement :

$$\theta_{\text{nouveau}} = \theta_{\text{ancien}} - \eta \nabla \mathcal{J}(\theta_{\text{ancien}}).$$

Tous les composants utilisent les dérivées évaluées aux anciennes valeurs. Les remplacer successivement en recalculant une partie du gradient définirait une autre procédure. Un paramètre dont la dérivée est négative augmente ; soustraire un nombre négatif revient à l'ajouter.

À comprendre. Pourquoi cette direction peut-elle améliorer la perte ?

L'approximation locale donne, pour le déplacement $\Delta\theta = -\eta \nabla \mathcal{J}$, une variation d'environ $-\eta \|\nabla \mathcal{J}\|^2$. Elle est négative si le gradient est non nul. Le raisonnement reste local : un pas trop grand peut sortir de la région où cette approximation est pertinente.

Sur $\mathcal{J}(w) = (w - 3)^2$, partons de $w = 1$. Le gradient vaut -4 . Avec $\eta = 0,1$, on obtient $w_{\text{nouveau}} = 1,4$ et une perte 2,56, contre 4 auparavant. Avec $\eta = 1,5$, on obtient 7 et une perte 16 : suivre la bonne direction initiale n'empêche pas de dépasser une région favorable.

Dans `train`, le programme emploie `torch.optim.AdamW`, non cette descente élémentaire. AdamW adapte ses mises à jour à l'historique des gradients et applique une décroissance des poids. La formule précédente explique l'intuition, mais ne reproduit pas ses étapes exactes. Les détails de l'entraînement seront développés dans les fiches 73 à 96.

L'ordre effectif du code est important : le passage avant calcule les logits et la perte, `optimizer.zero_grad(set_to_none=True)` efface les anciennes dérivées, `loss.backward()` calcule les nouvelles, puis `optimizer.step()` modifie les paramètres. Le calcul des gradients et leur application constituent deux opérations distinctes.

Nous disposons désormais de la chaîne mathématique essentielle : les indices désignent les données ; les vecteurs et matrices les représentent ; les fonctions composées donnent des logits ; le softmax fournit des probabilités ; l'entropie croisée mesure leur accord avec les cibles ; les dérivées indiquent comment modifier les paramètres. Les fiches suivantes pourront donc préciser les données et les représentations avant l'attention, réservée aux fiches 49 à 72.

Exercice. Une perte a pour gradient $(2, -3)$ au point $(1, 1)$. Avec $\eta = 0,05$, calculer le nouveau point et la variation de perte prédite au premier ordre.

Correction détaillée. Le déplacement vaut $(-0,1, 0,15)$, donc le nouveau point est $(0,9, 1,15)$. La variation prévue est $2(-0,1) + (-3)(0,15) = -0,65$. C'est une prévision locale, pas une garantie que la perte réelle diminue exactement de cette quantité.

Fiche 25 — Se repérer : scripts, imports et notebook

```
/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/02_dataset.py
```

```
load_local_module
```

Objectif. Identifier le chemin réellement exécuté avant d’expliquer les données.

Le dépôt propose une progression lisible : tokenisation, jeux de données, attention, blocs Transformer, modèle, entraînement, génération. Cette progression est pédagogique, mais elle n’impose pas que chaque programme utilise exactement la démonstration du précédent. Pour suivre une expérience, partons de son point d’entrée, puis descendons vers les fonctions appelées. Le répertoire de référence est `/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt`.

À comprendre. Un fichier Python n’est pas toujours importable par une instruction ordinaire.

Les noms numérotés commencent par des chiffres : écrire une instruction d’importation avec ce nom comme identifiant Python serait invalide. La fonction étudiée construit donc un chemin depuis `BASE_DIR`, obtient une spécification avec `importlib.util.spec_from_file_location`, crée un objet module, puis exécute son contenu avec `spec.loader.exec_module`. Le nom logique donné au module ne doit pas être confondu avec son nom de fichier. Ce mécanisme charge les définitions, et exécute aussi les instructions placées au niveau supérieur.

La garde `if __name__ == '__main__':` empêche en revanche de lancer automatiquement la démonstration lorsque le fichier est chargé comme module. Ainsi, récupérer `NextTokenDataset` ne signifie pas afficher les fenêtres de son `main`. Le chemin est calculé relativement à `__file__`, non relativement au dossier depuis lequel on lance Python.

À comprendre. Deux parcours de données coexistent.

La démonstration `main` du script de données lit tout le petit texte, apprend un tokenizer caractère et emploie `NextTokenDataset`. L’entraînement actif, défini par `train` dans `/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py`, appelle `prepare_corpus`, puis `MemmapTokenDataset`. La fonction historique `create_data loaders` existe encore, mais ce n’est pas le chemin appelé par `train`. Lire seulement son découpage à 90 % donnerait donc une description incomplète du pipeline actuel.

Le notebook `/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/mini_gpt_pedagogique.ipynb` sert à explorer interactivement. L’ordre d’exécution des cellules peut laisser des objets anciens en mémoire ; un script relancé repart dans un nouveau processus. Le guide `/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/README.md` décrit ces deux usages sans remplacer la lecture des appels effectifs.

Exercice. Un lecteur affirme : « L’entraînement mélange une liste de tous les IDs avec la fonction historique. » Comment vérifier cette affirmation ?

Correction détaillée. Suivre `main`, puis `train` dans le script d’entraînement. On rencontre la préparation des fichiers, les datasets `memmap` et un `RandomSampler` avec remise. La fonction historique n’est pas appelée. Il faut distinguer une définition présente dans le fichier d’une fonction effectivement exécutée : la proximité visuelle n’établit aucune relation d’appel.

Fiche 26 — Texte Unicode, octets et vocabulaire

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/01_tokenizer.py

train_tokenizer

Objectif. Distinguer ce que l'on voit, ce que Python compte et ce que le disque stocke.

Un texte Python de type `str` est une suite de points de code Unicode. Un fichier UTF-8 est une suite d'octets. Un octet possède 256 valeurs possibles ; un point de code peut demander plusieurs octets lors de son encodage UTF-8. Ni l'un ni l'autre ne correspond toujours à une lettre visuelle complète : un accent combinant ou plusieurs signes composant un emoji peuvent participer à un seul graphème perçu par le lecteur.

À comprendre. La longueur dépend de l'unité choisie.

Le texte composé de « A », « é » et du caractère U+1F642 contient trois points de code. En UTF-8, leurs tailles respectives sont un, deux et quatre octets. On obtient donc :

$$N_{\text{caractères}} = 3, \quad N_{\text{octets}} = 1 + 2 + 4 = 7.$$

Le tokenizer caractère produit ici trois IDs, mais leur stockage ultérieur en entiers de huit octets occupera vingt-quatre octets, hors métadonnées. Transformer du texte en nombres n'est donc pas automatiquement le compresser. La tokenisation prépare une interface pour le modèle ; sa taille disque dépend également du type numérique choisi.

À comprendre. Aucune normalisation Unicode n'est appliquée.

Un « é » précomposé est un seul point de code ; un « e » suivi d'un accent aigu combinant en comporte deux. Ces formes peuvent sembler identiques à l'écran, mais `set(text)` et `encode` les traitent différemment. Le code ne les convertit pas vers une forme commune, ne met pas le texte en minuscules et ne retire pas les espaces. Une politique de nettoyage serait une décision supplémentaire, absente de cette implémentation.

Le vocabulaire est une association entre unités et entiers, pas un dictionnaire de définitions linguistiques. Sa taille est notée V . Un ID valide appartient à $\{0, \dots, V - 1\}$. L'ID d'une lettre n'exprime ni sa fréquence ni sa proximité avec une autre lettre : ces entiers désigneront des lignes d'une table d'embeddings. La dimension D de cette table sera une autre quantité, étudiée ensuite.

Point de vigilance. Dans ce chapitre, « caractère » désigne le point de code manipulé par Python, sauf mention explicite d'un graphème.

Cette convention rend les calculs reproductibles, notamment pour le découpage du corpus. Compter les octets du fichier ne suffit pas à retrouver la frontière de validation calculée en caractères.

Exercice. Comparer « é » et la séquence `e` suivie de U+0301 : combien de caractères et d'octets UTF-8 chacune contient-elle ?

Correction détaillée. La première contient un caractère et deux octets. La seconde contient deux caractères et trois octets : un pour `e`, deux pour l'accent. Sans normalisation, le tokenizer caractère peut donc produire une ou deux positions pour deux affichages visuellement voisins.

Fiche 27 — Construire le tokenizer caractère

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/01_tokenizer.py

SimpleTokenizer

Objectif. Reconstruire exactement le vocabulaire, y compris l'entrée inconnue.

Le constructeur accepte un itérable de chaînes. Dans l'usage caractère, cet itérable fournit les caractères du texte ou l'alphabet déjà collecté. La première opération est `sorted(set(vocab) - {'<unk>'})`. L'ensemble élimine les répétitions ; le tri fixe un ordre indépendant de l'ordre de découverte. Pour des caractères isolés, ce tri suit l'ordre Unicode, et non l'ordre alphabétique d'un dictionnaire français.

À comprendre. Deux tables réciproques rendent les conversions explicites.

La liste `itos` associe un indice à un token. Le dictionnaire `stoi` associe un token à son indice. Avant les caractères triés, le constructeur ajoute l'entrée spéciale `<unk>`. Elle reçoit donc toujours l'ID zéro, enregistré dans `unk_id`. Cette entrée représente une information inconnue, pas une fin de phrase, un remplissage de batch ou un début de document.

Prenons le texte original « baba é ». Son alphabet comporte une espace, `a`, `b` et `é`. La liste construite est :

$$\text{itos} = [\text{<unk>}, \text{espace}, a, b, \text{é}], \quad V = 5.$$

Le `b` reçoit l'ID 3 même s'il apparaît en premier. Le mot « baba » s'encode donc en 3, 2, 3, 2. Répéter ce texte mille fois ne change pas le vocabulaire : les fréquences seront utiles au modèle, mais elles n'interviennent pas dans cette construction particulière.

À comprendre. Apprentissage par flux et apprentissage depuis une chaîne.

`from_text` transmet directement le texte au constructeur. `train_tokenizer`, en mode `char`, parcourt au contraire les blocs successifs et enrichit un ensemble `alphabet`. Il refuse un alphabet vide, puis construit le même type de tokenizer. L'ordre des blocs ne change pas les IDs si leur ensemble de caractères reste identique. Le paramètre `vocab_size` est ignoré dans cette branche : on ne tronque pas l'alphabet à une taille demandée.

Point de vigilance. Le marqueur spécial est une chaîne entière.

Lorsque le corpus contient littéralement les cinq caractères de `<unk>`, le mode caractère les parcourt séparément. Ils ne sont pas reconnus comme une occurrence magique du token spécial. Le retrait dans le constructeur évite surtout de dupliquer ce marqueur si un vocabulaire fourni le contient déjà comme entrée.

Exercice. Construire le vocabulaire de « cabac », puis encoder « bac ». Que change une demande de vocabulaire de taille 2 en mode caractère ?

Correction détaillée. On obtient `['<unk>', 'a', 'b', 'c']`, donc $V = 4$, et « bac » devient `[2, 1, 3]`. La taille demandée ne change rien : les trois caractères observés et l'entrée inconnue sont conservés.

Fiche 28 — Encoder, décoder et perdre une information

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/01_tokenizer.py

SimpleTokenizer

Objectif. Savoir quand l’aller-retour restitue le texte et quand il ne le peut pas.

Encoder consiste à parcourir le texte caractère par caractère. Pour chacun, `stoi.get(char, unk_id)` fournit soit son ID connu, soit zéro. Décoder parcourt les IDs, cherche les tokens correspondants, puis concatène leurs représentations. Dans cette dernière opération, l’entrée spéciale `<unk>` est rendue par un point d’interrogation.

À comprendre. L’aller-retour est exact sur l’alphabet connu.

Construisons le tokenizer avec le texte original « ab a ». Sa table est `['<unk>', ' ', 'a', 'b']`. L’encodage de « ba a » donne `[3, 2, 1, 2]`, dont le décodage redonne exactement « ba a ». L’espace fait partie des données au même titre que les lettres. Il n’est ni supprimé ni reconstruit par une règle grammaticale.

$$\text{decode}(\text{encode}(s)) = s \quad \text{si tous les caractères de } s \text{ sont connus.}$$

En revanche, « bz » devient `[3, 0]`, puis « b? ». Une chaîne « bΩ » suit le même trajet. Deux entrées distinctes ayant la même image, aucune fonction de décodage ne peut déterminer quel caractère inconnu était présent. Cette perte n’est pas une erreur du réseau : elle se produit avant son intervention. Elle illustre une application non injective, notion mathématique utile pour comprendre les limites d’une représentation.

À comprendre. Les méthodes d’inspection ne rendent pas toutes la même chaîne.

`id_to_token(0)` retourne le marqueur `<unk>`, tandis que `decode([0])` retourne `?`. De même, un entier négatif ou supérieur au dernier indice valide est converti en marqueur inconnu lors du décodage. Le test de borne empêche notamment qu’un indice négatif sélectionne le dernier élément, comme le ferait habituellement une liste Python.

Point de vigilance. Un vrai point d’interrogation peut lui-même appartenir au vocabulaire.

L’affichage décodé ne permet alors pas de distinguer ce caractère d’un inconnu remplacé. Pour mesurer les inconnus, il faut compter `unk_id` dans les IDs, pas compter les points d’interrogation du texte reconstruit. Le taux obtenu décrit la couverture de l’alphabet d’entraînement. Sur une validation contenant une nouvelle écriture, il peut être élevé, même si le programme fonctionne exactement comme prévu.

Exercice. Avec le vocabulaire précédent, décoder `[2, -1, 99, 3]`, puis donner le résultat de `id_to_token(-1)`. Peut-on retrouver les inconnus ?

Correction détaillée. Le texte est « a ??b ». La méthode d’inspection retourne `<unk>`. Les deux IDs invalides ne révèlent aucun caractère initial ; le décodage fournit une représentation de remplacement, pas une réparation. Il faut conserver le texte brut si cette information doit rester accessible.

Fiche 29 — Sauvegarder l'état caractère sans réapprendre

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/01_tokenizer.py

tokenizer_from_state

Objectif. Comprendre pourquoi les poids du modèle ne suffisent pas à interpréter ses IDs.

Une ligne d'embedding correspond à un ID précis. Si l'on recharge les poids avec un vocabulaire différent, les mêmes nombres peuvent désigner d'autres caractères. Un modèle ayant associé la ligne 2 à **a** ne reconnaît pas spontanément que cette ligne désigne désormais **b**. Sauvegarder l'état du tokenizer fait donc partie de la sauvegarde du sens des données, et pas seulement du confort de programmation.

À comprendre. Un état compact suffit pour le mode caractère.

`to_state` renvoie un dictionnaire comportant `kind`: `'char'` et la liste `vocab`, égale à `itos[1:]`. Le marqueur inconnu est omis parce que le constructeur le réintroduira en position zéro. La restauration appelle `SimpleTokenizer(state['vocab'])`, sans relire le corpus ni réapprendre quoi que ce soit. Pour un état produit par ce constructeur, le tri reconstruit exactement les mêmes tables.

```
state = tokenizer.to_state()
restored = tokenizer_from_state(state)
assert restored.encode("aba") == tokenizer.encode("aba")
```

Ce court exemple suppose les objets déjà chargés. L'égalité concerne des IDs, ce qui est plus exigeant que la seule égalité de taille du vocabulaire. Deux vocabulaires peuvent avoir dix entrées et pourtant attribuer des significations différentes à chacun des dix indices.

Dans `prepare_corpus`, l'état est écrit en JSON UTF-8 avec `ensure_ascii=False`. Les accents peuvent ainsi rester lisibles dans le fichier `/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/checkpoints/tokens/tokenizer.json` pour la sortie par défaut. Une sérialisation JSON avec échappements Unicode aurait le même sens après lecture; ce choix concerne surtout la lisibilité. Dans `train`, le checkpoint conserve également `tokenizer.to_state()`, ainsi qu'un champ `vocab` historique pour le mode caractère.

Point de vigilance. Un état enregistré n'est pas une validation complète de son contenu.

Le restaurateur contrôle le type annoncé par `kind`, mais ne transforme pas un dictionnaire arbitraire en état fiable. Il faut garder ensemble le bon tokenizer, les fichiers d'IDs et les poids correspondants. Modifier manuellement le vocabulaire après préparation peut rendre les fichiers sémantiquement incohérents sans changer leur longueur en octets.

Exercice. Un état possède `vocab=['a', 'c']`. Quelle table sera restaurée? Que se passe-t-il si l'on ajoute `'b'` avant de recharger les mêmes IDs?

Correction détaillée. La table initiale est `['<unk>', 'a', 'c']`. Après ajout et tri, elle devient `['<unk>', 'a', 'b', 'c']`. L'ancien ID 2, qui signifiait `c`, signifie alors `b`. L'absence d'erreur informatique ne garantit donc pas la cohérence du modèle.

Fiche 30 — BPE : apprendre des fusions, à la main

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/01_tokenizer.py

train_tokenizer

Objectif. Comprendre le principe des paires fréquentes avant les détails de la bibliothèque.

Le mode caractère impose une position par caractère. BPE cherche à représenter des suites fréquentes par des unités plus longues. Son nom, *Byte Pair Encoding*, rappelle une méthode fondée sur le regroupement de paires. Pour isoler cette idée, travaillons d’abord sur une version jouet dont les symboles initiaux sont des lettres. La version byte-level du dépôt sera précisée à la fiche suivante.

À comprendre. Une fusion remplace deux symboles adjacents par un nouveau symbole.

Considérons trois occurrences séparées du mot inventé « baba ». Chaque occurrence commence comme $[b, a, b, a]$. Au total, la paire (b, a) apparaît six fois et la paire (a, b) trois fois. On choisit donc (b, a) , puis chaque occurrence devient $[ba, ba]$. La paire (ba, ba) apparaît maintenant trois fois : une seconde fusion produit le symbole $[baba]$.

$$[b, a, b, a] \longrightarrow [ba, ba] \longrightarrow [baba].$$

Les frontières entre occurrences sont ici imposées : on ne compte pas de paire reliant la fin d’une occurrence au début de la suivante. Cette hypothèse explicite évite d’attribuer au calcul des paires inexistantes dans l’exemple.

La taille de la séquence diminue, tandis que le vocabulaire gagne des entrées. Les anciens symboles restent utiles pour des mots différents. Si l’on rencontre « bac », la fusion apprise (b, a) permet $[ba, c]$, mais le symbole **baba** ne s’applique pas à toute la chaîne. Apprendre BPE ne revient donc pas à enregistrer seulement une liste de mots.

À comprendre. L’ordre des fusions fait partie de l’apprentissage.

Lorsqu’on ajoute une fusion, les comptages pertinents évoluent. Des égalités de fréquence peuvent nécessiter une règle de départage. Notre exemple possède des maxima simples ; il ne prétend pas prédire les IDs ni toutes les décisions internes de la bibliothèque. Le code du dépôt délègue ces mécanismes à **tokenizers**, au lieu de réimplémenter les compteurs et leurs mises à jour.

Point de vigilance. La compression n’est pas un score de compréhension.

Un symbole long très fréquent peut économiser des positions sans porter un sens linguistique stable. Le Transformer apprendra ensuite à prédire les unités obtenues. Changer de tokenizer change aussi l’unité de sa perte.

Exercice. Avec deux occurrences de « abac », compter les paires initiales. Si l’on impose de fusionner (a, b) , quelle représentation obtient-on ?

Correction détaillée. Les paires (a, b) , (b, a) et (a, c) apparaissent chacune deux fois : il y a égalité, donc aucune fusion unique n’est déductible des seules fréquences. Avec le choix imposé, chaque occurrence devient $[ab, a, c]$, soit trois tokens au lieu de quatre.

Fiche 31 — BPE byte-level : partir des 256 octets

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/01_tokenizer.py

BPETokenizer

Objectif. Expliquer pourquoi un caractère absent de l'apprentissage reste représentable.

Dans le dépôt, le BPE n'apprend pas seulement sur l'alphabet des lettres rencontrées. Son prétraitement est `ByteLevel(add_prefix_space=False)` et son alphabet initial contient les 256 valeurs d'octet. La bibliothèque représente ces octets par des symboles internes ; les tokens affichés ne doivent donc pas toujours être lus comme des fragments ordinaires de texte Unicode.

À comprendre. Couverture des octets et couverture des caractères sont différentes.

Un emoji absent du texte d'entraînement possède néanmoins une représentation UTF-8 constituée d'octets. Ces octets sont couverts par l'alphabet initial, même lorsqu'aucune fusion apprise ne les regroupe. L'encodage peut être plus long que pour un motif fréquent, mais le décodage byte-level peut reconstituer le caractère. C'est une différence fondamentale avec le remplacement par `unk_id` du tokenizer caractère.

Le formateur BPE conserve aussi `<unk>` comme token spécial. Le vocabulaire dispose donc d'un plancher :

$$V \geq 256 + 1 = 257.$$

Demander `vocab_size=100` ne supprime pas les octets nécessaires : la taille réelle peut dépasser la demande. Inversement, demander une grande taille ne garantit pas assez de motifs utiles pour l'atteindre. Le modèle doit utiliser `tokenizer.vocab_size`, propriété effective du backend, et non supposer que la valeur demandée est la taille obtenue.

À comprendre. L'absence de normalisation est un choix concret.

Aucun normaliseur n'est installé. L'option de préfixe est désactivée : on n'ajoute pas une espace artificielle au début du texte. Le décodeur `ByteLevel` accompagne le pré-tokenizer. L'encodage utilise `add_special_tokens=False` et le décodage `skip_special_tokens=False`. Ces réglages préservent notamment le traitement explicite du marqueur littéral étudié par les tests. Cela ne promet pas qu'une suite arbitraire d'IDs produite par un modèle forme toujours des octets UTF-8 valides.

Point de vigilance. La dépendance BPE est facultative, pas implicite.

`_bpe_library` importe `tokenizers` seulement à la demande. Le parcours caractère n'en a pas besoin. Son installation est décrite dans `/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/requirements-bpe.txt`. La restauration d'un état BPE exige elle aussi cette bibliothèque, même si aucun nouvel apprentissage du tokenizer n'est prévu.

Exercice. Un tokenizer BPE appris sur des lettres latines reçoit « Ω ». Faut-il ajouter obligatoirement cette lettre au vocabulaire ? Que représente la borne 257 ?

Correction détaillée. Non : les octets UTF-8 de « Ω » sont déjà représentables. Une fusion spécifique pourrait raccourcir sa représentation, sans être nécessaire à sa couverture. La borne compte 256 symboles d'octet et le token spécial ; elle ne compte pas 257 caractères Unicode différents.

Fiche 32 — Le budget d'apprentissage BPE et son biais

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/01_tokenizer.py

_bpe_training_chunks

Objectif. Séparer le texte utilisé pour apprendre le tokenizer du texte utilisé pour entraîner le réseau.

Le BPE pourrait accumuler beaucoup de statistiques si on lui soumettait sans limite un corpus immense. Le dépôt borne donc son apprentissage : `BPE_TRAINING_CHAR_LIMIT` vaut un million de caractères et `BPE_CHUNK_SIZE` vaut 4096. Ces unités sont des caractères Python, pas des octets du fichier ni des tokens déjà calculés. Ce choix limite les données confiées au formateur de la bibliothèque.

À comprendre. Le générateur découpe et décrémente un budget.

Il parcourt les chaînes reçues dans leur ordre, les fractionne si nécessaire, soustrait chaque longueur à `remaining`, puis s'arrête à zéro. Les chaînes vides ne produisent pas de bloc. La borne concerne le préfixe cumulé de l'itérable : elle n'est pas renouvelée pour chaque chaîne. Un corpus plus court est naturellement consommé en entier, sans remplissage artificiel.

Avec un budget miniature de sept caractères et une taille de bloc de trois, les chaînes `["", 'abcd', 'efghijk', 'unused']` produisent `['abc', 'd', 'efg']`. Le bloc `'d'` reste séparé du suivant : le générateur ne rassemble pas les fins de chaînes pour remplir chaque bloc. Cet exemple, également présent comme cas de test, rend visibles à la fois la borne totale et les frontières d'entrée.

À comprendre. Apprendre un code et apprendre à prédire sont deux opérations.

Après ce budget d'apprentissage BPE, `prepare_corpus` encode tout le texte d'entraînement. Les passages situés après le premier million de caractères ne sont pas abandonnés par le réseau. Ils utilisent simplement des fusions choisies sur un sous-ensemble initial.

$$N_{\text{apprentissage BPE}} \leq \min(N_{\text{train}}, 1\,000\,000).$$

Les octets garantissent leur représentabilité, mais pas une segmentation aussi compacte que celle des thèmes rencontrés au début.

Point de vigilance. Un préfixe borné n'est pas un échantillon aléatoire représentatif.

Si un corpus commence par un long catalogue puis continue par des dialogues, les fusions peuvent privilégier les références du catalogue. Changer la graine du réseau ne corrige pas ce biais de sélection. Il faut réfléchir à l'organisation du texte avant la préparation, sans y faire entrer la validation pour améliorer artificiellement sa couverture. La borne contrôle une dépense de calcul et de mémoire ; elle ne garantit ni équilibre linguistique ni qualité des unités.

Exercice. Pour 1 200 000 caractères d'entraînement, combien sont au maximum vus par le formateur BPE ? Combien sont ensuite proposés à l'encodage ?

Correction détaillée. Le formateur voit au maximum 1 000 000 caractères. L'encodage traite les 1 200 000 caractères. Les 200 000 derniers utilisent le vocabulaire déjà fixé ; ils ne déclenchent pas de nouvelles fusions.

Fiche 33 — État BPE et frontières de tokenisation

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/01_tokenizer.py

BPETokenizer

Objectif. Comprendre ce qui est sauvegardé et pourquoi deux encodages du même texte peuvent différer.

Pour BPE, une simple liste de caractères serait insuffisante. Il faut conserver le vocabulaire appris, les règles de fusion et la configuration du traitement des octets. `BPETokenizer.to_state` renvoie donc `kind: 'bpe'` et `tokenizer_json`, une sérialisation complète du backend. `tokenizer_from_state` la recharge avec `Tokenizer.from_str`, sans recommencer l'apprentissage.

À comprendre. Restaurer le tokenizer ne restaure pas automatiquement toutes les conditions d'encodage.

L'état décrit le backend. La taille des blocs de lecture utilisés par `prepare_corpus` appartient, elle, au code de préparation. Pour reproduire exactement les fichiers d'IDs, il faut conserver le texte, son découpage brut, le tokenizer et la stratégie de blocs. L'égalité des textes décodés est un critère moins strict que l'égalité des séquences d'entiers.

Imaginons, uniquement pour raisonner, qu'une fusion apprise représente **ab** par un seul token. Encoder **ab** d'un seul tenant peut produire un ID. Encoder **a**, puis **b** dans deux appels séparés ne peut pas appliquer cette fusion à travers les appels. Les deux listes concaténées contiennent alors deux IDs.

$$E(uv) \neq E(u) \parallel E(v) \quad \text{peut se produire avec BPE.}$$

Le symbole $\parallel$ désigne la concaténation de listes, non une addition. Dans les deux cas, le décodage des IDs issus du texte peut reconstituer **ab**. Une représentation différente n'implique donc pas nécessairement une information textuelle perdue.

À comprendre. Deux types de frontières interviennent dans le pipeline.

Les blocs d'apprentissage interdisent certaines fusions lors de leur acquisition. Les blocs d'encodage interdisent ensuite d'appliquer une fusion à cheval sur deux appels. Enfin, la frontière entre entraînement et validation est respectée séparément : aucun appel ne fusionne un morceau de train avec un morceau de validation.

Point de vigilance. Une frontière de tokenisation n'est pas une réinitialisation du modèle.

Les IDs des blocs successifs d'un même split sont écrits à la suite. Les fenêtres du dataset pourront donc traverser ces anciennes frontières. Aucun token séparateur n'est ajouté pour les signaler. Seule la séparation entre les deux fichiers interdit une fenêtre mélangeant train et validation.

Exercice. Deux préparations emploient le même état BPE mais des blocs de 4096 et 2048 caractères. Doivent-elles avoir les mêmes tailles binaires ?

Correction détaillée. Non. Les frontières supplémentaires peuvent empêcher des fusions et changer le nombre d'IDs. Il faut mesurer ce nombre après encodage. Même si les deux textes reconstruits sont identiques, les fenêtres en tokens et les quantités de contexte textuel qu'elles couvrent peuvent différer.

Fiche 34 — Autorégression : une entrée, sa cible décalée

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/02_dataset.py

NextTokenDataset

Objectif. Transformer une suite d'IDs en questions de prédiction sans créer d'étiquettes manuelles.

L'autorégression apprend à prédire le prochain élément à partir des précédents. Dans un texte, chaque token observé peut devenir la cible du token situé juste avant lui. Les données fournissent ainsi leurs propres réponses : on parle d'apprentissage auto-supervisé. Il ne s'agit pas de demander à un humain de classer chaque phrase, mais de réorganiser un texte déjà disponible.

À comprendre. Une fenêtre de longueur T demande $T + 1$ tokens sources.

Notons $z_0, \dots, z_{N-1}$ les IDs du split. Pour un départ i , on prépare :

$$X_t = z_{i+t}, \quad Y_t = z_{i+t+1}, \quad 0 \leq t < T.$$

La longueur maximale de contexte du modèle est notée C . Les datasets de ce dépôt construisent des fenêtres fixes de taille $T = C$, même si la notation distingue utilement longueur courante et capacité maximale. Ni T ni C n'est la taille V du vocabulaire.

Prenons la suite artificielle $[4, 7, 2, 9, 5]$ et $T = 3$. Au départ zéro, $X = [4, 7, 2]$ et $Y = [7, 2, 9]$. La première position doit prévoir 7 après avoir reçu 4. La deuxième peut utiliser 4 et 7 pour prévoir 2. La troisième peut utiliser 4, 7 et 2 pour prévoir 9. Le token 5 n'intervient pas dans cette première fenêtre, mais apparaîtra dans la cible de la fenêtre suivante.

À comprendre. Le décalage seul ne garantit pas la causalité.

Les cibles sont justes, mais un réseau sans masque pourrait regarder X_{t+1} , qui est précisément Y_t . Il disposerait de la réponse à l'intérieur de l'entrée. Le masque causal étudié dans le chapitre Transformer empêche cette consultation du futur. Ici, nous préparons les bonnes paires ; le modèle devra encore respecter la bonne circulation d'information.

Point de vigilance. On ne tire pas de nouveaux tokens pour construire Y .

Les cibles viennent du corpus observé. Pendant l'entraînement, les entrées sont également les vrais tokens du texte, pas les prédictions précédentes du réseau. La génération fonctionnera autrement : le modèle réinjectera ses propres sorties lorsqu'aucun texte futur n'est fourni. Cette différence explique pourquoi une erreur générée peut modifier tout le contexte suivant.

Exercice. Avec $[8, 1, 6, 3]$ et $T = 3$, écrire les paires et expliquer pourquoi utiliser $Y = X$ ne convient pas.

Correction détaillée. On obtient $X = [8, 1, 6]$ et $Y = [1, 6, 3]$. Choisir $Y = X$ demanderait de recopier le token courant au lieu de prévoir le suivant. Le dernier token 3 est une cible valable précisément parce qu'un quatrième token source est disponible.

Fiche 35 — Compter les fenêtres du dataset en mémoire

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/02_dataset.py

NextTokenDataset

Objectif. Retrouver la formule de longueur à partir des indices admissibles.

`NextTokenDataset` garde une liste d'IDs et une longueur de contexte. Il ne fabrique pas d'avance une liste contenant toutes les fenêtres. À chaque appel, deux tranches de la liste source sont converties en tenseurs `torch.long`. C'est le dataset simple de la démonstration, utile pour vérifier à la main la géométrie des exemples.

À comprendre. Le dernier indice de la cible impose la borne.

Pour un départ i et une longueur C , la dernière cible utilise l'ID d'indice $i + C$. Elle doit rester dans la liste de longueur N , dont le dernier indice est $N - 1$. Par conséquent :

$$0 \leq i \leq N - C - 1, \quad M = \text{len}(\text{dataset}) = N - C.$$

Le nombre d'entiers entre zéro et $N - C - 1$, bornes incluses, est bien $N - C$. Ce « plus un » du comptage des indices est une source fréquente d'erreurs ; partir de la dernière cible évite de deviner la formule.

Avec sept IDs $[0, 1, 2, 3, 4, 5, 6]$ et $C = 4$, il existe trois fenêtres. Leurs départs sont 0, 1 et 2. La dernière a $X = [2, 3, 4, 5]$ et $Y = [3, 4, 5, 6]$. Un départ 3 demanderait une cible d'indice 7, inexistante. Si l'on réduit le contexte à deux, le nombre de fenêtres passe à cinq, mais chaque prédiction dispose de moins d'historique.

À comprendre. Les fenêtres se recouvrent fortement.

Deux départs voisins partagent presque tous leurs tokens. Ils ne sont donc pas des observations indépendantes. Un token central peut contribuer à plusieurs cibles avec des préfixes différents. Ce recouvrement est voulu pour exploiter le texte ; il faudra cependant éviter de disperser de telles fenêtres entre entraînement et validation.

Point de vigilance. Cette classe reste volontairement peu défensive.

Le constructeur vérifie que la liste est plus longue que le contexte. Il n'effectue pas les validations détaillées de type, de positivité ou de bornes présentes dans `MemmapTokenDataset`. En particulier, il ne faut pas supposer que les indices négatifs reproduisent le comportement du dataset memmap : les tranches Python peuvent donner des fenêtres vides ou tronquées. Dans l'usage normal, le chargeur parcourt les indices valides de zéro à $M - 1$.

Exercice. Un corpus possède douze IDs. Combien de fenêtres obtient-on avec $C = 5$, puis $C = 11$? Pourquoi $C = 12$ échoue-t-il ?

Correction détaillée. On obtient sept fenêtres, puis une seule. Avec $C = 12$, il manque le treizième token nécessaire à la dernière cible. Le constructeur refuse donc cette configuration plutôt que fabriquer une cible incomplète.

Fiche 36 — Du dataset au batch : empiler les fenêtres

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

evaluate

Objectif. Lire une forme $[B, T]$ et traiter correctement le dernier lot.

Un dataset répond à la question « quel exemple correspond à cet indice ? ». Un `DataLoader` choisit des indices, récupère les exemples, puis les regroupe. Avec des fenêtres de même longueur, la collation standard empile les entrées dans un tenseur X et les cibles dans un tenseur Y . La dimension de batch B compte les fenêtres simultanées ; T compte les positions dans chacune.

À comprendre. Un batch ne concatène pas les fenêtres en une phrase.

Si deux exemples sont $[1, 2, 3]$ et $[4, 5, 6]$, l'entrée du batch possède deux lignes et trois colonnes, non une ligne de six positions. Le modèle traite chaque ligne comme une séquence distincte. La suite d'une ligne n'est pas le début de la suivante ; aucun contexte linguistique ne traverse la dimension de batch.

$$X, Y \in \{0, \dots, V-1\}^{B \times T}.$$

Les entrées restent des IDs entiers, pas des vecteurs d'embedding. La dimension D apparaîtra seulement quand le modèle cherchera les représentations associées à ces IDs.

Supposons dix fenêtres et une taille demandée $B = 4$. Comme les chargeurs du dépôt ne demandent pas `drop_last=True`, ils produisent des lots de quatre, quatre et deux fenêtres. Le dernier tenseur a donc la forme $[2, T]$. Une taille demandée ne garantit pas une taille effective constante en fin de parcours. Le nombre de lots est le plafond de M/B .

À comprendre. Une moyenne de pertes doit tenir compte du nombre de cibles.

La fonction `evaluate` multiplie la perte moyenne du lot par `y.numel()`, accumule cette somme, puis divise par le nombre total de cibles. Un petit lot n'a ainsi pas le même poids qu'un grand. Avec T fixe, pondérer par BT revient à pondérer par le nombre de fenêtres ; la formulation par token exprime directement ce qui est prédit.

Point de vigilance. L'ordre dépend du chargeur et de son échantillonneur.

La démonstration simple et la validation utilisent un ordre séquentiel. L'entraînement actif choisit des fenêtres avec remise. Ce choix change quelles lignes sont réunies, mais ne change pas le décalage interne entre chaque ligne de X et sa ligne de Y .

Exercice. Pour dix fenêtres et $B = 4$, les pertes moyennes des trois lots sont 1, 2 et 5. Calculer la perte globale.

Correction détaillée. Le contexte étant identique, la moyenne correcte est $(4 \times 1 + 4 \times 2 + 2 \times 5) / 10 = 2,2$. La moyenne non pondérée $8/3$ surévaluerait le petit dernier lot. Le code pondère précisément pour éviter ce biais arithmétique.

Fiche 37 — Éviter la fuite : séparer avant d'apprendre

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/02_dataset.py

prepare_corpus

Objectif. Définir une validation qui n'a participé ni au vocabulaire appris ni aux fenêtres d'entraînement.

La validation doit mesurer le comportement sur une portion tenue à l'écart de l'apprentissage. Une fuite de données apparaît quand une information de cette portion influence la construction du système évalué. La fuite ne se limite pas aux poids du réseau : choisir des fusions BPE à partir du texte de validation utilise déjà ses régularités.

À comprendre. La séparation intervient sur le texte brut.

Pour N_{car} caractères et une fraction f , le code calcule la longueur d'entraînement par `int(total * (1 - validation_fraction))`. Pour cette quantité positive, l'intention mathématique est :

$$n_{\text{train}} = \lfloor N_{\text{car}}(1 - f) \rfloor, \quad n_{\text{val}} = N_{\text{car}} - n_{\text{train}}.$$

L'implémentation effectue le produit en virgule flottante : près d'une frontière entière, il faut reprendre son calcul exact pour reproduire le découpage. Les nombres ci-dessus comptent des caractères, pas les IDs BPE qui n'existent pas encore.

Le préfixe sert à apprendre le tokenizer. Les deux portions sont ensuite encodées séparément avec ce même tokenizer et écrites dans deux fichiers. Les fenêtres sont enfin construites indépendamment dans chacun. Aucune fenêtre ne franchit la séparation. Les caractères du suffixe peuvent naturellement exister aussi dans le préfixe : « disjoint » signifie ici positions sources distinctes, pas alphabets sans intersection.

Prenons « abcabcZZ » et $f = 0,25$. Six caractères vont au train, deux à la validation. Le vocabulaire caractère apprend seulement a, b et c, plus l'inconnu. Les deux Z deviennent donc deux IDs zéro. Les apprendre dans le vocabulaire avant la séparation masquerait cette nouveauté réelle. Cet exemple exact fait partie des tests du dépôt.

Point de vigilance. Découper des fenêtres déjà créées est une mauvaise protection.

Deux fenêtres voisines se recouvrent. Si on en envoie une au train et l'autre en validation, une grande partie du passage évalué a déjà été utilisée. Le découpage brut avant fenêtrage élimine ce mécanisme, mais pas une répétition du même document à deux endroits du corpus. La déduplication des contenus constitue un contrôle supplémentaire, non réalisé par cette fonction.

Exercice. Un collègue apprend BPE sur tout le texte, puis sépare parfaitement les fenêtres en deux fichiers. La validation est-elle totalement tenue à l'écart ?

Correction détaillée. Non. Même sans fenêtre commune, le vocabulaire et les fusions ont exploité les fréquences de validation. Il faut d'abord séparer le brut, apprendre BPE sur le préfixe seulement, puis encoder le suffixe sans modifier le tokenizer. L'évaluation porte ainsi sur une représentation fixée avant de voir ce suffixe.

Fiche 38 — Split contigu : ce que mesure la validation

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/02_dataset.py

prepare_corpus

Objectif. Interpréter un score en fonction de l'ordre du corpus, sans confondre séparation et représentativité.

Le pipeline réserve la dernière fraction du texte à la validation. Ce découpage contigu conserve l'ordre local et évite le mélange de fenêtres voisines entre les deux ensembles. Il possède cependant une conséquence : la validation représente la fin du fichier, pas nécessairement la population de textes à laquelle on souhaite appliquer le modèle.

À comprendre. Une distribution peut changer le long du document.

Imaginons un corpus original constitué de descriptions d'objets, suivies d'instructions techniques. Avec une coupure près du changement de thème, le modèle apprend surtout sur des descriptions et est évalué sur des consignes. Sa perte reflète alors à la fois l'apprentissage et ce changement de domaine. Une validation difficile n'indique pas automatiquement un défaut d'optimisation ; elle peut signaler que la tâche évaluée diffère de la tâche entraînée.

À l'inverse, un suffixe répétant presque mot pour mot le début peut rendre la validation artificiellement facile. Les positions sont disjointes, mais le contenu ne l'est pas vraiment. Le code ne détecte ni doublons de documents ni versions proches.

À comprendre. La proportion brute n'est pas toujours la proportion tokenisée.

Avec le tokenizer caractère, un caractère produit un ID, même inconnu : la proportion de tokens suit celle des caractères, à l'arrondi près. Avec BPE, des régions très répétitives peuvent être davantage regroupées. Une validation de 10 % des caractères ne garantit donc pas 10 % des tokens, ni 10 % des fenêtres. Pour un contexte C et un pas un, chaque split perd en outre C départs par rapport à son nombre de tokens.

Point de vigilance. On ne doit pas corriger le biais en réutilisant silencieusement le train.

Le dépôt refuse les portions trop courtes ; il ne remplace pas la validation par des exemples d'entraînement. Pour une expérience mieux équilibrée, on peut préparer en amont un ordre de documents adapté à la question étudiée, en conservant des groupes indépendants. Cette opération extérieure au pipeline doit être consignée. Une séparation chronologique convient aussi pour évaluer la prévision de textes futurs.

Exercice. Un fichier de 10 000 caractères possède un suffixe de 1000 caractères dans une écriture absente du préfixe. Que prévoir en mode caractère ?

Correction détaillée. Avec une fraction de 0,1, ce suffixe sert à la validation. Les caractères non observés deviennent inconnus ; le score porte donc en partie sur la prédiction du marqueur zéro, pas sur la restitution de cette écriture. Il faut rapporter la couverture du vocabulaire pour interpréter honnêtement la perte.

Fiche 39 — Lire par blocs Unicode : les trois passages

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/02_dataset.py

_text_chunks

Objectif. Comprendre une lecture bornée qui préserve les fins de ligne.

`_text_chunks` ouvre le fichier avec `encoding='utf-8'` et `newline=""`. Il demande au lecteur texte des blocs d'au plus 4096 caractères. Le décodage UTF-8 est assuré par Python : la fonction n'effectue pas de découpage arbitraire dans les octets d'un caractère multioctet. Une ligne gigantesque reste donc lue par morceaux, au lieu d'être chargée intégralement avec un itérateur de lignes.

À comprendre. Les positions comptent des caractères déjà décodés.

La variable `position` avance de `len(chunk)`. Si une borne `stop` est fournie, la prochaine demande de lecture est réduite pour ne pas la dépasser. La borne `start` supprime la partie initiale des blocs qui précède la zone souhaitée. Elle ne déclenche pas un saut direct à un offset binaire : le fichier est lu depuis son début, puis les caractères antérieurs sont ignorés.

Par exemple, prenons les cinq caractères « a », « é », U+1F642, « b », « c ». La plage `[1, 4[` sélectionne « é », U+1F642 et « b ». Chercher directement les octets de positions 1 à 4 ne produirait pas cette même sélection. L'option `newline=""` laisse subsister les caractères `\r` et `\n` au lieu de convertir une paire CRLF en un unique saut de ligne. Cette fidélité concerne la lecture ; le mode caractère peut encore remplacer un signe absent de son alphabet.

À comprendre. La préparation organise trois passages logiques.

Le premier compte tous les caractères pour déterminer la coupure. Le deuxième fournit le préfixe au tokenizer ; en BPE, son budget peut arrêter cette consommation avant la fin du préfixe. Le troisième relit tout le corpus et écrit les IDs des deux splits, en coupant exactement le bloc qui contient leur frontière. Cela évite de conserver en RAM une copie complète du texte.

Point de vigilance. La source doit rester stable entre ces lectures.

Si un autre processus ajoute, retire ou modifie du texte, le comptage, le vocabulaire et les IDs peuvent ne plus décrire le même corpus. Le programme ne prend pas de cliché immuable et ne vérifie pas une empreinte entre les passages. « Trois passages » décrit une stratégie de mémoire, pas un protocole de synchronisation avec des rédacteurs concurrents. Le coût d'entrées-sorties demeure réel, même quand la mémoire est bornée.

Exercice. Une fin de fichier contient « A » suivi de CRLF puis « é ». Combien de caractères le comptage conserve-t-il ?

Correction détaillée. Quatre : A, retour chariot, saut de ligne et é. Les octets UTF-8 sont au nombre de cinq. La préservation de CRLF empêche que la coupure soit calculée sur un texte implicitement raccourci par conversion des fins de ligne.

Fiche 40 — Préparer les artefacts : protection et limites

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/02_dataset.py

prepare_corpus

Objectif. Distinguer écriture préparatoire, publication et transaction globale.

La préparation produit deux fichiers d'IDs et un état de tokenizer. Pour la sortie par défaut, le répertoire concerné est `/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/checkpoints/tokens`. Avant de toucher aux sorties définitives, le code vérifie la fraction, le type de tokenizer, l'absence de collision avec le chemin du corpus, puis la longueur brute des deux portions. Il apprend ensuite le tokenizer.

À comprendre. Le staging retarde le remplacement des sorties existantes.

Les IDs sont d'abord écrits sous des noms portant le suffixe `.partial`. Les compteurs de tokens doivent atteindre deux dans chaque split. L'état JSON est ensuite écrit sous sa forme préparatoire. Seulement après ces étapes, une boucle appelle `source.replace(target)` successivement pour chaque artefact. Un bloc `finally` tente de supprimer les fichiers préparatoires restants, y compris lorsqu'une exception interrompt la préparation.

Les destinations par défaut sont

`/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/checkpoints/tokens/train.bin`,
`/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/checkpoints/tokens/val.bin` et
`/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/checkpoints/tokens/tokenizer.json`.

L'écriture préparatoire du premier utilise notamment

`/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/checkpoints/tokens/train.bin.partial`. Le même suffixe est appliqué aux deux autres destinations.

Point de vigilance. Ce n'est pas une transaction « tout ou rien » sur le trio.

Même lorsqu'un remplacement individuel est atomique dans le système de fichiers, trois remplacements successifs ne constituent pas une seule opération atomique. Si le premier réussit et le deuxième échoue, le répertoire peut mélanger une nouvelle version et des anciennes. Le nettoyage des fichiers préparatoires ne remet pas le premier artefact dans son état antérieur. Le code n'ajoute ni verrou de répertoire ni journal de restauration, et ne promet pas une durabilité face à une coupure électrique.

À comprendre. Le bon usage réduit les risques sans inventer de garanties.

Il faut garder la source inchangée, prévoir assez d'espace disque et ne pas partager un répertoire de sortie entre préparations simultanées. La coexistence de sorties anciennes et préparatoires peut demander davantage d'espace que la seule taille finale. Après une interruption pendant la publication, reconstruire un ensemble cohérent est plus sûr que supposer que tous les noms désignent la même préparation.

Exercice. Une erreur « split trop court en tokens » survient avant la boucle de remplacement. L'ancien fichier d'entraînement est-il déjà écrasé ? Et si le deuxième remplacement échoue ?

Correction détaillée. Dans le premier cas, l'ancien fichier reste intact et les préparatoires sont nettoyés : ce scénario est testé. Dans le second, le premier remplacement peut déjà avoir publié sa nouvelle version. Le test du premier scénario ne prouve donc pas l'atomicité globale du second.

Fiche 41 — Des IDs binaires de huit octets

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/02_dataset.py

MemmapTokenDataset

Objectif. Calculer l'espace disque et reconnaître ce que le format ne décrit pas.

La préparation convertit les IDs de chaque bloc avec `np.asarray(..., dtype=np.int64)`, puis écrits avec `tofile`. Un entier `int64` occupe huit octets. Les écritures successives s'ajoutent dans le même flux binaire ; aucun séparateur n'est nécessaire entre les blocs puisque chaque valeur possède une taille fixe.

À comprendre. La taille donne le nombre de tokens, pas leur signification.

Pour N IDs, le volume utile du fichier est :

$$S = 8N \text{ octets}, \quad N = S/8.$$

Un million de tokens demande donc huit millions d'octets, soit environ 7,63 Mio. Les deux unités diffèrent : un mégaoctet décimal vaut un million d'octets, tandis qu'un mébioctet vaut 2^{20} octets. L'état du tokenizer et les autres sorties s'ajoutent à cette estimation.

Le dataset lit la taille du fichier et refuse une longueur qui n'est pas multiple de huit. Cette vérification détecte, par exemple, un fichier de dix-neuf octets. Elle ne détecte pas un fichier arbitraire de vingt-quatre octets : trois groupes de huit octets peuvent être interprétés comme des entiers même s'ils ne viennent pas du tokenizer attendu.

À comprendre. Le format est volontairement minimal.

Il n'existe ni en-tête, ni numéro de version, ni taille de vocabulaire, ni nom de tokenizer dans les binaires. Le type NumPy est natif : l'ordre des octets dépend de la plateforme. Transférer ces fichiers vers une architecture d'endianness différente demanderait une convention explicite ou une conversion ; le programme ne l'effectue pas automatiquement. Sur la plateforme courante, NumPy et PyTorch utilisent ici des entiers compatibles pour indexer les embeddings.

Point de vigilance. Taille valide ne signifie pas IDs valides.

Les IDs produits normalement sont compris entre zéro et $V - 1$. `MemmapTokenDataset` ne vérifie pas chaque valeur contre V , qu'il ne reçoit d'ailleurs pas en argument. Une corruption ou un tokenizer incohérent peut donc n'être découverte qu'au moment d'accéder aux embeddings, voire produire des valeurs sémantiquement fausses sans dépasser les bornes. Conserver l'association entre données, état et expérience reste indispensable.

Exercice. Deux splits comportent 120 000 et 30 000 tokens. Quel volume binaire total prévoir ? Un fichier supplémentaire de 65 octets est-il un fichier d'IDs acceptable ?

Correction détaillée. Il faut $8(120\,000 + 30\,000) = 1\,200\,000$ octets, sans compter le JSON ni les checkpoints. Soixante-cinq n'est pas divisible par huit : le constructeur le refuse. Pour autant, arrondir sa taille à 64 octets ne réparerait pas son contenu et ne garantirait aucune cohérence des IDs.

Fiche 42 — Fenêtres memmap : formule et pas de déplacement

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/02_dataset.py

MemmapTokenDataset

Objectif. Dériver le nombre d'exemples lorsque les départs sont espacés par un stride.

Le dataset memmap travaille sur un fichier d'IDs plutôt que sur une liste complète en mémoire. Son constructeur en déduit le nombre N de tokens à partir de la taille binaire. Il reçoit un contexte C et un pas $s = \text{stride}$, tous deux entiers strictement positifs. Le pas règle l'espacement des départs de fenêtres, pas l'espacement des tokens à l'intérieur d'une fenêtre.

À comprendre. Le départ du i -ème exemple vaut is .

La dernière cible de cet exemple a l'indice $is + C$. Pour rester dans le fichier, il faut $is + C \leq N - 1$. Les indices admissibles donnent donc :

$$M = \left\lfloor \frac{N - C - 1}{s} \right\rfloor + 1.$$

Lorsque $s = 1$, cette expression se simplifie en $N - C$, la formule du dataset historique. Lorsque $N = C + 1$, il existe exactement une fenêtre, quelle que soit la valeur positive du pas. Le constructeur refuse $N \leq C$.

Prenons onze IDs $[0, 1, \dots, 10]$, $C = 4$ et $s = 3$. Les départs sont 0, 3 et 6, soit trois exemples. Le dernier possède $X = [6, 7, 8, 9]$ et $Y = [7, 8, 9, 10]$. On ne reçoit pas $[6, 9, \dots]$: le contenu de chaque fenêtre reste contigu. Un départ supplémentaire à 9 dépasserait la limite. Avec $s = 4$, seuls les départs 0 et 4 seraient retenus ; certains tokens de fin ne contribueraient alors à aucune cible.

À comprendre. Augmenter le pas change la couverture, pas seulement la vitesse.

Un petit pas multiplie les contextes presque identiques. Un grand pas réduit le nombre de fenêtres et peut laisser des intervalles inexploités lorsqu'il dépasse la largeur utile des exemples. Les comparaisons d'entraînement doivent donc enregistrer ce paramètre. Dans le chemin actif, le même pas est appliqué aux datasets d'entraînement et de validation : le score peut porter sur moins de fenêtres de validation.

Point de vigilance. Les bornes sont contrôlées explicitement.

Un indice négatif est ramené depuis la fin, comme dans une séquence Python : `dataset[-1]` désigne le dernier exemple. Après cette conversion, les indices hors intervalle provoquent une erreur, plutôt qu'une tranche raccourcie. Les valeurs non entières sont également refusées par la conversion d'indice.

Exercice. Pour $N = 20$, $C = 5$ et $s = 4$, calculer M , le dernier départ et les indices sources de sa cible.

Correction détaillée. On obtient $M = \lfloor 14/4 \rfloor + 1 = 4$. Les départs sont 0, 4, 8 et 12. La dernière cible utilise les indices 13 à 17 inclus ; les tokens 18 et 19 ne sont pas exploités par ces fenêtres.

Fiche 43 — Memmap, copies et séparation des processus

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/02_dataset.py

MemmapTokenDataset

Objectif. Expliquer la mémoire économisée sans promettre une consommation nulle.

Un memory mapping associe le contenu d'un fichier à une zone adressable par le processus. Le système d'exploitation charge les pages nécessaires au fil des accès et peut les garder en cache. Le dataset évite ainsi de charger d'emblée tous les IDs dans une liste Python et ne mémorise pas non plus un tableau de tous les départs de fenêtres.

À comprendre. L'ouverture est paresseuse et propre au processus courant.

Le constructeur place `_tokens` et `_pid` à `None`. Au premier accès, il ouvre `np.memmap(..., mode='r')` et mémorise l'identifiant du processus avec `os.getpid()`. Si un accès ultérieur vient d'un processus différent, le test d'identifiant entraîne une nouvelle ouverture. Cela importe notamment lorsqu'un chargeur utilise plusieurs processus de travail. Le chargeur actif du dépôt ne configure toutefois pas explicitement de workers supplémentaires.

À comprendre. Les tenseurs retournés sont des copies de fenêtres.

Les tranches X et Y sont chacune copiées avant l'appel à `torch.from_numpy`. PyTorch partage alors la mémoire de ces petits tableaux copiés, pas celle du fichier mappé. Modifier un élément de X ne modifie donc ni le fichier ni une future lecture de la même fenêtre. Les deux copies sont également indépendantes entre elles, malgré le recouvrement des positions sources.

$$S_{\text{copies par exemple}} = 2 \times C \times 8 = 16C \quad \text{octets utiles.}$$

Cette estimation ne comprend pas les objets Python, les batches empilés, le cache du système, les transferts vers le GPU ni les activations du modèle.

À comprendre. La sérialisation transporte une description, pas le mapping ouvert.

`__getstate__` copie le dictionnaire d'attributs et remet `_tokens` ainsi que `_pid` à `None` dans l'état exporté. Après restauration, le prochain accès rouvrira le fichier. Le chemin absolu doit donc encore exister et désigner les mêmes données ; le pickle n'est pas une archive autonome du corpus.

Point de vigilance. Les fichiers doivent rester inchangés pendant leur utilisation.

Les remplacer ou les tronquer pendant qu'un dataset les consulte invalide les hypothèses de taille et de cohérence. La lecture seule protège contre les écritures par ce mapping, pas contre un autre programme.

Exercice. Avec $C = 128$, quelle taille représentent les deux copies utiles ? Une modification de `x[0]` persiste-t-elle après relecture ?

Correction détaillée. Elles représentent $16 \times 128 = 2048$ octets, soit 2 Kio, avant les surcoûts. La modification reste locale à la copie retournée. Une relecture retrouve la valeur du fichier ; cette propriété est précisément vérifiée dans les tests de mutation et de restauration du dataset.

Fiche 44 — Échantillonner avec remise n'est pas mélanger

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

train

Objectif. Interpréter une époque dans le chemin d'entraînement actuel.

L'entraînement utilise `RandomSampler` avec `replacement=True` et `num_samples=len(dataset)`. Il demande donc autant de tirages qu'il existe de fenêtres, mais chaque tirage peut reprendre un indice déjà obtenu. Le choix évite une permutation complète des indices dont la taille augmenterait avec le corpus. Il faut distinguer cette économie de mémoire de la sélection des fenêtres par `stride`, effectuée en amont.

À comprendre. Une permutation visite chaque indice ; des tirages avec remise non.

Pour quatre fenêtres, une permutation possible est $[2, 0, 3, 1]$. Un parcours avec remise possible est $[2, 2, 0, 2]$: trois visites de la fenêtre 2, une de la fenêtre 0 et aucune des deux autres. Ces listes sont des exemples de principe, pas des sorties annoncées pour une graine particulière du dépôt.

Si les M fenêtres ont la même probabilité à chaque tirage, la probabilité qu'une fenêtre donnée ne soit jamais choisie lors des M tirages vaut :

$$p_{\text{absente}} = \left(1 - \frac{1}{M}\right)^M.$$

L'espérance du nombre de fenêtres distinctes visitées est alors $M(1 - p_{\text{absente}})$. Pour un grand M , cela représente environ 63,2 % du dataset. Ce pourcentage moyen ne décrit pas une garantie pour une époque particulière ; il résume une distribution de résultats possibles.

À comprendre. La validation suit un autre protocole.

Son chargeur emploie `shuffle=False` : les fenêtres retenues par le pas sont parcourues séquentiellement. La validation complète n'est donc pas un échantillon avec remise. De plus, les fenêtres se recouvrent : visiter 63 % des fenêtres ne signifie pas avoir lu exactement 63 % des positions textuelles distinctes.

Point de vigilance. Les limites d'entraînement et les appels auxiliaires comptent.

Un `max_steps` peut interrompre l'époque avant tous les tirages. Avant la boucle d'optimisation, le programme prélève aussi un batch pour afficher les formes. Ce prélèvement utilise le générateur aléatoire, mais ne constitue pas une étape d'optimisation. La graine est fixée avec `torch.manual_seed` ; la reproduction d'un ordre précis suppose de conserver aussi la suite des appels et l'environnement pertinent.

Exercice. Avec $M = 4$, calculer la probabilité qu'une fenêtre donnée soit absente d'une époque complète et le nombre moyen de fenêtres distinctes.

Correction détaillée. La probabilité est $(3/4)^4 = 81/256$, environ 31,6 %. L'espérance vaut $4(1 - 81/256) = 175/64$, environ 2,73 fenêtres. Une époque de quatre tirages ne garantit donc pas quatre exemples différents.

Fiche 45 — Corpus trop court : identifier l'étape qui refuse

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/02_dataset.py

prepare_corpus

Objectif. Diagnostiquer une erreur sans modifier au hasard la fraction ou le contexte.

La préparation et le dataset contrôlent des conditions différentes. Il faut d'abord compter les caractères des splits, puis les tokens produits, et enfin les tokens nécessaires à une fenêtre. Un texte long en caractères peut être court en BPE si ses motifs se regroupent fortement. La réussite de la préparation ne garantit donc pas que le contexte demandé soit utilisable.

À comprendre. Premier seuil : les caractères bruts.

Après calcul de la coupure, chaque portion doit contenir au moins deux caractères. Sinon, le message exact est :

```
Training and validation splits must each contain at least two characters; use a larger corpus or
adjust validation_fraction.
```

Cette erreur survient avant l'apprentissage du tokenizer et l'écriture des sorties préparatoires. Elle s'applique notamment à un corpus vide. Une fraction hors intervalle, infinie ou non numérique au sens de NaN est rejetée encore plus tôt par le contrôle de fraction.

À comprendre. Deuxième seuil : les tokens encodés.

Après écriture préparatoire, chaque split doit contenir au moins deux IDs. Par exemple, si l'entraînement n'en produit qu'un, le message exact est `Training split has only 1 tokens; at least two are required`. La formulation conserve `tokens` même pour un seul élément. Pour la validation, le premier mot devient `Validation`. Ce contrôle utilise le résultat effectif du tokenizer, pas une estimation fondée sur le nombre de caractères.

À comprendre. Troisième seuil : le contexte.

`MemmapTokenDataset` exige au moins $C + 1$ tokens. Son message exact est :

```
Token file must contain at least context_length + 1 tokens.
```

La classe historique utilise une autre formulation : `Le corpus doit être plus long que context_length`. Ne pas confondre ces deux messages aide à retrouver le parcours réellement exécuté. Les contextes ou pas non positifs, fractionnaires ou booléens sont également refusés par le constructeur `memmap`.

Point de vigilance. Réduire le contexte modifie la tâche ; recycler le train fausse l'évaluation.

On peut fournir davantage de texte ou ajuster la fraction pour agrandir la portion limitante. Réduire C est aussi possible, mais raccourcit l'historique accessible. Le code ne remplit pas les splits par répétition et ne substitue jamais le train à une validation insuffisante.

Exercice. Une préparation donne 80 tokens d'entraînement et 12 de validation. Que se passe-t-il avec $C = 16$, puis $C = 11$, à pas un ?

Correction détaillée. La préparation passe les deux seuils de tokens. Avec $C = 16$, la validation échoue au troisième seuil. Avec $C = 11$, elle possède une fenêtre, contre 69 pour l'entraînement. Le programme peut alors construire les datasets, mais cette validation minuscule reste statistiquement fragile.

Fiche 46 — Lire les tests : preuves locales et angles morts

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/02_dataset.py

MemmapTokenDataset

Objectif. S'appuyer sur les assertions existantes sans leur attribuer une garantie universelle.

Le fichier de test est `/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/tests/test_data.py` et utilise `unittest`, NumPy et PyTorch. Les cas construisent de petits corpus contrôlés et vérifient des propriétés observables du pipeline. Lire un test consiste à identifier ses données, son action et son assertion. Le nom d'un test suggère son intention ; seules les assertions montrent ce qui est effectivement vérifié.

À comprendre. Les tests de tokenisation couvrent plusieurs contrats distincts.

Le parcours caractère vérifie un aller-retour Unicode, le remplacement d'un inconnu, la restauration JSON et la collecte d'alphabet par flux. Un import BPE volontairement bloqué démontre que le mode caractère reste utilisable et que le mode BPE signale la dépendance absente. Le test du budget réduit les constantes à sept et trois afin de comparer exactement les blocs produits. Ce n'est pas une mesure de mémoire sur un véritable million de caractères.

Les cas BPE vérifient, lorsque la bibliothèque est disponible, le décodage d'Unicode absent de l'apprentissage, les retours de ligne, le marqueur littéral et la restauration de l'état. Ils sont décorés par `skipUnless` : une suite réussie avec ces cas ignorés n'a pas exercé la fonctionnalité BPE. Il faut donc lire aussi les tests sautés, pas seulement l'absence d'échec.

À comprendre. Les tests de corpus contrôlent la séparation et les accès bornés.

Un suffixe `ZZ` prouve que ces caractères n'entrent pas dans l'alphabet appris. Une grande ligne Unicode instrumente les appels à `read` et exige une taille positive ne dépassant pas le bloc configuré. Le test protège donc contre une lecture textuelle non bornée, mais ne mesure pas toute la RAM du processus. Un encodeur simulé produisant un seul token vérifie la conservation d'un ancien artefact avant publication et le nettoyage des préparatoires.

À comprendre. Les tests memmap relient le nouveau parcours au parcours simple.

Ils contrôlent le pas, les indices négatifs, les bornes, le type des tenseurs, l'indépendance des copies et une restauration par pickle. Ils comparent aussi toutes les fenêtres d'un petit exemple avec `NextTokenDataset`, puis vérifient un batch et son décalage. Les entrées invalides et les tailles binaires mal formées ont leurs cas dédiés.

Exercice. Ces tests prouvent-ils une publication atomique du trio et le fonctionnement de plusieurs workers sur tous les systèmes ?

Correction détaillée. Non. Le cas de préservation échoue avant les remplacements ; il ne simule pas une panne au milieu de leur boucle. Le pickle démontre une remise à zéro du mapping, mais n'est pas une campagne multiprocessus multiplateforme. Ces limites n'annulent pas les preuves locales : elles en délimitent honnêtement la portée.

Fiche 47 — Atelier : préparer puis observer char et BPE

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

main

Objectif. Planifier deux expériences comparables sans annoncer de résultats non mesurés.

Les commandes ci-dessous sont à exécuter par le lecteur ; elles ne constituent pas un compte rendu d'exécution. Elles utilisent le corpus pédagogique fourni, un contexte volontairement court et deux répertoires distincts. Elles illustrent le chemin actif : même avec deux étapes d'optimisation, la préparation des données précède le modèle et la validation reste effectuée. La limite d'étapes ne transforme pas la préparation en lecture partielle du corpus.

À comprendre. Commencer par le mode caractère, sans dépendance BPE.

Après installation des dépendances ordinaires décrites dans /home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/requirements.txt, on peut lancer :

```
python /home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py \
--corpus /home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/data/tiny_corpus.txt \
--checkpoint-dir /home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/atelier-char \
--tokenizer char --preset tiny --context-length 4 \
--validation-fraction 0.2 --batch-size 2 --epochs 1 \
--max-steps 2 --device cpu --seed 42
```

Relever le vocabulaire effectif, les formes affichées et les tailles binaires des deux splits dans le sous-répertoire de tokens de cette sortie. Diviser chaque taille binaire par huit donne son nombre d'IDs. Pour $C = 4$ et le pas par défaut, soustraire quatre donne le nombre de fenêtres. Ce calcul vérifie les unités, sans exiger une bonne qualité de génération après seulement deux mises à jour.

À comprendre. Passer ensuite au mode BPE dans une autre sortie.

```
python -m pip install -r /home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/requirements-bpe.txt
python /home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py \
--corpus /home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/data/tiny_corpus.txt \
--checkpoint-dir /home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/atelier-bpe \
--tokenizer bpe --vocab-size 280 --preset tiny --context-length 4 \
--validation-fraction 0.2 --batch-size 2 --epochs 1 \
--max-steps 2 --device cpu --seed 42
```

Ces deux répertoires évitent de remplacer les artefacts caractère par un état BPE incompatible. Le petit corpus permet une manipulation, mais le guide avertit qu'il ne suffit pas à apprendre un BPE réellement utile. Pour une étude linguistique, il faudrait fournir un corpus personnel substantiel et représentatif, avec le même protocole de séparation.

Point de vigilance. Comparer des unités équivalentes.

Les nombres de tokens et les pertes peuvent changer parce que les unités changent. Une perte BPE plus petite ne prouverait pas à elle seule une meilleure modélisation du même texte. Consigner aussi le corpus, la fraction, le contexte, la graine et le vocabulaire effectif. Ne pas lancer simultanément deux préparations dans une même sortie.

Exercice. Les deux expériences produisent-elles nécessairement le même nombre de fenêtres puisque leur corpus et leur fraction sont identiques ?

Correction détaillée. Non. La coupure brute est la même, mais BPE peut regrouper plusieurs caractères par token. Il faut recalculer les fenêtres à partir des tailles binaires réellement obtenues. Aucune valeur de perte, de durée ou de compression n'est présumée dans cet atelier.

Fiche 48 — Auditer tout le pipeline avant le Transformer

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/02_dataset.py

prepare_corpus

Objectif. Relier les unités, les invariants et les limites dans un diagnostic complet.

Avant d'étudier l'attention, on peut déjà auditer les données sans entraîner un grand réseau. Le trajet est désormais explicite : texte UTF-8 stable, comptage Unicode, séparation contiguë, apprentissage du tokenizer sur le train, encodage par blocs, publication des artefacts, fenêtres, puis batches. Chaque flèche change la représentation ou l'organisation des informations ; aucune ne doit changer silencieusement la question évaluée.

À comprendre. Quatre registres évitent les confusions.

Le registre textuel compte caractères et octets sources. Le registre de tokenisation fixe le vocabulaire V , les IDs et l'état nécessaire à leur interprétation. Le registre du dataset compte N tokens, un contexte C , un pas s et M fenêtres. Enfin, le registre du calcul compte B fenêtres par batch et $T = C$ positions par fenêtre. Les dimensions D , les H têtes, les F unités du réseau intermédiaire et les L couches appartiendront au modèle, pas au format binaire d'entrée.

À comprendre. Un invariant doit être formulé avec son domaine de validité.

Le décalage impose $X_{t+1} = Y_t$ aux positions internes d'une fenêtre. La couverture caractère peut perdre les signes inconnus. Le byte-level permet leur représentation, mais les frontières de blocs peuvent changer les IDs. Le mmap évite la matérialisation du corpus entier, sans supprimer les copies ni le cache du système. La séparation empêche les fenêtres communes aux deux splits, sans rechercher les doublons textuels éloignés.

Point de vigilance. L'audit doit inclure l'exploitation, pas seulement les formules.

Un corpus modifié pendant les passages, un tokenizer remplacé isolément, un fichier tronqué pendant sa lecture ou deux écrivains concurrents peuvent invalider des calculs pourtant corrects. Conserver une source stable, des sorties distinctes et des paramètres consignés est une partie de la reproductibilité. Les tests ne transforment pas le staging en transaction globale.

Exercice. On prépare « abcabcZZ » en mode caractère avec $f = 0,25$, $C = 1$, $s = 2$ et $B = 2$. Donner vocabulaire, tailles binaires, fenêtres et limite d'interprétation de la validation.

Correction détaillée. Le train est « abcabc », la validation « ZZ ». La table est ['<unk>', 'a', 'b', 'c']. Les IDs sont [1, 2, 3, 1, 2, 3] et [0, 0], soit 48 et 16 octets. Le train compte $\lfloor (6-1-1)/2 \rfloor + 1 = 3$ fenêtres, de départs 0, 2 et 4 : les paires sont [1] → [2], [3] → [1], [2] → [3]. La validation possède une paire [0] → [0]. Ses deux caractères sont inconnus : prédire correctement zéro ne signifie pas avoir appris à produire « Z ». Les lots de train contiennent deux, puis une fenêtre lors d'un parcours complet ; avec remise, leurs indices peuvent se répéter.

Fiche 49 — Un identifiant choisit une ligne

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/05_model.py

MiniGPT.token_embedding

Objectif. Passer d'un entier désignant un token à un vecteur entraînable, sans confondre le numéro du token avec une grandeur numérique.

À comprendre. Une table d'embeddings est une matrice de paramètres. Elle possède V lignes, une par entrée du vocabulaire, et D colonnes. Le token numéro i sélectionne sa ligne : son identifiant est une adresse, pas une mesure de son sens.

Dans le constructeur de MiniGPT, `nn.Embedding(vocab_size, embedding_dim)` crée cette table. Dans `forward`, `self.token_embedding(x)` transforme les identifiants entiers de forme (B, T) en nombres réels de forme (B, T, D) . La longueur de la séquence ne change pas ; chaque case reçoit maintenant D coordonnées.

On peut comprendre cette sélection avec un vecteur indicateur, aussi appelé *one-hot*. Il contient V coordonnées, toutes nulles sauf celle d'indice i , égale à un. Si E désigne la table, alors

$$e_i = (0, \dots, 1, \dots, 0), \quad h_i = e_i E = E_{i,:}.$$

Le produit matriciel sélectionne exactement la même ligne. PyTorch n'a toutefois pas besoin de fabriquer tous ces zéros : l'accès indexé exprime directement l'opération voulue. Cette équivalence sert à comprendre les mathématiques, pas à recommander une implémentation plus volumineuse.

Par exemple, prenons une table pédagogique dont les trois lignes sont $(1, 0)$, $(0, 2)$ et $(-1, 3)$. La séquence d'identifiants $(2, 0, 2)$ devient $((-1, 3), (1, 0), (-1, 3))$. Deux occurrences du même identifiant donnent ici le même vecteur. Elles pourront devenir différentes après l'ajout des positions et le passage dans les blocs.

La table contient VD paramètres, indépendamment de B et de T . Pendant l'apprentissage, les occurrences d'un même token contribuent à la modification d'une seule ligne partagée. Ce partage n'implique pas que deux tokens voisins dans la liste aient des embeddings voisins : l'ordre du vocabulaire reste conventionnel.

Point de vigilance. Un embedding n'est pas une probabilité : ses coordonnées peuvent être négatives et leur somme n'est pas imposée. Il est appris avec le reste du modèle, sans dictionnaire sémantique fourni à l'avance.

Exercice. Pour $V = 23$, $D = 16$, $B = 2$ et $T = 8$, combien la table contient-elle de paramètres ? Quelle est la forme obtenue ? Deux tokens identiques de deux fenêtres différentes utilisent-ils deux lignes distinctes ?

Correction détaillée. La table contient $23 \times 16 = 368$ paramètres et la sortie a la forme $(2, 8, 16)$, soit 256 nombres. Ces nombres sont des activations, pas 256 paramètres supplémentaires. Les tokens identiques consultent la même ligne, quelle que soit leur fenêtre ; seule leur utilisation ultérieure peut différencier leurs représentations.

Fiche 50 — Ajouter des positions apprises

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/05_model.py

MiniGPT.position_embedding

Objectif. Comprendre pourquoi MiniGPT ajoute une information de position et comment une table unique est partagée entre toutes les fenêtres d'un lot.

À comprendre. La table des tokens ne distingue pas les occurrences d'un même identifiant. En mode `learned`, une seconde table contient une ligne par position autorisée. Son rôle est de fournir au réseau une information sur la place occupée dans la fenêtre.

Le constructeur crée `nn.Embedding(context_length, embedding_dim)`. Cette table possède C lignes de dimension D , donc CD paramètres. Sans cache, les positions sont les entiers de zéro à $T - 1$. L'appel à `unsqueeze(0)` leur donne la forme $(1, T)$; la table produit alors un tenseur $(1, T, D)$.

Les embeddings de tokens ont, eux, la forme (B, T, D) . L'addition utilise le *broadcasting* : la dimension initiale égale à un est compatible avec les B fenêtres. Toutes reçoivent les mêmes vecteurs positionnels aux mêmes indices, sans créer B tables de paramètres distinctes.

$$h_{b,t,:} = E_{x_{b,t,:}} + P_{t,:}, \quad (B, T, D) + (1, T, D) \longrightarrow (B, T, D).$$

La somme, et non la concaténation, conserve une largeur D . Par exemple, un token représenté par $(1, 2)$, placé à une position représentée par $(3, -1)$, fournit $(4, 1)$. À une autre position de vecteur $(0, 2)$, le même token fournit $(1, 4)$. Le réseau reçoit ainsi des vecteurs différents avant toute attention.

Le code applique ensuite le dropout à cette somme. En évaluation, ce dropout est désactivé. En entraînement, il peut masquer certaines coordonnées ; il n'efface pas des entrées de la table elle-même. Avec un cache contenant déjà p tokens, les nouvelles positions commencent à p , pas à zéro.

Point de vigilance. En mode `rope`, `position_embedding` vaut `None`. MiniGPT n'ajoute alors aucune table de positions : les rotations des requêtes et des clés fournissent une autre organisation positionnelle. Dans les deux modes, le contrat du modèle conserve la limite C .

Exercice. Une table utilise $C = 8$ et $D = 16$. Un cache contient trois tokens et l'entrée nouvelle en contient deux, pour $B = 2$. Donnez les indices positionnels, la forme des positions encodées et le nombre de paramètres positionnels.

Correction détaillée. Les indices sont trois et quatre. Après lookup, les positions ont la forme $(1, 2, 16)$, diffusée sur les deux fenêtres lors de l'addition. La table complète contient $8 \times 16 = 128$ paramètres, même si cet appel n'en consulte que deux lignes. La longueur totale cinq reste inférieure à huit.

Fiche 51 — Requêtes, clés et valeurs : trois projections

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

MultiHeadSelfAttention.q_proj

Objectif. Donner un sens opérationnel aux lettres Q , K et aux valeurs, sans attribuer au réseau des intentions humaines.

À comprendre. Une projection linéaire transforme chaque vecteur de dimension D en un autre vecteur. Ici, les couches `q_proj`, `k_proj` et `v_proj` sont trois applications affines indépendantes, appliquées au même tenseur d'entrée.

Le mot *requête* désigne le vecteur utilisé du côté de la position qui reçoit de l'information. Une *clé* désigne le vecteur avec lequel cette requête est comparée. Une *valeur* désigne le contenu numérique ensuite combiné. Ce vocabulaire décrit trois rôles dans un calcul ; il n'affirme pas qu'un token pose consciemment une question.

Pour une représentation ligne x_t , on peut écrire, en respectant le stockage des poids de `nn.Linear`,

$$q_t = x_t W_Q^\top + b_Q, \quad k_t = x_t W_K^\top + b_K, \quad v_t = x_t W_{\text{val}}^\top + b_{\text{val}}.$$

Chaque matrice de poids a la forme (D, D) , chaque biais possède D coordonnées. Les mêmes paramètres sont utilisés à toutes les positions et pour toutes les fenêtres. Les trois résultats ont initialement la forme (B, T, D) ; leur séparation en têtes vient ensuite.

Pour une expérience de pensée, fixons les trois matrices à l'identité et leurs biais à zéro. On obtient trois copies de l'entrée. Cela permet de suivre les opérations, mais ce n'est pas l'initialisation du dépôt. Les matrices réellement entraînaibles peuvent sélectionner des combinaisons de coordonnées très différentes.

Les valeurs ne servent pas directement à calculer les scores de compatibilité : ceux-ci utilisent les requêtes et les clés. Inversement, les clés ne sont pas directement moyennées pour fabriquer la sortie ; ce sont les valeurs qui le sont. Séparer ces rôles donne au réseau davantage de liberté qu'un simple moyennage des embeddings d'entrée.

Point de vigilance. Les lettres ne désignent pas trois séquences de tokens différentes. Dans cette auto-attention, les trois projections partent du même tenseur. Pour éviter une ambiguïté, nous noterons V_{att} la matrice des valeurs, et réserverons V à la taille du vocabulaire.

Exercice. Avec $D = 4$, comptez les paramètres des trois projections, biais compris. Si x_t est nul, les trois résultats sont-ils nécessairement nuls après apprentissage ?

Correction détaillée. Chaque projection contient $4 \times 4 + 4 = 20$ paramètres, donc 60 au total. Pour une entrée nulle, les sorties sont les biais respectifs. Ceux-ci sont initialisés à zéro par MiniGPT, mais restent entraînaibles : des sorties non nulles sont donc possibles après apprentissage.

Fiche 52 — Construire la grille des scores

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

scaled_dot_product_attention

Objectif. Lire le produit $QK^\top$ comme une grille de comparaisons, en identifiant précisément ses axes et ses sommes.

À comprendre. Un score est un produit scalaire entre une requête et une clé. La ligne indique la position qui reçoit l'information ; la colonne indique la position susceptible d'en fournir. À ce stade, le score n'est pas une probabilité.

Pour une tête, chaque requête possède d_k coordonnées. Le produit scalaire multiplie les coordonnées correspondantes puis les additionne. Avec T requêtes et T clés, cette opération remplit une matrice carrée :

$$S_{ij} = \sum_{r=0}^{d_k-1} q_{i,r} k_{j,r}, \quad (T, d_k)(d_k, T) \longrightarrow (T, T).$$

La transposition de K rend les dimensions internes compatibles. Sans elle, le produit matriciel demandé n'aurait généralement pas de sens. Le code utilise `k.transpose(-2, -1)` : seuls les deux derniers axes sont échangés, pas ceux du lot ou des têtes.

Dans le modèle multi-têtes, Q et K ont la forme (B, H, T, d_k) . PyTorch réalise le produit séparément pour chaque fenêtre et chaque tête. Il obtient (B, H, T, T) ; les fenêtres d'un lot ne s'échangent donc aucune information. Les têtes ne sont pas non plus mélangées lors de ce produit.

Prenons $q_i = (2, -1)$ et deux clés $(1, 3)$ et $(-2, 0)$. Les scores valent respectivement $2 - 3 = -1$ et $-4 + 0 = -4$. Le premier score est plus grand, bien qu'il soit négatif. Le softmax dépendra de leur différence ; le signe négatif n'interdit pas qu'une position reçoive un poids important.

Avec un cache de longueur p , seules les nouvelles requêtes sont calculées pour cet appel, tandis que les clés anciennes sont concaténées aux nouvelles. La grille devient rectangulaire, de forme $(B, H, T, p + T)$. Ce changement de forme explique pourquoi un simple triangle carré n'est plus toujours le bon masque.

Point de vigilance. Le calcul construit toutes les comparaisons avant de masquer le futur dans le chemin manuel. Le masque ne supprime donc pas ce coût de construction. Sans cache, la grille de scores contient BHT^2 nombres, une croissance quadratique en longueur.

Exercice. Pour $B = 2$, $H = 4$, $T = 8$ et $d_k = 4$, donnez la forme des scores et leur nombre d'éléments. Que devient ce nombre si T double ?

Correction détaillée. La forme est $(2, 4, 8, 8)$, soit $2 \times 4 \times 64 = 512$ scores. Avec seize positions, on obtient $2 \times 4 \times 256 = 2048$ scores. Doubler les deux axes positionnels multiplie le nombre par quatre, sans changer le nombre de paramètres appris.

Fiche 53 — L'exemple minuscule, sans masque

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

tiny_numeric_example

Objectif. Refaire exactement le calcul présenté dans le script, en conservant des expressions exactes plutôt que des décimales approximatives.

À comprendre. La fonction `tiny_numeric_example` appelle l'attention sans fournir de masque. Les deux positions peuvent donc utiliser les deux valeurs. Cette démonstration numérique n'est pas, à elle seule, une attention causale de modèle de langage.

Le script crée un lot de taille un contenant deux requêtes, deux clés et deux valeurs. En omettant seulement cet axe de lot dans l'écriture, les données et les scores divisés sont

$$Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad K = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}, \quad V_{\text{att}} = \begin{pmatrix} 10 & 0 \\ 0 & 5 \end{pmatrix}, \quad \frac{QK^\top}{\sqrt{2}} = \begin{pmatrix} a & a \\ a & 0 \end{pmatrix}, \quad a = \frac{1}{\sqrt{2}}.$$

La première requête sélectionne la première coordonnée des clés : elle produit deux scores identiques. La seconde sélectionne leur deuxième coordonnée : elle distingue donc les deux clés. La division utilise $\sqrt{2}$, puisque les requêtes ont deux coordonnées.

Le softmax s'applique séparément à chaque ligne. Dans la première, les deux exponentielles sont égales, donc les poids valent exactement une moitié. Dans la seconde, posons $r = e^a/(e^a + 1)$. Comme a est positif, r dépasse une moitié, mais reste inférieur à un.

$$A = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} \\ r & 1 - r \end{pmatrix}, \quad AV_{\text{att}} = \begin{pmatrix} 5 & \frac{5}{2} \\ 10r & 5(1 - r) \end{pmatrix}.$$

Les coordonnées de sortie sont obtenues par deux moyennes pondérées distinctes, utilisant les mêmes poids sur chaque coordonnée. Par exemple, la première coordonnée de la seconde sortie vaut $r \times 10 + (1 - r) \times 0$. Aucun arrondi n'est nécessaire pour vérifier cette relation.

La fonction renvoie trois objets dans cet ordre : sortie, poids, scores divisés éventuellement masqués. Ses affichages permettent ainsi de remonter du résultat aux étapes intermédiaires. Il faut distinguer cet ordre de retour du module multi-têtes, qui ne renvoie pas les scores.

Point de vigilance. Le softmax de toute la matrice aplatie donnerait un autre résultat : il ferait partager une seule somme égale à un entre toutes les cases. Ici, chacune des deux lignes possède sa propre normalisation.

Exercice. Remplaçons seulement les deux valeurs par $(4, 2)$ et $(4, 2)$. Les poids changent-ils ? Calculez les deux sorties sans recalculer les exponentielles.

Correction détaillée. Les poids restent identiques, car ils dépendent seulement de Q et K . Chaque ligne combine maintenant deux vecteurs égaux avec des coefficients dont la somme vaut un. Les deux sorties valent donc exactement $(4, 2)$, même si leurs distributions d'attention diffèrent.

Fiche 54 — Interdire le futur avant le softmax

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

causal_mask

Objectif. Construire le masque causal et comprendre pourquoi il intervient avant la normalisation, pas après.

À comprendre. À la position i , MiniGPT peut utiliser les positions $j \leq i$. La position courante est autorisée, car le token fourni à cet endroit sert à prédire le suivant. Seules les colonnes strictement futures sont interdites.

La fonction `causal_mask` construit un triangle supérieur avec `diagonal=1`, puis le convertit en booléens. Dans le chemin manuel, `True` signifie *interdit*. Le score correspondant est remplacé par $-\infty$ avant le softmax ; son exponentielle vaut alors zéro.

$$M_{ij} = (j > i), \quad M = \begin{pmatrix} F & V & V \\ F & F & V \\ F & F & F \end{pmatrix} \quad (T = 3),$$

où V signifie vrai et F faux. La première position ne peut donc utiliser qu'elle-même ; la deuxième peut utiliser les deux premières. Aucune ligne n'est entièrement interdite dans ce masque carré.

Reprenons exactement les données de la fiche précédente, mais fournissons cette fois le masque causal de taille deux. Le score en haut à droite devient $-\infty$. Avec les mêmes $a = 1/\sqrt{2}$ et $r = e^a/(e^a + 1)$, on obtient

$$S_{\text{masqué}} = \begin{pmatrix} a & -\infty \\ a & 0 \end{pmatrix}, \quad A = \begin{pmatrix} 1 & 0 \\ r & 1-r \end{pmatrix}, \quad O = \begin{pmatrix} 10 & 0 \\ 10r & 5(1-r) \end{pmatrix}.$$

Seule la première sortie change. Cette observation constitue une vérification locale de causalité : la valeur future ne participe plus à son calcul. Elle ne prétend pas que tout programme utilisant cette fonction sera automatiquement correct.

Masquer les poids après le softmax, sans les renormaliser, laisserait ici une première ligne $(1/2, 0)$. Sa somme serait une moitié, et la sortie $(5, 0)$ serait erronée. Le placement du masque garantit que toute la masse est redistribuée sur les positions autorisées.

Le même masque est diffusé sur les fenêtres et les têtes : il décrit une règle temporelle commune, pas un ensemble de paramètres que chaque tête devrait apprendre à respecter.

Point de vigilance. La fonction générale accepte un masque fourni par l'appelant. Une ligne entièrement masquée peut produire des valeurs non numériques : aucune distribution ne peut normaliser uniquement des exponentielles nulles. La convention booléenne de SDPA sera différente.

Exercice. Un cache contient trois positions et l'appel ajoute deux tokens. Pour les requêtes globales trois et quatre, quelles colonnes parmi zéro à quatre sont interdites ?

Correction détaillée. La requête trois interdit seulement la colonne quatre. La requête quatre n'interdit aucune colonne. Le masque manuel rectangulaire a donc les lignes (F, F, F, F, V) et cinq fois faux. Le décalage du cache est indispensable pour obtenir ces autorisations.

Fiche 55 — Pourquoi diviser par la racine de la dimension ?

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

scaled_dot_product_attention

Objectif. Relier le facteur $1/\sqrt{d_k}$ à un calcul de variance accessible, en explicitant les hypothèses qui le rendent valable.

À comprendre. Un produit scalaire additionne d_k contributions. Même si chaque contribution reste modérée, la dispersion de la somme tend à augmenter avec le nombre de coordonnées. La division réduit cette dépendance à la largeur des têtes.

Faisons un modèle probabiliste simplifié, pas une description exacte des activations entraînées. Supposons les coordonnées des requêtes et des clés indépendantes, centrées, de variance un ; supposons aussi les produits de coordonnées indépendants entre eux. Alors chaque produit $q_r k_r$ est centré et sa variance vaut un.

$$S = \sum_{r=0}^{d_k-1} q_r k_r, \quad \text{Var}(S) = d_k, \quad \text{Var}\left(\frac{S}{\sqrt{d_k}}\right) = 1.$$

La dernière égalité utilise une règle simple : multiplier une variable par un nombre multiplie sa variance par le carré de ce nombre. L'écart-type de S vaut donc $\sqrt{d_k}$, et non d_k . C'est ce qui motive la racine au dénominateur.

Pourquoi la dispersion des scores importe-t-elle ? Le softmax compare des exponentielles. Pour deux scores dont la différence vaut Δ , le rapport des poids vaut e^Δ . Une différence importante peut concentrer presque tout le poids sur une seule case, même au début de l'apprentissage. La division modère cet effet ; elle ne garantit pas une attention uniforme.

Avec $d_k = 4$, le code divise par deux. Avec $d_k = 64$, il divise par huit. Si deux scores bruts diffèrent de huit, les rapports de poids après division sont respectivement e^4 et e^1 . Ces expressions exactes suffisent à comparer leur concentration sans annoncer une mesure expérimentale.

Dans le code, `d_k = q.size(-1)` lit la dimension réellement présente, puis `math.sqrt(d_k)` calcule le facteur. Cette méthode reste valable pour une tête et pour plusieurs têtes déjà séparées.

Point de vigilance. Après projection et apprentissage, les coordonnées peuvent être corrélées et leurs variances différentes de un. La démonstration explique une règle de mise à l'échelle ; elle ne prouve pas que tous les scores réels ont une variance exactement égale à un.

Exercice. Sous les hypothèses précédentes, prenons $d_k = 16$. Quelle variance obtiendrait-on en divisant par seize au lieu de quatre ? Pourquoi serait-ce une autre règle ?

Correction détaillée. La variance brute vaut seize. Diviser par seize donnerait $16/16^2 = 1/16$, tandis que diviser par quatre donne un. La première règle réduit de plus en plus la dispersion quand la dimension augmente ; elle ne cherche donc pas la même stabilité d'échelle.

Fiche 56 — Les poids combinent les valeurs

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

scaled_dot_product_attention

Objectif. Interpréter le produit final comme une moyenne pondérée de vecteurs et identifier les limites exactes de cette interprétation.

À comprendre. Après masquage et softmax, les poids d’une ligne sont positifs ou nuls et leur somme vaut un, à l’arrondi numérique près. La sortie correspondante est une combinaison convexe des valeurs autorisées.

Le terme *convexe* signifie ici que les coefficients forment une moyenne : aucune valeur n’est multipliée par un coefficient négatif et la masse totale vaut un. Si A contient les poids et V_{att} les valeurs, le code réalise

$$O = AV_{\text{att}}, \quad o_i = \sum_j A_{ij}v_j, \quad A_{ij} \geq 0, \quad \sum_j A_{ij} = 1.$$

Toutes les coordonnées d’un vecteur utilisent la même ligne de poids. On ne choisit donc pas indépendamment une position source pour chaque coordonnée. Cette contrainte maintient le résultat dans l’ensemble des mélanges possibles des vecteurs sources.

Pour deux valeurs $(10, 0)$ et $(0, 5)$, des poids $(1/4, 3/4)$ donnent $(5/2, 15/4)$. La première coordonnée reste entre zéro et dix ; la seconde entre zéro et cinq. Plus généralement, chaque coordonnée de la sortie se situe entre le minimum et le maximum de cette coordonnée parmi les valeurs autorisées.

Cependant, cette propriété porte sur les valeurs *projetées* et sur la sortie d’une tête avant les opérations suivantes. Elle ne dit pas que la sortie finale du bloc reste entre les embeddings initiaux. La projection de sortie, son biais, la connexion résiduelle, la normalisation et le réseau feed-forward modifient ensuite le vecteur.

Elle ne signifie pas non plus que l’attention calcule une probabilité de vérité. Les poids organisent un mélange interne. Deux distributions différentes peuvent produire le même mélange, notamment lorsque les valeurs sont identiques. Une carte d’attention ne suffit donc pas à expliquer toutes les causes d’une prédiction.

Point de vigilance. Dans cette implémentation, aucun dropout n’est appliqué aux poids du softmax. Le dropout arrive après la fusion des têtes. Une variante appliquant du dropout aux probabilités ne conserverait pas nécessairement une somme égale à un pour chaque réalisation aléatoire.

Exercice. Deux valeurs scalaires valent -2 et 6 . Peut-on obtenir dix par cette moyenne ? Trouvez les poids donnant quatre, puis expliquez comment une projection ultérieure peut néanmoins produire dix.

Correction détaillée. Dix est hors de l’intervalle $[-2, 6]$, donc impossible à ce stade. Les poids $1/4$ et $3/4$ donnent $-2/4 + 18/4 = 4$. Une transformation affine ultérieure telle que $z \mapsto 2z + 2$ transforme quatre en dix : la borne ne survit pas à toutes les opérations du module.

Fiche 57 — Découper les coordonnées en têtes

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

MultiHeadSelfAttention.forward

Objectif. Suivre les opérations `view` et `transpose` sans confondre découpage des coordonnées et découpage de la séquence.

À comprendre. Les têtes traitent les mêmes positions, mais utilisent des groupes différents de coordonnées projetées. Chaque tête calcule sa propre grille de scores et ses propres mélanges de valeurs.

MiniGPT exige que D soit divisible par H . La dimension d'une tête vaut $d_k = D/H$. Chaque projection produit d'abord (B, T, D) , puis le code réorganise les deux derniers axes :

$$(B, T, D) \xrightarrow{\text{view}} (B, T, H, d_k) \xrightarrow{\text{transpose}(1,2)} (B, H, T, d_k).$$

Ces opérations ne créent pas de nouveaux paramètres et ne calculent pas de moyennes. Elles changent la manière d'interpréter et de parcourir les coordonnées déjà présentes. La transposition met l'axe des têtes avant celui des positions pour que les produits matriciels utilisent les deux derniers axes.

Prenons un token dont le vecteur projeté est $(0, 1, 2, 3, 4, 5, 6, 7)$ avec $D = 8$ et $H = 4$. Les quatre têtes reçoivent respectivement $(0, 1)$, $(2, 3)$, $(4, 5)$ et $(6, 7)$. Chacune reçoit ce découpage pour *tous* les tokens, pas seulement pour un quart de la fenêtre.

Les projections sont apprises avant le découpage. Une coordonnée attribuée à une tête peut donc dépendre de toutes les coordonnées de l'entrée, grâce à la matrice dense. Il serait trompeur de penser que la première tête ne voit que les premiers caractères ou seulement les premières coordonnées de l'embedding initial.

Le nombre de paramètres des trois projections reste $3(D^2 + D)$ quand on change uniquement H . En revanche, le nombre de grilles de scores augmente avec les têtes, tandis que leur dimension interne diminue. Des têtes supplémentaires ne constituent donc pas une multiplication indépendante de toute la largeur du modèle.

Point de vigilance. Les paramètres positifs doivent être des entiers véritables : le constructeur rejette notamment les booléens. En mode RoPE, une contrainte supplémentaire impose que d_k soit pair, car les coordonnées seront utilisées par paires.

Exercice. L'entrée a la forme $(2, 6, 8)$ et l'on choisit quatre têtes. Donnez les formes de Q après chaque réorganisation et des poids d'attention. Peut-on choisir trois têtes sans modifier D ?

Correction détaillée. Après `view`, la forme est $(2, 6, 4, 2)$; après transposition, $(2, 4, 6, 2)$. Les poids ont la forme $(2, 4, 6, 6)$. Trois têtes sont refusées, car huit n'est pas divisible par trois : aucun découpage en groupes de taille entière identique n'existe.

Fiche 58 — Fusionner puis projeter : l'ordre exact

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

MultiHeadSelfAttention.out_proj

Objectif. Reconstituer la sortie multi-têtes et localiser précisément le dropout de cette implémentation.

À comprendre. Après le produit des poids par les valeurs, la sortie possède la forme (B, H, T, d_k) . La fusion remet côte à côte les vecteurs des têtes appartenant au même token, afin de retrouver une largeur D .

Le code transpose d'abord les axes des têtes et des positions. Il obtient une vue de forme (B, T, H, d_k) . Cette vue peut parcourir la mémoire dans un ordre non contigu. L'appel `contiguous()` garantit alors une disposition compatible avec le `view` final, qui rassemble les deux derniers axes.

```
output = output.transpose(1, 2).contiguous().view(
    batch_size, sequence_length, self.embedding_dim
)
output = self.out_proj(self.dropout(output))
```

Ces lignes reformattent l'extrait du dépôt sans changer son ordre. La concaténation implicite ne somme pas les têtes : avec deux sorties $(1, 2)$ et $(3, 4)$, le vecteur fusionné est $(1, 2, 3, 4)$. La projection de sortie peut ensuite mélanger toutes ces coordonnées.

Point essentiel : le dropout arrive *avant* `out_proj`. Pour un taux ρ strictement inférieur à un, l'entraînement conserve aléatoirement certaines coordonnées et les multiplie par $1/(1 - \rho)$. En évaluation, le module dropout restitue son entrée. Le biais de la projection est ajouté après ce masquage.

$$y = \text{Linear}_{\text{out}}(\text{Dropout}(z)), \quad z \in \mathbb{R}^{B \times T \times D}.$$

Déplacer le dropout après la projection conserve les formes, mais change généralement le résultat. Avant projection, supprimer une coordonnée peut modifier plusieurs coordonnées de sortie. Après projection, supprimer une coordonnée de sortie ne retire pas de la même façon sa contribution aux autres.

Cette distinction concerne aussi les comparaisons entre chemins manuel et SDPA : conserver la place du dropout permet de comparer la même architecture, plutôt que deux architectures presque semblables.

Point de vigilance. La fusion ne produit pas encore la sortie du bloc Transformer. Le bloc doit ajouter l'entrée résiduelle puis normaliser. Ici, `out_proj` possède une matrice (D, D) et un biais de longueur D , soit $D^2 + D$ paramètres.

Exercice. Prenons $z = (2, 4)$, un dropout de taux une moitié qui conserve seulement la première coordonnée, puis la projection $y = (z_1 + z_2, z_1 - z_2)$ sans biais. Calculez la sortie.

Correction détaillée. Le dropout fournit $(4, 0)$, car la coordonnée conservée est doublée. La projection produit donc $(4, 4)$. Sans dropout, elle aurait fourni $(6, -2)$. Appliquer ensuite le même motif de dropout à cette autre sortie aurait donné $(12, 0)$, résultat différent.

Fiche 59 — Auditer le parcours de l'attention

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

MultiHeadSelfAttention.forward

Objectif. Lire un `forward` comme un contrat de transformations et de retours, plutôt que comme une suite opaque d'instructions.

À comprendre. Le résultat dépend de trois choix distincts : l'encodage positionnel, le moteur de calcul de l'attention et la demande éventuelle de cache ou de poids. Ces choix doivent être suivis sans perdre les axes des tenseurs.

Partons d'une entrée (B, T, D) . Les trois projections et le découpage produisent des requêtes, clés et valeurs (B, H, T, d_k) . Si un cache est fourni, ses caractéristiques sont vérifiées. En mode RoPE, les requêtes et les nouvelles clés sont tournées avec le décalage approprié. Les anciennes clés, déjà tournées, sont ensuite concaténées aux nouvelles, comme les valeurs.

Le masque dépend de la présence d'un passé non vide. Sans passé, il est causal carré ; avec un passé de longueur p , il autorise les clés dont la position globale ne dépasse pas celle de la requête. Le calcul donne une sortie par tête, puis fusion, dropout et projection rendent (B, T, D) .

sortie : (B, T, D) , poids : $(B, H, T, p + T)$, $K_{\text{présent}}, V_{\text{présent}}$: $(B, H, p + T, d_k)$.

Ces formes rendent les retours faciles à vérifier. Sans option, le module renvoie seulement la sortie. Avec `return_attention=True`, il renvoie sortie puis poids. Avec `use_cache=True`, il renvoie sortie puis paire clé-valeur. Avec les deux options, l'ordre est sortie, poids, paire clé-valeur.

Demander les poids force le chemin manuel, même si `attention_backend` vaut `'sdpa'`. La branche SDPA utilisée ici ne fournit pas ces poids. Le choix explicite permet donc de les inspecter, mais peut changer les calculs intermédiaires et leur coût mémoire.

Consommer un cache et demander son retour sont également deux décisions différentes. Le paramètre `past_key_value` fournit le passé ; `use_cache` décide si le passé enrichi doit être rendu.

Point de vigilance. L'option `return_attention` appartient au module d'attention. Ni `TransformerBlock.forward` ni `MiniGPT.forward` ne l'exposent directement. Ne supposez pas qu'un argument accepté à un niveau traverse automatiquement tous les autres.

Exercice. On appelle une attention en évaluation avec $B = 2$, $H = 4$, $D = 16$, deux nouveaux tokens et trois tokens en cache, en demandant poids et cache. Combien d'objets sont renvoyés, et de quelles formes ?

Correction détaillée. Trois objets sont renvoyés : la sortie $(2, 2, 16)$, les poids $(2, 4, 2, 5)$, puis une paire dont chaque tenseur vaut $(2, 4, 5, 4)$. Le dernier objet contient deux tenseurs, mais reste un seul élément du tuple principal. Les deux nouvelles requêtes ont accès à cinq clés au maximum.

Fiche 60 — Le réseau feed-forward et GELU

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/04_transformer.py

FeedForward

Objectif. Comprendre le calcul effectué à chaque position après l'attention et donner une première lecture probabiliste de GELU.

À comprendre. Le réseau feed-forward transforme les coordonnées d'un token sans mélanger directement les positions. Les mêmes deux couches affines et la même activation sont appliquées à chaque vecteur de la séquence.

Le premier `nn.Linear` transforme une largeur D en une largeur F . GELU agit séparément sur chacune de ces F coordonnées. Le second `nn.Linear` ramène la largeur à D , puis le dropout est appliqué. C'est exactement l'ordre du `nn.Sequential` du dépôt.

$$u = xW_1^\top + b_1, \quad \text{FFN}(x) = \text{Dropout}\left(\text{GELU}(u)W_2^\top + b_2\right).$$

Pourquoi insérer une activation ? Sans elle, la composition de deux applications affines resterait affine. Augmenter la dimension intermédiaire ne suffirait pas à créer la non-linéarité attendue. GELU introduit une réponse qui dépend autrement de la valeur de chaque coordonnée.

Pour la comprendre, imaginons une variable aléatoire Z suivant une loi normale centrée réduite, la courbe en cloche de moyenne zéro et d'écart-type un. Notons $\Phi(u)$ la probabilité que Z soit inférieur ou égal à u . Alors

$$\text{GELU}(u) = u\Phi(u), \quad \Phi(0) = \frac{1}{2}.$$

Le coefficient $\Phi(u)$ est compris entre zéro et un : les grandes valeurs positives sont peu atténuées, les valeurs très négatives le sont beaucoup. Cette lecture ne signifie pas que GELU tire une variable aléatoire à chaque appel ; l'activation elle-même est déterministe.

Par symétrie de la loi normale, $\Phi(-u) = 1 - \Phi(u)$. On en déduit notamment que GELU peut produire des valeurs négatives, contrairement à une activation qui remplacerait toutes les entrées négatives par zéro. Elle ne transforme pas non plus le vecteur complet en distribution de probabilités.

Point de vigilance. Le dropout de ce FFN se trouve après la deuxième couche linéaire, et non entre les deux couches. Les exemples classiques d'autres bibliothèques ne remplacent pas la lecture de cette séquence précise.

Exercice. Pour $D = 16$ et $F = 32$, comptez les paramètres du FFN, biais compris. Puis, en posant $\Phi(1) = p$, exprimez GELU en un et en moins un.

Correction détaillée. La première couche possède $16 \times 32 + 32 = 544$ paramètres ; la seconde $32 \times 16 + 16 = 528$, soit 1072 au total. GELU et le dropout n'ajoutent aucun paramètre. Les activations demandées valent p et $-(1 - p)$. Leur différence vaut exactement un, sans qu'il soit nécessaire de connaître une approximation décimale de p .

Fiche 61 — La connexion résiduelle et le gradient

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/04_transformer.py

TransformerBlock.forward

Objectif. Comprendre ce que change l'addition d'une entrée à sa transformation, puis distinguer cet avantage de ses limites dans un bloc post-normalisé.

À comprendre. Une connexion résiduelle additionne deux tenseurs de même forme. Elle permet à une sous-couche d'apprendre une modification de la représentation plutôt qu'un remplacement complet de celle-ci.

Le bloc calcule d'abord $\mathbf{x} + \text{attn_output}$, puis, après normalisation, $\mathbf{x} + \text{ffn_output}$. Les sorties d'attention et de FFN ont donc nécessairement la même largeur D que le chemin qui les contourne. Aucune concaténation n'a lieu : la largeur reste inchangée.

Pour saisir le rôle de l'addition dans la rétropropagation, prenons un cas scalaire sans normalisation. Si f est une fonction dérivable et si ℓ mesure une erreur en sortie, alors

$$y = x + f(x), \quad \frac{dy}{dx} = 1 + f'(x), \quad \frac{d\ell}{dx} = \frac{d\ell}{dy}(1 + f'(x)).$$

Le terme un vient du chemin direct. Si la dérivée de f est petite, ce chemin peut conserver une contribution que la seule composition de nombreuses transformations aurait fortement réduite. Pour des vecteurs, le rôle du nombre un est joué par l'identité.

Un exemple élémentaire suffit : avec $f(x) = 0,1x$, la transformation seule multiplie localement les variations par 0,1, tandis que la transformation résiduelle les multiplie par 1,1. Il s'agit d'une comparaison locale, pas d'une promesse que multiplier de nombreux facteurs 1,1 serait toujours souhaitable.

Dans MiniGPT, la normalisation intervient après l'addition. Le résultat réel est donc $\text{LayerNorm}(x + f(x))$. Le gradient traverse aussi cette normalisation ; on ne peut pas considérer que le bloc entier possède un passage identité totalement intact.

Point de vigilance. Les résidus ne garantissent ni l'absence de gradients nuls ni la stabilité universelle. Si $f'(x) = -1$ dans l'exemple scalaire, la dérivée de la somme est nulle. Leur utilité tient à la structure de calcul qu'ils offrent, pas à une impossibilité mathématique d'échouer.

Exercice. Une représentation vaut $(1, 2)$ et la sous-couche produit $(3, -1)$. Quelle somme entre dans la normalisation ? Si la sous-couche produit plutôt zéro partout, le résultat final du bloc est-il nécessairement l'entrée ?

Correction détaillée. La somme vaut $(4, 1)$. Avec une sortie de sous-couche nulle, la somme vaut bien $(1, 2)$, mais la LayerNorm qui suit transforme généralement ce vecteur. Ainsi, sous-couche nulle ne signifie pas bloc identité dans cette architecture post-normalisée ; il faut lire toute la chaîne, pas seulement le symbole plus.

Fiche 62 — LayerNorm : normaliser un token

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/04_transformer.py

TransformerBlock.norm_1

Objectif. Calculer une LayerNorm à la main et identifier ses axes, son epsilon et ses paramètres entraînaibles.

À comprendre. Avec `nn.LayerNorm(embedding_dim)`, la moyenne et la variance sont calculées sur les D coordonnées de chaque token. Elles ne sont pas calculées sur toutes les positions ni sur toutes les fenêtres du lot.

Pour un vecteur $x = (x_1, \dots, x_D)$, PyTorch utilise une variance dont le diviseur est D , parfois appelée variance biaisée dans le vocabulaire statistique. L'objectif est de normaliser les coordonnées présentes, pas d'estimer sans biais une variance inconnue à partir d'un échantillon.

$$\mu = \frac{1}{D} \sum_{r=1}^D x_r, \quad \sigma^2 = \frac{1}{D} \sum_{r=1}^D (x_r - \mu)^2, \quad y_r = \gamma_r \frac{x_r - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta_r.$$

Les réglages par défaut utilisés ici donnent $\varepsilon = 10^{-5}$. Les vecteurs γ et β , de longueur D , sont entraînaibles : une normalisation possède donc $2D$ paramètres. Initialement, γ vaut un et β zéro. Les statistiques sont recalculées à chaque passage, aussi en évaluation.

Prenons $x = (1, 3)$. Sa moyenne vaut deux et sa variance vaut $((1 - 2)^2 + (3 - 2)^2)/2 = 1$. Avant l'affinité finale, les coordonnées deviennent $-1/\sqrt{1 + \varepsilon}$ et $1/\sqrt{1 + \varepsilon}$. Elles sont proches de moins un et de un, mais pas exactement égales à ces nombres lorsque l'epsilon est conservé.

L'epsilon évite notamment de diviser par zéro pour un vecteur constant. Avec $(3, 3)$, la partie centrée est nulle ; la sortie vaut donc β . La présence de γ et β signifie aussi que les sorties finales n'ont pas nécessairement moyenne zéro et variance un après apprentissage.

À la différence d'une BatchNorm usuelle, cette LayerNorm ne calcule pas ses statistiques en réunissant plusieurs exemples du lot et ne maintient pas de moyennes courantes pour l'évaluation. Une fenêtre ne modifie donc pas la normalisation d'une autre fenêtre.

Sur un tenseur $(2, 8, 16)$, on effectue ainsi seize normalisations distinctes, chacune sur seize coordonnées. Les paramètres γ et β restent pourtant partagés entre ces seize vecteurs : statistiques locales et paramètres communs ne se contredisent pas.

Point de vigilance. Remplacer le diviseur D par $D - 1$, ou oublier l'epsilon, empêche de reproduire exactement le module. Ces différences deviennent particulièrement visibles quand le nombre de coordonnées est petit ou que leur variance est presque nulle.

Exercice. Pour $(1, 3)$, prenons $\gamma = (2, 1)$ et $\beta = (1, -1)$. Donnez la sortie exacte et le nombre de paramètres d'une LayerNorm de largeur seize.

Correction détaillée. La sortie est $(1 - 2/\sqrt{1 + \varepsilon}, -1 + 1/\sqrt{1 + \varepsilon})$. Une largeur seize donne seize coefficients multiplicatifs et seize biais, soit 32 paramètres. La moyenne et la variance calculées ne sont pas des paramètres supplémentaires.

Fiche 63 — Le bloc est post-normalisé

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/04_transformer.py

TransformerBlock

Objectif. Écrire les deux équations du bloc réel et comprendre pourquoi déplacer une normalisation change l'architecture.

À comprendre. Dans un bloc post-normalisé, chaque normalisation vient après l'addition résiduelle. MiniGPT suit cet ordre pour l'attention puis pour le réseau feed-forward. Il ne s'agit pas d'un bloc pré-normalisé.

Notons $\mathcal{A}$ le module complet d'attention, comprenant fusion des têtes, dropout de sortie et projection. Notons $\mathcal{F}$ le FFN complet, comprenant son dropout final. Pour l'entrée X , les deux étapes du dépôt s'écrivent

$$U = \text{LN}_1(X + \mathcal{A}(X)), \quad Y = \text{LN}_2(U + \mathcal{F}(U)).$$

Les tenseurs X , U et Y ont tous la forme (B, T, D) . La seconde sous-couche reçoit U , déjà normalisé après l'attention, et non l'entrée originale X . Les deux LayerNorm possèdent leurs propres paramètres.

Un bloc pré-normalisé ferait plutôt apparaître une sous-couche appliquée à une entrée normalisée avant l'addition. Même si les composants portent les mêmes noms et si les formes restent identiques, ces compositions ne sont pas interchangeables : en général, normaliser une somme n'est pas additionner un vecteur à une transformation d'entrée normalisée.

Pour le constater sans calcul compliqué, annulons les deux sorties de sous-couches. Ici, le bloc fournit alors $\text{LN}_2(\text{LN}_1(X))$, généralement différent de X . Un schéma pré-normalisé avec deux branches nulles conserverait au contraire l'entrée par ses additions résiduelles.

Le code ne place pas de dropout supplémentaire juste après chacune des sommes. Les dropouts sont déjà à l'intérieur des sous-modules, avec leurs positions propres. Il faut donc éviter de compléter mentalement ce bloc à partir d'un diagramme de Transformer provenant d'une autre implémentation.

La causalité traverse le bloc : l'attention respecte le masque, puis le FFN et les normalisations agissent séparément à chaque position. Ces dernières opérations peuvent transformer l'information disponible, mais n'introduisent pas de lecture des tokens futurs.

Point de vigilance. La présence d'une LayerNorm finale dans MiniGPT ne transforme pas les blocs en pré-normalisation. Il faut examiner l'emplacement des normalisations à l'intérieur de chaque bloc pour nommer correctement l'architecture.

Exercice. Pour $D = 16$, combien de paramètres appartiennent aux deux normalisations d'un bloc ? Si l'attention reçoit $(2, 8, 16)$, quelle forme la première addition exige-t-elle en sortie ?

Correction détaillée. Chaque normalisation possède $2D = 32$ paramètres, donc 64 pour les deux. La sortie d'attention doit avoir la forme $(2, 8, 16)$ pour l'addition coordonnée par coordonnée. Ses poids d'attention, de forme $(2, H, 8, 8)$, ne peuvent pas remplacer cette sortie : ils décrivent une autre étape du calcul.

Fiche 64 — Traverser MiniGPT de bout en bout

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/05_model.py

MiniGPT.forward

Objectif. Garder une carte des formes depuis les identifiants jusqu’aux scores de vocabulaire et comprendre le rôle des blocs enregistrés.

À comprendre. MiniGPT combine des tables d’embeddings, une liste de blocs Transformer, une normalisation finale et une tête de prédiction. Les identifiants d’entrée ne sont jamais directement traités comme des valeurs continues à prédire.

Pour un appel sans cache, l’entrée est une matrice d’entiers (B, T) , avec une séquence non vide et $T \leq C$. La table des tokens produit (B, T, D) . En mode appris, les positions $(1, T, D)$ s’ajoutent par diffusion ; en mode RoPE, cette addition est absente. Le dropout agit ensuite sur les représentations obtenues.

$$(B, T) \longrightarrow (B, T, D) \xrightarrow{L \text{ blocs}} (B, T, D) \xrightarrow{\text{LayerNorm puis tête}} (B, T, V).$$

Chaque bloc conserve la forme, mais pas le contenu. Le premier peut construire des combinaisons à partir des embeddings ; les suivants travaillent sur des représentations déjà transformées et contextualisées. Conserver une dimension ne signifie donc pas répéter une opération identique.

Les blocs sont rangés dans `nn.ModuleList`. Cette structure enregistre leurs paramètres auprès du modèle, pour qu’ils figurent dans `parameters()` et `state_dict()`. La boucle de `forward` applique explicitement chaque bloc. Les blocs ont la même architecture, mais leurs paramètres sont distincts.

Avec la petite configuration des tests, $B = 2$, $T = 8$, $D = 16$, $L = 2$ et $V = 23$. Les deux blocs reçoivent et rendent chacun $(2, 8, 16)$. La normalisation finale conserve cette forme, puis la tête fournit $(2, 8, 23)$, soit 368 scores. Ces scores sont des activations dépendant des entrées.

L’option `verbose` affiche les dimensions au niveau des embeddings, après chaque bloc et à la sortie. Elle aide à localiser une erreur de forme, mais ne demande ni les cartes d’attention ni un diagnostic de qualité linguistique.

Point de vigilance. Le modèle valide explicitement le rang et la longueur de l’entrée. Les identifiants doivent aussi être compatibles avec une table d’embeddings : des indices valides, dans le vocabulaire, et un type entier accepté par PyTorch. Une forme correcte ne suffit pas.

Exercice. Si l’on passe de deux blocs à quatre en conservant toutes les autres dimensions, quelles formes changent ? Le nombre de scores finaux double-t-il ?

Correction détaillée. Aucune forme extérieure ne change : l’entrée reste $(2, 8)$, les représentations $(2, 8, 16)$ et la sortie $(2, 8, 23)$. Deux transformations supplémentaires sont exécutées et leurs paramètres s’ajoutent. Le nombre de scores finaux reste 368 ; profondeur du calcul et largeur de la sortie sont deux choix distincts.

Fiche 65 — La tête produit des logits, pas des probabilités

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/05_model.py

MiniGPT.lm_head

Objectif. Distinguer les scores du vocabulaire de leur normalisation et comprendre l'aplatissement utilisé pour calculer la perte.

À comprendre. La tête de langage est une couche affine de largeur d'entrée D et de largeur de sortie V . Elle attribue un score réel à chaque token possible, séparément pour chaque position et chaque fenêtre.

Après `final_norm`, la couche `lm_head` calcule un vecteur de logits. Ils peuvent être négatifs et leur somme n'est pas fixée. Pour une position donnée, on pourrait les convertir en probabilités avec

$$z = hW_{\text{lm}}^{\top} + b_{\text{lm}}, \quad p_j = \frac{e^{z_j}}{\sum_{k=0}^{V-1} e^{z_k}}.$$

MiniGPT ne réalise pas ce softmax dans le retour de `forward`. Les logits bruts permettent notamment de confier à la fonction de perte un calcul numériquement stable, sans former auparavant des probabilités susceptibles d'être extrêmement petites.

Lorsque `targets` est fourni, le code appelle `F.cross_entropy`. Cette fonction attend ici une matrice comportant un exemple par ligne et une cible entière par exemple. Le modèle réunit donc les axes du lot et du temps :

$$(B, T, V) \longrightarrow (BT, V), \quad (B, T) \longrightarrow (BT).$$

Les `reshape` conservent l'ordre correspondant des positions. Avec $B = 2$ et $T = 8$, la perte compare seize distributions implicites à seize identifiants cibles. Avec la réduction par défaut et des cibles ordinaires valides, elle moyenne les pertes de ces positions.

Les cibles doivent déjà désigner les tokens à prédire. Cette méthode ne décale pas automatiquement la séquence : le décalage entrée-cible relève de la construction des données. Un appel utilisant les mêmes tokens comme entrée et cible peut vérifier une formule dans un test sans représenter l'objectif habituel de prédiction du prochain token.

Si aucune cible n'est fournie, `loss` reste `None`. Sans demande de cache, le retour conserve pourtant deux éléments, `(logits, loss)` : l'absence de perte ne transforme pas le contrat en retour d'un seul tenseur.

Point de vigilance. Un score élevé indique une préférence relative du modèle, pas une certitude factuelle. Ajouter la même constante à tous les logits d'une position laisse leurs probabilités inchangées.

Exercice. Pour un vocabulaire de deux tokens, les logits valent $(0, \log 3)$ et la cible est le second token. Calculez sa probabilité et la perte correspondante.

Correction détaillée. Les exponentielles valent un et trois : la probabilité cible est $3/4$. La perte vaut $-\log(3/4) = \log(4/3)$. Si la cible était le premier token, elle vaudrait $\log 4$. Le softmax intervient conceptuellement dans ce calcul, mais le code transmet directement les logits à l'entropie croisée.

Fiche 66 — Initialiser : écart-type et poids indépendants

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/05_model.py

MiniGPT._init_weights

Objectif. Lire les paramètres d’initialisation sans confondre variance et écart-type, et repérer les matrices qui ne sont pas partagées.

À comprendre. Une initialisation définit les valeurs de départ des paramètres, avant tout apprentissage. Dans MiniGPT, la méthode `_init_weights` traite les couches linéaires et les tables d’embeddings.

Le constructeur termine par `self.apply(self._init_weights)`, ce qui visite récursivement les sous-modules. Pour chaque `nn.Linear` ou `nn.Embedding`, les poids sont tirés avec une moyenne nulle et `std=0.02`. Les biais des couches linéaires sont mis à zéro.

$$\mathbb{E}[W_{ij}] = 0, \quad \text{écart-type}(W_{ij}) = 0,02, \quad \text{Var}(W_{ij}) = (0,02)^2 = 0,0004.$$

Écrire seulement une loi normale avec deux nombres sans préciser la convention peut induire en erreur : certains livres utilisent la variance comme second paramètre. Ici, le nom de l’argument Python tranche. Il s’agit bien d’un écart-type de deux centièmes.

Cette règle concerne aussi les projections internes de chaque bloc et les deux couches du FFN. Elle ne réinitialise pas les LayerNorm, dont les paramètres gardent leurs valeurs par défaut : poids multiplicatifs à un et biais à zéro. Les modules dropout et GELU n’ont pas de paramètres à tirer.

La table des tokens et la matrice de la tête de langage ont toutes deux une forme (V, D) , mais elles ne sont pas liées. Le constructeur crée séparément `token_embedding` et `lm_head`, sans réaffecter leurs poids à un même paramètre. L’entrée et la sortie possèdent donc chacune leurs VD nombres appris.

Le biais de sortie existe aussi : `nn.Linear(embedding_dim, vocab_size)` utilise un biais par défaut, de taille V . Le compter reste nécessaire même lorsqu’il est initialement nul ; zéro décrit une valeur, pas l’absence d’un paramètre.

Point de vigilance. Des tirages provenant de la même loi ne donnent pas des matrices identiques. Les moyennes et variances d’une petite matrice effectivement tirée ne sont pas nécessairement exactement celles de la loi théorique. Aucun résultat d’entraînement ne découle à lui seul de cette initialisation.

Exercice. Avec $V = 23$ et $D = 16$, combien de paramètres appartiennent à la table des tokens et à la tête réunies ? Combien en retirerait un partage hypothétique des poids, sans retirer le biais ?

Correction détaillée. La table contient 368 paramètres ; la tête en contient $368 + 23 = 391$. Le total réel est donc 759. Un partage hypothétique supprimerait une matrice indépendante de 368 paramètres, laissant 391 paramètres uniques pour cet ensemble. Ce partage n’est pas réalisé dans MiniGPT.

Fiche 67 — Compter tous les paramètres

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/05_model.py

MiniGPT.count_parameters

Objectif. Établir une formule contrôlable qui inclut les biais et les normalisations, puis l'appliquer à la configuration minuscule des tests.

À comprendre. Compter les paramètres revient à additionner le nombre d'éléments des tenseurs entraînaables enregistrés. Les activations temporaires, les scores d'attention et les caches ne sont pas des paramètres.

Dans un bloc, les projections Q , K , valeurs et sortie constituent quatre couches affines $D \rightarrow D$. Elles totalisent $4D^2 + 4D$ nombres. Les deux couches du FFN apportent $DF + F$ puis $FD + D$. Les deux LayerNorm apportent chacune $2D$.

$$P_{\text{bloc}} = 4D^2 + 2DF + 9D + F, \quad P = 2VD + I_{\text{appris}}CD + LP_{\text{bloc}} + 2D + V.$$

Ici, I_{appris} vaut un pour les positions apprises et zéro pour RoPE. Le terme $2VD$ compte séparément embeddings de tokens et poids de sortie. Le dernier $2D$ correspond à la normalisation finale ; le dernier V au biais de la tête. Ni RoPE ni le choix manuel/SDPA n'ajoutent de matrice entraînable.

Appliquons la formule à $V = 23$, $C = 8$, $D = 16$, $H = 4$, $L = 2$ et $F = 32$, la configuration de base des tests. Un bloc contient 1024 paramètres matriciels d'attention, 1024 paramètres matriciels de FFN, puis $144 + 32 = 176$ autres paramètres : au total 2224.

$$P_{\text{appris}} = 736 + 128 + 2 \times 2224 + 32 + 23 = 5367, \quad P_{\text{RoPE}} = 5367 - 128 = 5239.$$

On peut vérifier autrement : 1088 pour les quatre affinités d'attention, 1072 pour le FFN et 64 pour les deux normalisations donnent bien 2224 par bloc. Cette décomposition est utile pour repérer l'oubli fréquent des biais ou des coefficients multiplicatifs de normalisation.

À largeur D fixe, H n'apparaît pas dans la formule. Changer le découpage des têtes ne change pas la taille des matrices de projection. Cela peut néanmoins changer les calculs, la mémoire des scores et les contraintes de validité.

La méthode `count_parameters` additionne `param.numel()` pour chaque paramètre enregistré. Elle fournit donc un contrôle direct de cette comptabilité. Le nombre de fenêtres et la longueur effectivement traitée n'interviennent pas : réutiliser davantage de fois une même matrice ne crée pas de nouveaux coefficients.

Point de vigilance. Une estimation de mémoire en octets demande aussi le type numérique et les autres objets conservés. Multiplier ce seul total par la taille d'un nombre ne compte ni gradients, ni état d'optimiseur, ni activations.

Exercice. Dans cette configuration avec positions apprises, quel total obtient-on avec trois blocs ? Et avec deux blocs mais $C = 16$?

Correction détaillée. Ajouter un bloc ajoute 2224 paramètres : $5367 + 2224 = 7591$. Doubler seulement le contexte ajoute huit lignes positionnelles de seize nombres, soit 128 paramètres : le total devient 5495. En mode RoPE, cette modification de C n'ajouterait aucun paramètre positionnel appris.

Fiche 68 — RoPE : tourner une paire de coordonnées

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

MultiHeadSelfAttention._apply_rope

Objectif. Comprendre RoPE à partir des rotations du plan étudiées avec le sinus et le cosinus, sans recourir aux nombres complexes.

À comprendre. RoPE encode la position en tournant des paires de coordonnées des requêtes et des clés. Contrairement aux positions apprises, il n'ajoute pas un vecteur positionnel à l'embedding du token.

Pour une paire (a, b) et un angle θ , la rotation utilisée dans le code donne

$$R_\theta(a, b) = (a \cos \theta - b \sin \theta, a \sin \theta + b \cos \theta).$$

Les coordonnées d'indices pairs sont extraites dans **even**, celles d'indices impairs dans **odd**. La fonction calcule les deux nouvelles composantes, les empile puis les aplatit pour reconstituer l'ordre des paires adjacentes. Une tête de dimension quatre contient donc les paires d'indices $(0, 1)$ et $(2, 3)$.

Pourquoi parler de rotation ? Développons la somme des carrés des composantes obtenues. Les termes croisés se compensent et $\cos^2 \theta + \sin^2 \theta = 1$, d'où

$$(a \cos \theta - b \sin \theta)^2 + (a \sin \theta + b \cos \theta)^2 = a^2 + b^2.$$

La longueur de la paire est conservée en arithmétique exacte. RoPE change son orientation, pas sa norme. En calcul flottant, on attend cette propriété à l'arrondi près, pas comme une identité bit à bit pour tous les angles.

Pour $\theta = 0$, le vecteur reste inchangé. Pour $\theta = \pi/2$, $(1, 0)$ devient $(0, 1)$. Ces angles simples aident à comprendre la transformation ; le code utilise cependant des angles définis par les positions entières et les fréquences, pas nécessairement des multiples de $\pi/2$.

Dans le test de rotation connue, une tête de dimension deux reçoit la paire $(1, 0)$ avec un décalage de trois. Sa fréquence vaut un, donc l'angle vaut trois radians. Le résultat attendu est $(\cos 3, \sin 3)$, et non une permutation simple.

Point de vigilance. Les valeurs d'attention ne sont pas tournées. Seules les requêtes et les clés passent dans **_apply_rope**. La dimension d_k doit être paire ; une largeur totale D paire ne suffit pas si son quotient par le nombre de têtes est impair.

Exercice. Une paire vaut $(3, 4)$. Calculez sa rotation d'angle $\pi/2$ et comparez les carrés des normes. La configuration $D = 12$, $H = 4$ convient-elle à RoPE ?

Correction détaillée. La paire devient $(-4, 3)$. Les deux carrés des normes valent $3^2 + 4^2 = 25$. La configuration proposée donne $d_k = 3$, impair : elle est refusée pour RoPE, même si douze est divisible par quatre et conviendrait aux positions apprises.

Fiche 69 — Fréquences, distance relative et décalage

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

MultiHeadSelfAttention._apply_rope

Objectif. Relier les angles RoPE à la position et comprendre ce que signifie leur dépendance relative, sans promettre un contexte illimité.

À comprendre. Toutes les paires d’une tête ne tournent pas à la même vitesse. Le code attribue une fréquence à chaque paire, puis multiplie cette fréquence par la position globale du token.

Si r numérote les paires à partir de zéro, les expressions utilisées sont

$$\omega_r = 10000^{-2r/d_k}, \quad \theta_{m,r} = m\omega_r, \quad 0 \leq r < d_k/2.$$

Pour $d_k = 4$, les fréquences sont un et $1/100$. À la position trois, les angles valent donc trois radians et trois centièmes de radian. Certaines paires changent rapidement d’orientation, d’autres plus lentement. Ces fréquences sont calculées par une formule fixe, sans paramètres appris.

La propriété relative apparaît dans le produit scalaire entre une requête tournée à la position m et une clé tournée à la position n . Pour une paire et une fréquence ω ,

$$\langle R_{m\omega}q, R_{n\omega}k \rangle = \langle q, R_{(n-m)\omega}k \rangle.$$

Tourner les deux vecteurs du même angle ne change pas leur produit scalaire ; il reste leur rotation relative. Cette égalité explique comment le décalage entre positions intervient dans les scores. Elle ne signifie pas que le score dépend uniquement de la distance : les contenus q et k comptent toujours.

Avec un cache de longueur p , la fonction crée les positions de p à $p + T - 1$. Les nouvelles requêtes et clés sont tournées à ces positions. Les clés anciennes stockées dans le cache sont déjà tournées et ne doivent pas subir la transformation une seconde fois.

Le calcul des fréquences et angles utilise des nombres en `float32`, puis sinus et cosinus sont convertis vers le type du tenseur. Cette précision fait partie des détails à considérer lorsque l’on compare des calculs sous précision réduite.

Point de vigilance. L’absence de table positionnelle apprise ne supprime pas la limite de contexte imposée par `MiniGPT.forward`. L’identité relative ne prouve ni une bonne extrapolation à des distances inédites, ni l’absence d’erreurs numériques pour des positions très grandes.

Exercice. Pour $d_k = 4$, un cache contient cinq tokens et l’on ajoute deux tokens. Quels angles la seconde paire utilise-t-elle ? Pourquoi repartir de zéro serait-il incorrect ?

Correction détaillée. Les positions nouvelles sont cinq et six ; avec la fréquence $1/100$, les angles valent $0,05$ et $0,06$ radian. Repartir de zéro donnerait zéro et $0,01$: les nouvelles clés seraient placées dans un repère incompatible avec les anciennes clés déjà tournées. Les distances représentées ne correspondraient plus à la séquence.

Fiche 70 — SDPA : même calcul, contrat de masque différent

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

MultiHeadSelfAttention.forward

Objectif. Identifier les conditions d'utilisation de SDPA, la convention de son masque et le maintien exact de la place du dropout.

À comprendre. SDPA est l'interface PyTorch de calcul d'attention par produit scalaire mis à l'échelle. Son implémentation effective dépend du matériel et des opérations disponibles ; demander SDPA ne prouve pas qu'un noyau accéléré particulier est utilisé.

La branche du dépôt est prise si `attention_backend` vaut `'sdpa'`, si les poids ne sont pas demandés et si la fonction correspondante existe dans `torch.nn.functional`. Sinon, le calcul manuel reste disponible.

Sans passé non vide, le code transmet un masque nul et `is_causal=True`. Avec un cache non vide, il construit un masque rectangulaire tenant compte des positions globales. Attention au sens des booléens : dans l'interface SDPA, vrai signifie *autorisé*, alors que le masque manuel utilise vrai pour *interdit*.

$$M_{\text{manuel}}(i, j) = (j > p + i), \quad M_{\text{SDPA}} = \neg M_{\text{manuel}}.$$

Le code applique donc `~mask`. Avec trois tokens en cache et deux requêtes nouvelles, le masque SDPA contient une première ligne (V, V, V, V, F), puis une ligne entièrement vraie. Il utilise alors `is_causal=False`, puisque la causalité est déjà exprimée explicitement.

Autre détail central : l'appel SDPA reçoit toujours `dropout_p=0.0`, même pendant l'entraînement. Il ne supprime donc aucun poids d'attention. Le dropout configuré dans le module reste appliqué à la sortie fusionnée, avant `out_proj`, comme dans le chemin manuel.

Sur processeur, PyTorch peut choisir son implémentation mathématique quand les noyaux fusionnés ne conviennent pas. Si l'appel lève `RuntimeError` ou `NotImplementedError`, ce code autorise en plus un repli manuel sur CPU. Pour ces exceptions sur un autre type de périphérique, il relance l'erreur plutôt que de la masquer.

Point de vigilance. Demander `return_attention=True` contourne SDPA et calcule explicitement les poids. Des résultats proches entre moteurs n'impliquent ni une égalité bit à bit générale, ni un gain de vitesse mesuré. L'ordre des opérations flottantes peut différer.

Exercice. Une requête globale trois consulte cinq clés, numérotées de zéro à quatre. Écrivez ses deux masques booléens et indiquez le taux de dropout transmis à SDPA si le module est entraîné avec un taux de 0,2.

Correction détaillée. Le masque manuel est (F, F, F, F, V) ; celui de SDPA est son inverse. SDPA reçoit toujours zéro comme taux. Le dropout de 0,2 est maintenu ailleurs, après fusion des têtes et avant projection ; il n'est donc pas désactivé dans l'ensemble du module.

Fiche 71 — Le cache KV : un contrat d'inférence

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

MultiHeadSelfAttention._validate_past_key_value

Objectif. Savoir ce qu'un cache contient, ce que le modèle vérifie et comment fournir uniquement les tokens nouveaux sans changer le calcul causal.

À comprendre. Le cache mémorise les clés et les valeurs déjà calculées dans chaque couche. Les anciennes requêtes ne sont pas nécessaires pour traiter les nouvelles positions : seules les nouvelles requêtes consultent les clés disponibles.

Pour L couches et un passé de longueur p , `past_key_values` contient une paire par couche. Chaque paire possède deux tenseurs :

$$K_{\text{cache}}, V_{\text{cache}} \in \mathbb{R}^{B \times H \times p \times d_k}, \quad p + T \leq C.$$

Le modèle exige une liste ou un tuple contenant exactement autant de paires que de blocs. Chaque paire doit contenir deux tenseurs de rang quatre. Le lot, le nombre de têtes et la dimension de tête doivent correspondre au module. Les clés et valeurs doivent avoir les mêmes formes et le même type numérique.

Les tenseurs doivent être flottants et sur le même périphérique que l'entrée. Une vérification plus bas, après projection, exige aussi le même type que les nouvelles clés et valeurs. Toutes les couches doivent partager la même longueur passée ; un cache partiellement tronqué dans une seule couche est rejeté.

Dans MiniGPT, utiliser ou demander un cache impose le mode `eval()` et l'absence de cibles. L'appelant doit fournir seulement les nouveaux tokens. Le modèle rejette une entrée de longueur nulle et le dépassement de C ; la politique de génération pour gérer une fenêtre pleine est étudiée dans les fiches suivantes du manuel.

Avec `use_cache=True`, le retour du modèle est `(logits, loss, cache)`, où la perte vaut `None` dans cet usage. Fournir un cache sans demander son retour reste permis : on reçoit alors seulement `(logits, loss)`. En mode RoPE, les clés mémorisées sont déjà tournées.

Point de vigilance. `eval()` modifie notamment le dropout, mais ne désactive pas à lui seul le suivi des gradients. L'inférence utilise habituellement aussi `torch.no_grad()`. Les vérifications structurelles ne prouvent pas qu'un cache vient des bons tokens ou des mêmes poids : cette cohérence reste à la charge de l'appelant.

Exercice. Pour $B = 2$, $H = 4$, $d_k = 4$, $L = 2$ et $p = 3$, combien de nombres le cache contient-il ? Quelle forme prend chaque tenseur après deux nouveaux tokens ?

Correction détaillée. Chaque tenseur contient $2 \times 4 \times 3 \times 4 = 96$ nombres. Deux tenseurs par couche et deux couches donnent 384 nombres. Après l'appel, chaque tenseur a la forme $(2, 4, 5, 4)$, et le cache total contient 640 nombres. Ce sont des activations conservées, pas de nouveaux paramètres entraînaibles.

Fiche 72 — Ce que vérifient réellement les tests

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/tests/test_model.py

ModelTests

Objectif. Relier les garanties des tests aux calculs étudiés, en séparant compatibilité numérique, contrats d'interface et qualité du modèle.

À comprendre. Un test compare un comportement observé à une attente précise, sur des configurations déterminées. Il peut détecter une régression ; il ne prouve pas toutes les propriétés imaginables du réseau.

`test_default_state_dict_and_legacy_forward` vérifie les noms des paramètres, charge strictement un clone, puis reconstruit le calcul historique. Il reproduit notamment le dropout avant projection et les deux normalisations post-résiduelles. La comparaison des logits utilise `rtol=0` et `atol=0`, avec la graine réinitialisée pour reproduire les tirages de dropout.

`test_cached_chunks_match_full_sequence` compare un passage complet aux découpages en huit tokens isolés ou en morceaux de longueurs trois, deux et trois. Il couvre les deux encodages et les deux moteurs. Il contrôle aussi les formes des caches. Les logits sont comparés avec une tolérance absolue de 2×10^{-7} et relative de 10^{-5} .

Une comparaison approchée combine généralement les deux échelles :

$$|a - b| \leq \text{atol} + \text{rtol} |b|.$$

La partie absolue compte près de zéro ; la partie relative accompagne la taille de la valeur de référence. L'égalité attendue est donc plus précise que simplement « les nombres ont l'air proches ».

`test_sdpa_masks_and_attention_weights` vérifie le masque rectangulaire autorisé, l'absence d'appel SDPA quand les poids sont demandés, le poids futur nul et les sommes de lignes égales à un. Ce contrôle teste une manifestation concrète de causalité, sans constituer une preuve générale sur toutes les entrées.

`test_sdpa_training_dropout_and_gradients_match_manual` compare aussi les gradients, avec tolérance absolue 2×10^{-7} et relative 10^{-4} . Les poids sont partagés par chargement et les graines synchronisées. `test_autocast_cache_dtype` utilise des tolérances plus larges, 0,002 et 0,02, adaptées à son calcul en précision réduite.

Point de vigilance. Les tests de rotation, de repli CPU, de cache invalide, d'inférence seule et de configuration invalide complètent ces contrôles. Ils ne mesurent ni vitesse, ni perplexité après entraînement, ni qualité des textes. Aucun succès d'exécution n'est présumé ici.

Exercice. Deux implémentations donnent des sorties proches, mais l'une place son dropout après la projection. Pourquoi une comparaison en évaluation ne suffit-elle pas à valider cette modification ?

Correction détaillée. En évaluation, les deux dropouts sont inactifs : leur emplacement peut devenir invisible. Il faut comparer le comportement en entraînement avec mêmes paramètres et tirages contrôlés, puis les gradients si l'apprentissage est concerné. Une bonne vérification cible le mécanisme susceptible de différer, pas seulement la forme finale.

Fiche 73 — Choisir une configuration sans deviner les défauts

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

MODEL_PRESETS, train, main

Objectif. Construire une configuration effective en distinguant le préréglage, les options explicites et les valeurs du constructeur. Cette lecture évite de comparer deux expériences qui ne diffèrent pas seulement par le paramètre annoncé.

Le script propose trois préréglages. Dans l'ordre contexte, dimension, têtes, couches et largeur du réseau intermédiaire, leurs valeurs sont :

$$(C, D, H, L, F) = \begin{cases} (32, 64, 4, 2, 256) & \text{tiny,} \\ (128, 128, 4, 4, 512) & \text{small,} \\ (256, 256, 8, 6, 1024) & \text{medium.} \end{cases}$$

Le vocabulaire V provient du tokenizer effectivement appris, et non de cette table. En particulier, demander une taille de vocabulaire ne transforme pas le tokenizer caractère en tokenizer BPE.

À comprendre. La fonction `train` copie le préréglage, ajoute ses défauts généraux, puis remplace uniquement les options dont la valeur n'est pas `None`. Une option omise en ligne de commande ne doit donc pas effacer la valeur du préréglage. Les noms inconnus transmis directement à la fonction sont refusés.

Sans argument, l'entraînement utilise `tiny`, cinq époques, $B = 8$, un taux d'apprentissage de 0,0003, un dropout de 0,1, un pas de fenêtres de un et la graine 42. Les positions sont apprises, l'attention est manuelle, la précision est `float32`. Le périphérique est CUDA si disponible, sinon CPU : préciser CPU rend le choix explicite.

Le constructeur `MiniGPT`, dans le script du modèle, possède d'autres défauts : $D = 128$ et $F = 512$, tandis que C est obligatoire. L'entraînement lui transmet sa configuration ; il n'hérite donc pas accidentellement de ces deux dimensions. De même, la CLI de génération demande vingt nouveaux tokens par défaut, contre quarante pour l'appel direct à `generate`. La taille du batch n'appartient pas au constructeur du réseau : elle règle le regroupement des fenêtres par le chargeur de données.

Exercice. On choisit `small`, puis on impose $D = 96$, $H = 6$ et RoPE. Déterminer les dimensions conservées, la dimension d'une tête et la validité de ce choix. Expliquer pourquoi remplacer ensuite H par cinq échoue.

Correction détaillée. Les options explicites donnent $(C, D, H, L, F) = (128, 96, 6, 4, 512)$. La dimension d'une tête vaut $d_k = 96/6 = 16$: elle est entière et paire, donc compatible avec RoPE. Avec cinq têtes, $96/5$ n'est pas entier ; le constructeur refuse cette répartition avant tout apprentissage. Le préréglage ne réajuste automatiquement ni D ni F .

Point de vigilance. Noter la configuration finale, pas seulement le nom du préréglage. Une grande largeur augmente les besoins matériels sans garantir une meilleure validation. Un contexte agrandi exige aussi des portions de corpus assez longues ; changer uniquement le modèle peut donc rendre les données inutilisables.

Fiche 74 — Passer des logits à la perte du lot

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

train : CrossEntropyLoss et reshape

Objectif. Suivre exactement les prédictions et les étiquettes jusqu'au scalaire utilisé pour apprendre. Le passage des tenseurs à une moyenne ne doit perdre ni l'ordre des positions ni la correspondance entre chaque position et sa cible.

Le dataset fournit deux tenseurs d'entiers, chacun de forme $[B, T]$. L'entrée contient les tokens observés ; la cible contient les tokens suivants. Le décalage a déjà été construit dans les données. Le modèle produit ensuite des logits de forme $[B, T, V]$: pour chaque fenêtre et chaque position, une ligne de V scores propose le token suivant.

À comprendre. `CrossEntropyLoss` reçoit des scores bruts et des indices de classes. Le script transforme donc les logits en $[BT, V]$ avec `reshape(-1, logits.size(-1))`, et les cibles en $[BT]$ avec `reshape(-1)`. Le symbole Python `-1` demande de déduire la dimension manquante, sans modifier le nombre d'éléments.

En numérotant les $N = BT$ positions par i , la perte est

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \left(\log \sum_{j=0}^{V-1} e^{z_{ij}} - z_{i, y_i} \right).$$

Cette formule combine normalisation et logarithme. Appliquer auparavant un softmax ne serait pas une simple présentation différente : ses probabilités seraient prises pour de nouveaux logits. On optimiserait alors une autre expression, avec des scores artificiellement comprimés.

L'aplatissement rapproche les dimensions batch et temps, pas temps et vocabulaire. Pour $B = 2$, $T = 3$, $V = 5$, trente scores deviennent six lignes de cinq scores, accompagnées de six étiquettes. Il ne faut ni refaire le décalage des cibles ni les remplacer par les prédictions du réseau. Chaque étiquette reste un indice entier du vocabulaire, et non un vecteur de probabilités construit à la main.

Exercice. Deux positions ont respectivement les logits $(\log 3, 0)$ et $(0, \log 3)$; les deux cibles valent zéro. Calculer les probabilités utiles, les pertes individuelles et leur moyenne. Que montre cet exemple sur une prédiction très assurée mais incorrecte ?

Correction détaillée. Les probabilités de la cible sont $3/4$ puis $1/4$. Les pertes valent $-\log(3/4) \simeq 0,288$ et $-\log(1/4) \simeq 1,386$. La moyenne est $\log(4/\sqrt{3}) \simeq 0,837$ nat par cible. La seconde position pénalise davantage le modèle, même si ses scores sont aussi contrastés que ceux de la première. Ces nombres sont un calcul didactique, pas une mesure issue du corpus.

Point de vigilance. Le second objet renvoyé par `model(x)` est ici inutilisé : sans cibles transmises au modèle, sa perte interne vaut `None`. L'entraînement calcule explicitement sa propre perte, selon la même convention. Les tokens inconnus restent des cibles ordinaires ; aucun masque de padding supplémentaire n'est fourni dans cette boucle.

Fiche 75 — Respecter l'ordre réel de la rétropropagation

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

train : zero_grad, backward, step

Objectif. Relier le gradient étudié en mathématiques aux trois opérations qui changent effectivement les poids. Une itération correcte exige de distinguer le calcul de la perte, l'accumulation des gradients et la modification des paramètres.

PyTorch construit un graphe de calcul pendant le passage avant. Les tenseurs intermédiaires permettent de retrouver comment chaque paramètre a contribué à la perte. Appeler `loss.backward()` parcourt les dépendances en sens inverse et inscrit les dérivées dans les attributs `grad` des paramètres. Cette opération ne constitue pas encore une mise à jour des poids.

À comprendre. Dans la boucle réelle, l'ordre est : passage avant, calcul de la perte, `optimizer.zero_grad(set_to_none=True)`, `loss.backward()`, puis `optimizer.step()`. Le petit schéma introductif du fichier ne doit pas remplacer la lecture des instructions exécutées.

Effacer les anciens gradients après le passage avant est correct ici : ce passage n'a pas besoin des attributs `grad`. En revanche, les effacer entre `backward` et `step` priverait l'optimiseur des dérivées fraîchement calculées. L'option `set_to_none=True` retire les gradients stockés au lieu de remplir systématiquement leurs tenseurs de zéros.

Pour comprendre le risque d'accumulation, considérons deux pertes successives :

$$g^{(1)} = \nabla_{\theta} \mathcal{L}_1, \quad \text{sans remise à zéro : } \theta.\text{grad} = g^{(1)} + g^{(2)}.$$

L'accumulation peut être intentionnelle dans d'autres programmes. Ce script, lui, veut une mise à jour par lot et remet donc les gradients à zéro à chaque itération. Il ne réalise pas une accumulation contrôlée sur plusieurs micro-lots.

La valeur affichée par `loss.item()` correspond au calcul effectué avant la mise à jour. Elle devient un nombre Python pour les statistiques ; ce nombre ne conserve pas le graphe nécessaire à la rétropropagation. On ne remplace donc jamais `loss.backward()` par un appel sur cette valeur détachée.

Exercice. Pour une illustration en descente simple, prendre $\mathcal{L}(\theta) = (\theta - 3)^2$, $\theta = 1$ et $\eta = 0,1$. Calculer une mise à jour, puis expliquer les effets d'une remise à zéro effectuée au mauvais endroit.

Correction détaillée. La dérivée vaut $2(\theta - 3) = -4$. Une descente simple donnerait $\theta' = 1 - 0,1(-4) = 1,4$, donc un déplacement vers trois. Effacer le gradient juste avant la mise à jour supprime cette information. Ne jamais l'effacer additionne au contraire des dérivées calculées pour différents lots et différents poids. Le calcul numérique illustre le principe ; l'optimiseur réellement employé est AdamW, dont le déplacement diffère.

Point de vigilance. La construction du graphe dépend de l'enregistrement des gradients, pas uniquement du mode entraînement. Une perte obtenue entièrement sous `no_grad` ne peut pas entraîner normalement le modèle par rétropropagation.

Fiche 76 — Comprendre les deux mémoires d'Adam

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

train : construction de AdamW

Objectif. Expliquer pourquoi Adam adapte les déplacements sans prétendre que le script utilise une simple descente de gradient. Les deux moments se comprennent comme des moyennes pondérées des dérivées et de leurs carrés.

Dans une descente stochastique élémentaire, un paramètre suit $\theta_{t+1} = \theta_t - \eta g_t$. Deux coordonnées ayant des gradients de tailles très différentes reçoivent alors des déplacements très différents. Adam garde un historique pour lisser les changements et ajuster l'échelle de chaque coordonnée. Les opérations suivantes se lisent séparément pour chaque nombre du modèle, même si PyTorch les applique à des tenseurs entiers.

À comprendre. Le premier moment mémorise une direction moyenne. Le second mémorise une moyenne de carrés, donc une échelle toujours positive. Ce second moment n'est pas directement la variance : on n'y soustrait pas le carré de la moyenne.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

Les coefficients compris entre zéro et un donnent davantage de poids au passé lorsqu'ils sont proches de un. Comme les mémoires commencent à zéro, Adam corrige leur biais initial :

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad \Delta\theta_t = -\eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

Le petit ε évite une division par zéro et influence les très petites échelles. Le quotient ne signifie pas que toutes les coordonnées bougent toujours autant ; il dépend de leurs historiques respectifs.

Le dépôt appelle AdamW, qui reprend ces moments et ajoute une décroissance découplée des poids, étudiée ensuite. Les valeurs $\beta_1 = 0,9$ et $\beta_2 = 0,999$ sont des défauts usuels de PyTorch, non des arguments écrits dans cet appel. Pour une expérience strictement documentée, consulter aussi la version installée de la bibliothèque. Les carrés empêchent notamment des gradients positifs et négatifs de s'annuler dans la mémoire d'échelle, même lorsqu'ils s'annulent partiellement dans la mémoire de direction.

Exercice. Avec ces coefficients, des mémoires initiales nulles et un premier gradient $g_1 = 2$, calculer m_1 , v_1 , puis les moments corrigés. Comparer le déplacement Adam, en négligeant ε , à celui d'une descente simple lorsque $\eta = 0,01$.

Correction détaillée. On obtient $m_1 = 0,2$ et $v_1 = 0,004$. Les divisions par 0,1 et 0,001 donnent $\hat{m}_1 = 2$ et $\hat{v}_1 = 4$. Le quotient vaut un : Adam déplace donc le paramètre de $-0,01$, contre $-0,02$ pour la descente simple. La correction du biais est essentielle à cette comparaison.

Point de vigilance. Cet exemple isole les moments : il omet volontairement la décroissance AdamW. Adapter automatiquement une échelle ne garantit ni convergence globale ni absence de divergence ; le taux d'apprentissage reste un choix expérimental important et doit être jugé avec la validation.

Fiche 77 — Lire AdamW sans lui ajouter des mécanismes absents

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

train : optimizer et learning_rate

Objectif. Identifier ce que l’optimiseur fait réellement et ce que la boucle ne fait pas. La décroissance des poids constitue un choix distinct du taux d’apprentissage et de l’adaptation par les moments.

L’appel réel transmet tous les paramètres du modèle à `torch.optim.AdamW`, avec seulement `lr=config['learning_rate']` comme réglage explicite. Aucun groupe particulier n’exclut les biais ou les paramètres de normalisation. Les embeddings de tokens et la tête de sortie sont également deux ensembles de poids distincts : le modèle ne les lie pas.

À comprendre. AdamW réduit directement les poids, indépendamment du gradient utilisé pour alimenter les moments. Avec une décroissance λ , la représentation usuelle de la mise à jour d’une coordonnée est

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

Le premier terme rapproche les poids de zéro ; le second utilise l’information de la perte. Ajouter simplement $\lambda\theta$ au gradient avant Adam modifierait aussi ses mémoires et ne décrirait pas cette décroissance découplée. La distinction importe précisément parce que l’échelle d’Adam dépend des gradients antérieurs.

Dans les versions usuelles de PyTorch, AdamW prend par défaut une décroissance de 0,01, des coefficients (0,9, 0,999) et $\varepsilon = 10^{-8}$. Ces valeurs appartiennent à la bibliothèque, pas à une configuration explicitement figée par le dépôt. Une archive d’expérience sérieuse doit donc noter la version de PyTorch et vérifier ses défauts.

Le taux d’apprentissage est constant dans cette boucle : aucun ordonnanceur, échauffement progressif ou décroissance cosinusoidale n’est instancié. Le code ne limite pas non plus la norme des gradients. Une ligne affichant toujours le même taux n’est donc pas une panne d’un ordonnanceur caché ; elle correspond au fonctionnement prévu.

Exercice. Prendre, uniquement pour le calcul, $\theta = 2$, $\eta = 0,001$, $\lambda = 0,01$ et un quotient adaptatif égal à 0,5. Séparer la contribution de la décroissance et celle du gradient. Que reste-t-il si ce quotient est nul mais qu’un gradient nul existe ?

Correction détaillée. La décroissance retire $0,001 \times 0,01 \times 2 = 0,00002$. Le déplacement adaptatif retire $0,001 \times 0,5 = 0,0005$. Le nouveau poids vaut donc 1,99948. Si le quotient est nul, le poids devient 1,99998 par la seule décroissance. Un gradient absent, égal à `None`, doit être distingué d’un gradient numériquement nul : AdamW peut ignorer entièrement les paramètres sans gradient.

Point de vigilance. Ne pas annoncer de clipping, de scheduler ou de règles spéciales pour les normalisations dans un compte rendu de ce dépôt. Ces mécanismes peuvent être utiles ailleurs, mais leur présence doit être démontrée par un appel effectif, pas supposée à partir du nom d’un optimiseur moderne.

Fiche 78 — Séparer dropout, mode et enregistrement des gradients

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

train, evaluate ; 07_generate.py : generate

Objectif. Distinguer trois décisions souvent confondues : activer le comportement d'entraînement, tirer des masques de dropout et enregistrer les opérations pour la rétropropagation. Chaque décision possède une commande différente.

Le dropout désactive aléatoirement certaines composantes pendant l'entraînement. Pour une composante x et une probabilité de suppression p , PyTorch utilise le principe du dropout inversé :

$$\tilde{x} = \frac{M}{1-p}x, \quad \mathbb{P}(M=1) = 1-p, \quad \mathbb{E}[\tilde{x}] = x.$$

Le facteur compensateur préserve l'espérance, pas chaque réalisation. Avec $p=0,1$, une composante conservée est multipliée par $1/0,9$; elle peut aussi devenir nulle. Deux passages avant successifs peuvent donc différer alors que les poids n'ont pas changé.

À comprendre. `model.train()` active récursivement le mode entraînement. `model.eval()` désactive notamment le dropout, sans bloquer les gradients. `torch.no_grad()` empêche l'enregistrement du graphe, mais ne change pas à lui seul le comportement des couches de dropout.

Un modèle neuf est en mode entraînement. La boucle conserve ce mode, et `evaluate` le restaure après sa mesure. La fonction `generate`, elle, impose le mode évaluation et porte le décorateur `no_grad`. Ces deux actions complémentaires rendent la génération moins coûteuse en mémoire et désactivent les perturbations destinées à l'apprentissage.

Un détail révélateur apparaît avant les époques : le script affiche les formes avec un passage placé sous `no_grad`, mais sans appeler `eval`. Ce passage ne construit pas de graphe ; son dropout peut pourtant rester actif et consommer des tirages aléatoires. Dédurre le mode à partir de la seule présence de ce contexte serait donc incorrect.

Exercice. Une activation vaut six et $p=0,25$. Donner ses deux valeurs possibles en entraînement, leurs probabilités et son espérance. Indiquer ensuite le résultat en évaluation et expliquer si une dérivée reste calculable dans ce dernier mode.

Correction détaillée. L'activation vaut zéro avec probabilité $0,25$, ou $6/0,75 = 8$ avec probabilité $0,75$. Son espérance est $0,25 \times 0 + 0,75 \times 8 = 6$. En évaluation, le dropout la laisse égale à six. Une dérivée reste calculable si les tenseurs concernés demandent des gradients et si leur enregistrement n'a pas été désactivé ; `eval` ne remplace pas `no_grad`.

Point de vigilance. La génération gloutonne supprime le tirage du prochain token, mais `eval` seul ne rend pas un échantillonnage multinomial déterministe. Inversement, `no_grad` seul ne suffit pas pour le cache KV : le modèle exige explicitement le mode évaluation et l'absence de cibles.

Fiche 79 — Évaluer une moyenne réellement pondérée par token

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

evaluate

Objectif. Reconstituer la valeur de validation à partir des lots sans favoriser le dernier lot, souvent plus petit. Comprendre aussi pourquoi une fonction d'évaluation doit respecter le mode dans lequel son appelant utilisait le modèle.

`evaluate` repère d'abord le périphérique des paramètres, mémorise `model.training` dans `was_training`, puis appelle `model.eval()`. Son décorateur `no_grad` désactive l'enregistrement des gradients. Chaque lot est envoyé vers le même périphérique que le modèle, puis passe dans le même calcul de perte que pendant l'entraînement.

À comprendre. La perte renvoyée par le critère est déjà une moyenne sur les positions du lot. Pour réunir plusieurs lots, le code multiplie cette moyenne par `y.numel()`, additionne les contributions, puis divise par le nombre total de cibles.

$$\mathcal{L}_{\text{val}} = \frac{\sum_{r=1}^R n_r \ell_r}{\sum_{r=1}^R n_r}, \quad n_r = B_r T.$$

Une moyenne non pondérée des ℓ_r n'est correcte que lorsque tous les lots contiennent le même nombre de cibles. Or le chargeur n'écarte pas systématiquement le dernier lot incomplet. L'unité pertinente est donc la position prédite, pas le numéro du lot.

À la fin du parcours normal, `model.train(was_training)` rétablit le mode initial. Si le modèle était déjà en évaluation, il y reste ; s'il était en entraînement, le dropout redevient actif pour le prochain lot. Cette restauration n'est pas protégée par un bloc `finally` : une exception pendant le parcours interrompt ce scénario normal.

La division utilise `max(total_tokens, 1)` pour éviter un dénominateur nul. Un chargeur vide conduirait ainsi à zéro, valeur technique qui ne prouverait aucune qualité prédictive. Le pipeline normal construit des datasets non vides et refuse les portions trop courtes.

Exercice. Un lot de 32 cibles présente une perte moyenne de deux ; un second lot de huit cibles présente une perte de quatre. Calculer la validation correcte, la moyenne naïve et leur différence. Que compterait-on si deux fenêtres contenaient un même token du corpus ?

Correction détaillée. La somme pondérée vaut $32 \times 2 + 8 \times 4 = 96$. La validation correcte vaut $96/40 = 2,4$, contre trois pour la moyenne naïve, soit une surestimation de 0,6. Avec des fenêtres chevauchantes, chaque apparition comme cible compte séparément. La métrique porte sur les positions du dataset parcouru, et non sur les caractères distincts du fichier source.

Point de vigilance. Cette pondération suppose le critère moyen utilisé ici. Remplacer le critère par une somme sans adapter l'agrégation compterait deux fois les tailles de lots. Conserver le même contexte et le même pas de fenêtres reste nécessaire pour comparer des validations avec précision.

Fiche 80 — Interpréter les courbes sans inventer un test indépendant

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

train : train_losses, val_losses, loss_curve.png

Objectif. Utiliser les courbes comme des indices sur l'apprentissage, et non comme une certification de qualité. Une validation utile demande de connaître son origine, son mode de calcul et les décisions prises après l'avoir observée.

Le script trace les pertes train et validation par époque parcourue. Le texte est partagé avant apprentissage du tokenizer et création des fenêtres : la dernière fraction sert à la validation. Aucune fenêtre ne traverse cette frontière, mais aucun troisième ensemble de test n'est constitué.

À comprendre. La perte d'entraînement agrège des lots vus avec dropout et avec des poids qui changent au fil de l'époque. La validation est calculée ensuite, sans dropout, avec les poids de fin d'époque. Les deux courbes n'évaluent donc pas exactement le même objet au même instant.

Un écart peut être résumé par

$$G_e = \mathcal{L}_{\text{val},e} - \mathcal{L}_{\text{train},e}.$$

Un écart croissant, avec baisse du train et hausse durable de la validation, suggère un surapprentissage. Une oscillation isolée ne suffit pas : l'échantillonnage avec remise et des distributions différentes entre début et fin du texte peuvent aussi influencer les courbes.

Le script ne sélectionne pas automatiquement la meilleure époque, n'arrête pas l'apprentissage selon la validation et ne conserve pas un checkpoint distinct pour chaque point. La sauvegarde finale contient les derniers poids, même si une époque précédente avait une validation plus faible. Un arrêt par `max_steps` peut d'ailleurs terminer une époque partielle.

Une perte de validation inférieure au train n'est pas impossible : l'absence de dropout et les poids plus récents peuvent l'expliquer. À l'inverse, des phrases plaisantes générées après coup ne remplacent pas une mesure systématique sur des données dont l'utilisation est clairement définie.

Exercice. Considérer ces valeurs fictives : train (2,0; 1,4; 1,0), validation (2,1; 1,7; 1,9). Repérer la meilleure époque selon la validation, calculer les écarts et dire quel modèle la boucle sauvegarde si elle parcourt exactement ces trois époques.

Correction détaillée. La meilleure validation est 1,7, à la deuxième époque. Les écarts valent 0,1, 0,3, puis 0,9. Le dernier mouvement suggère un surapprentissage, sans suffire à établir une loi générale. La boucle sauvegarde pourtant les poids de la troisième époque : aucune comparaison des minima ne pilote sa sauvegarde. Ces valeurs illustrent une lecture de courbe, pas un résultat mesuré sur le dépôt.

Point de vigilance. Choisir plusieurs hyperparamètres en consultant toujours la même validation finit par adapter les décisions à cette portion. Pour annoncer une généralisation indépendante, prévoir un protocole de test supplémentaire ; il n'est pas implémenté ici et ne doit pas être sous-entendu.

Fiche 81 — Atelier CPU : vérifier tout le parcours en quelques pas

```
/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py
```

```
main, train
```

Objectif. Préparer une première expérience courte qui entraîne, évalue et sauvegarde sans écraser le checkpoint habituel. Le succès recherché est technique : constater que les étapes se raccordent, pas obtenir un texte de qualité après quelques mises à jour.

Les commandes suivantes sont à exécuter par le lecteur, depuis un environnement où les dépendances du projet sont déjà installées. Le chemin du script et celui du répertoire de sortie sont absolus. Le corpus fourni reste sélectionné par défaut ; il convient au pré réglage minuscule.

```
python /home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py \  
  --device cpu --precision float32 --preset tiny \  
  --epochs 5 --max-steps 2 \  
  --checkpoint-dir /tmp/mini-gpt-atelier-cpu
```

À comprendre. `max_steps` limite le compteur global des mises à jour, pas le nombre de pas de chaque époque. Dès que ce compteur atteint deux, la boucle termine le parcours d'entraînement en cours, effectue sa validation, puis sort également de la boucle des époques.

Le nombre de pas réalisé est le minimum entre cette limite et les pas disponibles sur les époques demandées. Même si cinq époques sont configurées, la courbe peut donc n'avoir qu'un point. La validation parcourt son chargeur complet : une limite courte sur l'entraînement ne borne pas directement son temps d'exécution.

Observer les formes annoncées : pour un lot plein, $[B, T] = [8, 32]$ et les logits ont la forme $[8, 32, V]$. Vérifier ensuite la présence de `mini_gpt.pt`, de `loss_curve.png` et du dossier `tokens` dans le répertoire choisi. Ces sorties documentent respectivement les poids rechargeables, l'historique graphique et les données prétraitées.

L'étape suivante utilise explicitement la sauvegarde de cet atelier :

```
python /home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/07_generate.py \  
  --checkpoint /tmp/mini-gpt-atelier-cpu/mini_gpt.pt \  
  --device cpu --prompt "bonjour " --max-new-tokens 8 --greedy
```

Exercice. Supposer que chacun des deux lots d'entraînement est plein. Combien de positions cibles ont contribué aux mises à jour ? Pourquoi ce nombre ne correspond-il pas nécessairement à autant de tokens distincts dans le corpus ?

Correction détaillée. Chaque lot apporte $8 \times 32 = 256$ cibles ; deux lots en apportent 512. Des fenêtres peuvent se chevaucher et l'échantillonnage s'effectue avec remise : plusieurs cibles peuvent donc provenir des mêmes positions du texte. Les cibles de validation ne sont pas ajoutées à ce total d'entraînement, même si leur traitement consomme du temps.

Point de vigilance. Aucun résultat n'est promis par ces commandes. Une sortie peu cohérente après deux pas est attendue comme possibilité, non comme anomalie. Employer un dossier neuf pour chaque expérience ; deux entraînements simultanés ne doivent pas partager leurs fichiers de tokens ni leur checkpoint.

Fiche 82 — Sauvegarder un modèle n’est pas reprendre un entraînement

```
/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py
```

```
train : checkpoint, torch.save
```

Objectif. Inventorier exactement les informations persistées et comprendre pourquoi un checkpoint suffisant pour générer ne permet pas nécessairement de reprendre la même trajectoire d’optimisation.

Le dictionnaire sauvegardé contient `model_state_dict`, `config`, `tokenizer`, `tokenizer_kind` et `metrics`. Pour le tokenizer caractère, il ajoute aussi `vocab`, afin de conserver une représentation compatible avec l’ancien format. La sauvegarde est écrite après la dernière époque parcourue et après la création de la courbe.

À comprendre. `model_state_dict` conserve les tenseurs enregistrés du modèle, notamment les poids appris. `config` fournit les dimensions et les options nécessaires pour reconstruire l’architecture. `tokenizer` conserve la correspondance entre texte et identifiants, indispensable pour donner le même sens aux lignes des embeddings et aux colonnes des logits.

Les métriques comprennent les courbes, le temps, la mémoire et le débit, mais pas le protocole complet. Le chemin du corpus, sa copie, son empreinte et la fraction de validation ne sont pas archivés dans la configuration. Les conserver séparément permet de reconstituer les données.

AdamW possède des mémoires propres, distinctes des poids du réseau. Schématiquement, l’état dynamique pertinent est

$$S_t = (\theta_t, m_t, v_t, t, \text{états aléatoires, position des données}).$$

Le fichier permet de retrouver θ_t , mais ne sauvegarde pas l’état de l’optimiseur. Il ne stocke pas non plus les états complets des générateurs aléatoires ni la position du sampler. La graine initiale inscrite dans la configuration ne remplace pas ces états après plusieurs tirages.

Le script d’entraînement ne propose d’ailleurs aucune option de reprise. Lancer à nouveau la commande construit un nouveau modèle et un nouvel optimiseur ; cela n’ouvre pas automatiquement le checkpoint présent dans le dossier de sortie.

Exercice. Deux modèles possèdent exactement les mêmes poids. Le premier optimiseur a $\hat{m} = 1$ et $\hat{v} = 4$ pour une coordonnée ; le second vient d’être créé. Peut-on conclure que leurs prochaines mises à jour seront identiques si le prochain lot est identique ?

Correction détaillée. Non. Le premier optimiseur transporte une direction et une échelle issues du passé, alors que le second calcule ses moments depuis zéro et utilise un autre compteur de correction du biais. Un même prochain gradient ne suffit donc pas à déterminer le même déplacement. Les poids identiques assurent plutôt des prédictions identiques dans des conditions d’évaluation identiques, sous réserve des arrondis numériques.

Point de vigilance. La sauvegarde convient à l’inférence et à l’étude des derniers poids. La présenter comme une reprise exacte serait trompeur. Une vraie reprise nécessiterait des champs et une logique supplémentaires, absents de cette implémentation pédagogique.

Fiche 83 — Recharger les formats récents, anciens et le cas absent

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/07_generate.py

load_model_and_tokenizer

Objectif. Suivre les branches du chargement sans confondre compatibilité, sécurité et génération aléatoire. Le chemin du checkpoint détermine ici un comportement important qu'un message de console permet ensuite de reconnaître.

Lorsqu'un fichier existe, le chargement appelle `torch.load` avec `map_location='cpu'` et `weights_only=True`. Les tenseurs sont d'abord chargés sur CPU; après reconstruction et insertion des poids, le modèle est déplacé vers le périphérique et le type numérique demandés. Le mode évaluation est ensuite activé.

À comprendre. Un checkpoint récent possède un champ `tokenizer`. La fonction `tokenizer_from_state` le reconstruit. Sinon, le chargeur suit la branche historique : il repart de `vocab`, restaure l'ordre `itos`, reconstruit `stoi` et retrouve l'identifiant de `<unk>`.

La configuration fournit C, D, H, L, F et le dropout. Si les champs correspondants manquent dans un ancien checkpoint, les positions deviennent `learned` et l'attention `manual`. Une option explicite peut remplacer le backend d'attention au chargement, mais elle ne transforme pas des positions apprises en RoPE : cette architecture doit rester cohérente avec les poids enregistrés.

Un fichier existant mais invalide provoque une erreur ; le programme ne masque pas cette erreur par un modèle aléatoire. Un chemin absent différent de la constante par défaut provoque également `FileNotFoundError`. En revanche, lorsque le chemin absent est exactement le chemin par défaut, le chargeur crée un tokenizer caractère depuis tout le petit corpus et un modèle neuf :

$$(C, D, H, L, F) = (32, 64, 4, 2, 256), \quad p_{\text{dropout}} = 0,1.$$

Les positions sont alors apprises et le backend vaut l'option demandée ou `manual`. La précision demandée s'applique aussi à ce modèle aléatoire. Le chargeur ne fixe pas lui-même une graine pour cette initialisation.

Exercice. Comparer trois situations : le checkpoint par défaut est absent ; un autre chemin explicite est absent ; un ancien fichier valide possède seulement vocabulaire, configuration historique et poids. Que fait chaque branche ?

Correction détaillée. La première construit des poids aléatoires et la CLI l'annonce. La deuxième lève une erreur de fichier absent. La troisième restaure le tokenizer caractère et les poids, en complétant les choix historiques par positions apprises et attention manuelle. Passer explicitement le chemin par défaut absent ne change pas la première branche : la comparaison porte sur le chemin, non sur la présence de l'option dans la commande.

Point de vigilance. `weights_only=True` restreint le chargement ; ce n'est pas une garantie absolue contre tout fichier malveillant ou corrompu. Utiliser des checkpoints de confiance. Le rechargement BPE requiert aussi la dépendance facultative correspondante ; son absence ne justifie pas de remplacer silencieusement le tokenizer.

Fiche 84 — Générer à partir du dernier logit seulement

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/07_generate.py

generate

Objectif. Dérouler une génération autoregressive et comprendre pourquoi le modèle calcule plusieurs positions alors qu'une seule sert à choisir le prochain token. La génération prolonge une suite ; elle ne remplit pas simultanément tout un paragraphe futur.

La fonction vérifie que le nombre de nouveaux tokens est non négatif et que la température est finie. Elle active ensuite l'évaluation, encode le prompt et refuse une suite d'identifiants vide. Le tenseur initial `idx` possède une seule ligne : $B = 1$. Son périphérique est celui des paramètres du modèle, et ses valeurs restent des entiers, même si les poids sont en précision réduite.

À comprendre. À chaque tour, `idx[:, -model.context_length:]` sélectionne les C derniers identifiants au maximum. Sans cache, le modèle traite toute cette fenêtre. Dans les logits renvoyés, `logits[:, -1, :]` extrait les V scores de la dernière position, celle qui prédit immédiatement après le texte actuellement disponible.

Les autres positions prédisent des tokens déjà présents dans la fenêtre : elles étaient utiles pendant l'entraînement, où toutes les cibles étaient connues, mais ne doivent pas produire des nouveaux tokens indépendamment. La prolongation respecte la factorisation

$$P(u_1, \dots, u_N | p) = \prod_{s=1}^N P(u_s | p, u_1, \dots, u_{s-1}),$$

avec, dans ce modèle, une condition effectivement tronquée au contexte accessible. Après sélection, un unique identifiant est concaténé à `idx`.

Le tableau complet des identifiants est conservé pour le décodage final, même lorsque le début du prompt n'est plus envoyé au modèle. Avec `verbose=True`, chaque tour affiche l'identifiant choisi et le texte partiel. Ces impressions sont pédagogiques ; elles peuvent ralentir une mesure si on les laisse actives.

Exercice. Un prompt donne cinq tokens, le contexte vaut quatre et on demande trois nouveaux tokens sans cache. Quelles longueurs d'entrée le modèle voit-il aux trois appels ? Quelle longueur possède la suite finale ? Que se passe-t-il si l'on demande zéro token ?

Correction détaillée. Les longueurs d'entrée sont quatre, quatre, puis quatre. La suite complète contient finalement huit identifiants : cinq initiaux et trois ajoutés. Avec zéro token, aucun passage du modèle n'est effectué dans la boucle ; la fonction renvoie le décodage du prompt encodé. Les validations initiales restent actives, donc un prompt vide est encore refusé.

Point de vigilance. Le texte renvoyé contient le prompt décodé et sa continuation. Un caractère inconnu peut devenir un point d'interrogation au décodage : zéro nouveau token ne garantit donc pas une identité textuelle pour tout prompt. Aucun token de fin ne stoppe la boucle ; seul le nombre demandé fixe sa longueur.

Fiche 85 — Calculer l'effet de la température

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/07_generate.py

generate : temperature

Objectif. Transformer l'idée intuitive de diversité en un calcul de probabilités. La température agit sur les écarts entre scores, sans réentraîner le modèle ni ajouter des connaissances à ses paramètres.

Pour une température strictement positive τ , la fonction divise les logits par τ , applique éventuellement le filtre top-k, puis calcule un softmax. Sans filtrage, la probabilité d'un token i est

$$p_i(\tau) = \frac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}}, \quad \frac{p_i(\tau)}{p_j(\tau)} = e^{(z_i - z_j)/\tau}.$$

La seconde égalité est particulièrement instructive : diminuer τ amplifie les rapports de probabilités entre scores différents. Augmenter τ les rapproche de un. L'ordre des scores reste inchangé, mais leur contraste probabiliste varie.

À comprendre. La valeur zéro ne peut pas être utilisée dans cette division. Le code définit donc une autre branche pour toute température inférieure ou égale à zéro : il choisit directement l'argmax des logits bruts. Cette branche ignore entièrement le top-k et le choix d'échantillonnage.

Une température négative n'inverse ainsi pas les préférences dans ce programme. Elle sert au même choix glouton que zéro. Une valeur non finie, comme l'infini ou `NaN`, est refusée avant la boucle ; elle ne sert pas de raccourci vers une distribution uniforme.

Pour une température positive, `sample=False` choisit l'argmax des probabilités au lieu d'appeler le tirage multinomial. Dans l'arithmétique idéale, changer une température positive ne change alors pas le gagnant : la division et le softmax préservent l'ordre, et le top-k conserve un maximum. Les égalités et les arrondis méritent néanmoins une attention particulière. Pour observer l'effet probabiliste de la température, il faut donc distinguer le calcul des probabilités de la règle finale de sélection : une distribution modifiée peut garder exactement le même maximum.

Exercice. Deux tokens ont les scores $(\log 4, 0)$. Calculer leurs probabilités pour $\tau = 1$, $\tau = 2$ et $\tau = 1/2$, sans top-k. Déterminer ensuite la sortie pour une température négative dans ce dépôt.

Correction détaillée. Pour $\tau = 1$, les poids exponentiels sont $(4, 1)$, donc les probabilités $(4/5, 1/5)$. Pour $\tau = 2$, ils deviennent $(2, 1)$, donc $(2/3, 1/3)$. Pour $\tau = 1/2$, ils deviennent $(16, 1)$, donc $(16/17, 1/17)$. Une température négative active directement l'argmax : le premier token est sélectionné, sans calculer ces probabilités.

Point de vigilance. Diversité et qualité ne sont pas synonymes. Une température élevée peut rendre plus fréquents des tokens peu plausibles ; une température faible peut favoriser des répétitions. Comparer les mêmes prompts et annoncer si la sélection est gloutonne ou aléatoire permet d'interpréter ce réglage sans lui attribuer des effets d'apprentissage inexistantes.

Fiche 86 — Top-k : un seuil qui conserve aussi les égalités

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/07_generate.py

top_k_logits, generate

Objectif. Lire précisément le filtre de vocabulaire et son interaction avec le tirage multinomial. Le nom top-k suggère une cardinalité, mais l'implémentation emploie un seuil : cette différence apparaît lorsque plusieurs scores sont égaux.

Le filtre ne fait rien si `top_k` vaut `None`, est inférieur ou égal à zéro, ou est supérieur ou égal à V . Sinon, `torch.topk` extrait les k plus grandes valeurs. La dernière de ces valeurs devient un seuil a , utilisé séparément pour chaque ligne de logits. Le remplacement est

$$z'_i = \begin{cases} -\infty & \text{si } z_i < a, \\ z_i & \text{si } z_i \geq a. \end{cases}$$

La comparaison est strictement inférieure. Tous les tokens à égalité au seuil restent admissibles, même si leur nombre fait dépasser k . Après softmax, les scores remplacés par $-\infty$ reçoivent une probabilité nulle ; les autres sont renormalisés pour que leur somme vaille un.

À comprendre. `torch.multinomial(probs, num_samples=1)` choisit un identifiant selon cette distribution, et non uniformément parmi les candidats conservés. Un token plus probable peut donc être sélectionné plus souvent, sans devoir gagner à chaque appel.

Avec une température positive, le filtre intervient après la division des scores. Cette division préserve leur ordre mathématique ; elle modifie ensuite les proportions du tirage parmi les candidats restants. Avec une température non positive, le code contourne le filtre et prend directement un maximum. Avec `sample=False`, la branche positive sélectionne également un maximum, mais après avoir calculé les probabilités.

Exercice. Prendre les logits $(\log 4, \log 2, \log 2, 0)$, une température égale à un et $k = 2$. Combien de tokens sont conservés ? Calculer la distribution filtrée, puis la probabilité que deux tirages indépendants à distribution fixe choisissent tous deux le premier token.

Correction détaillée. Le deuxième plus grand score vaut $\log 2$. Les trois premiers tokens sont donc conservés ; le quatrième est exclu. Leurs poids $(4, 2, 2)$ donnent les probabilités $(1/2, 1/4, 1/4, 0)$. Pour deux tirages indépendants de cette même distribution, la probabilité demandée est $(1/2)^2 = 1/4$. Pendant une génération réelle, le premier token ajouté modifie le contexte : la seconde distribution n'est généralement plus identique.

Point de vigilance. Même $k = 1$ peut conserver plusieurs tokens si les maxima sont égaux : un échantillonnage reste alors possible entre eux. Dire « top-k garde exactement k tokens » serait donc inexact ici. Ce programme n'implémente ni top-p, ni pénalité de répétition, ni recherche par faisceaux ; ne pas déduire ces méthodes de la présence d'un simple filtre top-k.

Fiche 87 — Construire puis prolonger le cache des clés et valeurs

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/07_generate.py

generate ; 05_model.py : forward

Objectif. Suivre les formes du cache KV pendant le préremplissage et la génération incrémentale. Le cache évite certains recalculs du passé, mais ne contient ni une réponse mémorisée ni tous les logits précédents.

Au premier appel avec cache, `past_key_values` vaut `None`. Le modèle traite le prompt, tronqué à C tokens si nécessaire. Ce préremplissage calcule les clés et les valeurs à chaque couche. Le retour possède alors trois éléments : les logits, la perte ici absente, et un tuple contenant une paire de tenseurs par couche.

À comprendre. Si P positions sont déjà mémorisées, chaque paire a deux tenseurs de forme $[B, H, P, d_k]$, avec $d_k = D/H$. La mémoire couvre les représentations propres à chacune des L couches : partager une seule paire entre toutes les couches ne représenterait pas le même modèle.

Tant que $P < C$, l'appel suivant reçoit seulement le dernier token ajouté. Les nouvelles projections ont une longueur temporelle de un, puis les clés et valeurs sont concaténées à celles du cache. Les formes utiles deviennent

$$Q : [B, H, 1, d_k], \quad K, V_{\text{att}} : [B, H, P + 1, d_k].$$

La requête du nouveau token consulte ainsi tout le passé encore conservé. Ses positions commencent à l'offset P : embeddings appris ou rotations RoPE utilisent ce décalage. Les anciennes clés RoPE, déjà tournées, ne doivent pas subir une seconde rotation.

Le masque causal compare les positions des requêtes à celles des clés ; il reste correct même lorsque ces deux longueurs diffèrent. Le modèle vérifie aussi que toutes les couches partagent la même longueur passée et que la somme passé plus nouvelles positions ne dépasse pas C .

Exercice. Avec $B = 1$, $H = 4$, $D = 64$, $L = 2$ et un prompt de trois tokens, donner la forme de chaque tenseur du cache après préremplissage, puis après l'appel incrémental suivant. Combien de nombres le cache contient-il à chacune de ces étapes ?

Correction détaillée. Chaque tête possède $d_k = 16$ composantes. Les formes sont $[1, 4, 3, 16]$, puis $[1, 4, 4, 16]$. Il existe deux tenseurs par couche et deux couches : les totaux sont $2 \times 2 \times 1 \times 4 \times 3 \times 16 = 768$, puis 1024 nombres. Juste après le choix d'un token, celui-ci figure dans `idx`, mais n'entre dans le cache qu'au prochain passage du modèle.

Point de vigilance. Le cache est réservé au mode évaluation sans cibles. La fonction de génération désactive aussi les gradients. Une vérification de formes ne prouve pas qu'un cache provient du bon prompt : l'appelant doit respecter la continuité de la séquence et ne jamais mélanger deux historiques.

Fiche 88 — Au débordement, reconstruire toute la fenêtre

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/07_generate.py

generate : condition de débordement du cache

Objectif. Comprendre le choix exact de fenêtre glissante et ses conséquences sur les positions, l'équivalence avec le calcul sans cache et les performances. Le comportement à saturation est aussi important que le premier gain incrémental.

À chaque tour, la génération prépare d'abord les C derniers tokens de `idx`. Si un cache existe et si sa longueur est strictement inférieure à C , elle remplace cette entrée par le seul dernier token. Dans tous les autres cas, elle remet le cache à `None` et traite la fenêtre complète préparée au début du tour.

À comprendre. Lorsque le cache atteint C , le programme n'enlève pas simplement sa première clé et sa première valeur. Il reconstruit toutes les couches sur les derniers C tokens, avec des positions qui repartent de zéro. C'est aussi la convention du passage sans cache sur cette fenêtre.

Pour les positions apprises, décaler des tokens change directement leurs vecteurs positionnels. Avec RoPE, une translation commune préserve certaines relations relatives des rotations, mais cela ne suffit pas à recycler naïvement toutes les représentations : les états cachés des couches précédentes ont aussi intégré des tokens désormais sortis de la fenêtre. Le dépôt applique donc la même reconstruction aux deux variantes.

Si s tokens sont présents avant un choix, la longueur traitée sans cache est

$$T = \min(s, C).$$

Après saturation, elle vaut aussi C avec cache. Chaque ajout fait glisser la fenêtre ; le cache plein est donc abandonné au tour suivant. Le gain incrémental disparaît durablement, pas seulement lors d'un passage.

Exercice. Prendre $C = 4$, un prompt de deux tokens et demander cinq nouveaux tokens avec cache. Donner les longueurs des entrées effectivement transmises aux cinq appels du modèle. Indiquer les positions utilisées lors du premier appel reconstruit.

Correction détaillée. Les longueurs sont deux pour le préremplissage, un pour l'ajout à un cache de longueur deux, puis un pour atteindre quatre positions. Au quatrième appel, le cache plein est abandonné : l'entrée contient quatre tokens. Au cinquième, une nouvelle fenêtre de quatre tokens est encore reconstruite. Les positions du quatrième appel sont zéro, un, deux et trois, et non les indices absolus dans toute la conversation.

Point de vigilance. Les tests comparent les sorties gloutonnes avec et sans cache de part et d'autre de cette frontière. Ils contrôlent une équivalence de comportement sur leurs exemples, pas un gain permanent de vitesse. RoPE ne rend ni le cache ni le contexte illimités, et un prompt déjà long peut placer la génération immédiatement dans le régime sans gain incrémental.

Fiche 89 — Estimer le coût du cache sans promettre une vitesse

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/03_attention.py

MultiHeadSelfAttention.forward ; 07_generate.py : generate

Objectif. Comparer des ordres de grandeur en séparant calcul, stockage et temps réellement mesuré. Une formule de complexité explique une tendance ; elle ne prédit pas seule le comportement d'un processeur ou d'un GPU.

Pour une séquence de longueur T , le produit entre requêtes et clés construit des interactions entre toutes les paires de positions. Sur les H têtes, le travail dominant de cette partie est proportionnel à BT^2D par couche. Les projections et le réseau intermédiaire ajoutent des coûts proportionnels à $BT D^2$ et $BTDF$. La projection finale vers le vocabulaire ajoute également un travail proportionnel à $BT D V$.

À comprendre. Sans cache, chaque token généré redemande ces calculs pour toute la fenêtre. Avant saturation, le cache permet de ne projeter que le nouveau token ; son attention consulte toutefois les P clés précédentes. La partie attention devient alors proportionnelle à $B(P+1)D$ par couche, au lieu d'une attention complète quadratique sur cette longueur.

Sur une croissance idéale de longueur un à N , sans débordement et en isolant les interactions d'attention, on compare

$$\sum_{t=1}^N t^2 = O(N^3) \quad \text{à} \quad \sum_{t=1}^N t = O(N^2).$$

Le préremplissage initial reste à payer. Ces ordres ne décrivent pas le régime saturé du dépôt : une fois la fenêtre pleine, les reconstructions répétées réintroduisent le calcul complet sur C positions.

Le cache consomme aussi de la mémoire. Pour P positions et s octets par nombre, les clés et valeurs de toutes les couches représentent

$$M_{KV} = 2LBHPd_k s = 2LBPDs.$$

Cette estimation ne compte ni les paramètres, ni les activations temporaires, ni les copies produites par les concaténations. Elle décrit seulement le contenu numérique persistant des paires KV.

Exercice. Calculer cette mémoire pour $L = 2$, $B = 1$, $P = 32$, $D = 64$ en `float32`, avec quatre octets par nombre. Que devient-elle en `bfloat16`, à deux octets ? Est-ce la mémoire totale de génération ?

Correction détaillée. On compte $2 \times 2 \times 1 \times 32 \times 64 = 8192$ nombres. Ils occupent 32768 octets, soit 32 Kio en `float32`, et 16384 octets, soit 16 Kio en `bfloat16`. Ce n'est pas le total : il faut notamment ajouter le modèle, les logits, les calculs intermédiaires et les allocations de la bibliothèque.

Point de vigilance. Sur un petit modèle CPU, les appels Python, les concaténations et le décodage peuvent masquer une économie arithmétique. Une attention SDPA peut aussi changer les coûts temporaires sans changer cette formule du cache. Mesurer sur le matériel visé reste indispensable ; aucune accélération numérique n'est déduite automatiquement de ces seuls ordres de grandeur.

Fiche 90 — Deux usages différents de bfloat16

```
/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py
```

```
train, evaluate ; 07_generate.py : load_model_and_tokenizer
```

Objectif. Distinguer calcul en précision mixte et conversion des poids. Le même nom d'option n'entraîne pas exactement les mêmes opérations pendant l'apprentissage et pendant le chargement pour la génération.

Un nombre `float32` occupe quatre octets, contre deux pour `bfloat16`. Ce dernier conserve une large plage d'exposants, mais moins de chiffres significatifs. Il peut donc représenter des valeurs de grandes amplitudes tout en arrondissant plus fortement des nombres proches. Réduire la taille ne signifie ni conserver toutes les décimales ni réduire automatiquement chaque allocation du programme.

À comprendre. Dans `train`, le modèle est envoyé vers le périphérique sans conversion explicite de son type : ses paramètres restent normalement en `float32`. Le passage avant et le calcul de perte sont entourés de `torch.autocast`, activé seulement lorsque la précision demandée est `bfloat16`. PyTorch choisit alors les types selon les opérations.

La rétropropagation intervient après ce contexte. Le script ne construit pas de `GradScaler` et ne propose pas l'option `float16`. La validation emploie le même mécanisme d'autocast, en plus du mode évaluation et de l'absence de gradients. Le passage initial d'affichage des formes, lui, n'est pas entouré de cet autocast.

À l'inférence, le chargeur crée le modèle, charge les poids, puis appelle `model.to(device=device, dtype=dtype)`. Ici, choisir `bfloat16` convertit réellement les paramètres flottants du modèle. La fonction `generate` n'ajoute pas un contexte d'autocast analogue à celui de l'entraînement. Ses identifiants de tokens restent des entiers.

Autour de un, l'écart entre nombres représentables illustre la différence :

$$\Delta_{\text{bf16}} = 2^{-7} = 0,0078125, \quad \Delta_{\text{fp32}} = 2^{-23}.$$

Deux logits très proches peuvent devenir indiscernables après arrondi, ce qui peut affecter un `argmax` et, par propagation, toute la suite générée.

Exercice. Un modèle contient exactement un million de paramètres flottants. Estimer leur stockage en `float32` puis après conversion `bfloat16`. La métrique des octets de paramètres doit-elle forcément être divisée par deux pendant l'entraînement avec autocast ?

Correction détaillée. Les paramètres seuls occupent quatre millions puis deux millions d'octets. Pendant l'entraînement décrit, l'autocast ne convertit pas durablement tous les paramètres : leur compteur d'octets peut rester celui de `float32`. Il serait donc incorrect d'assimiler automatiquement précision des calculs et taille des poids stockés.

Point de vigilance. Sur CPU, la disponibilité et la rapidité des opérations réduites dépendent du matériel et de PyTorch. Commencer par `float32`, puis comparer explicitement exactitude et durée. Une précision réduite n'est ni une quantification entière ni une garantie d'accélération sur toute machine.

Fiche 91 — Mesurer SDPA et le cache avec un protocole explicite

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/07_generate.py

main : benchmark

Objectif. Lire le chronomètre comme une expérience contrôlée. Deux chemins équivalents mathématiquement ne possèdent pas toujours la même vitesse, et une mesure isolée ne suffit pas à identifier une accélération fiable.

Le mode `-benchmark` compare successivement la génération sans cache, puis avec cache, sur le même modèle chargé. Il force une température nulle et désactive les impressions intermédiaires. Les options habituelles de température et de top-k ne pilotent donc pas cette comparaison : le but est de confronter des continuations gloutonnes.

```
python /home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/07_generate.py \
  --checkpoint /tmp/mini-gpt-atelier-cpu/mini_gpt.pt \
  --device cpu --prompt "bonjour " --max-new-tokens 16 \
  --attention-backend sdpa --benchmark
```

À comprendre. Avant chaque mesure, le programme effectue une génération d'un token d'échauffement. Sur CUDA, il synchronise le périphérique et remet à zéro le compteur de pic mémoire, puis lance le chronomètre. Une seconde synchronisation précède la lecture du temps final.

Les opérations GPU peuvent être lancées de manière asynchrone. Sans synchronisation finale, on risquerait de mesurer principalement leur soumission plutôt que leur achèvement. Sur CPU, ces appels CUDA sont absents. Le temps mesuré englobe la fonction de génération, notamment l'encodage du prompt, les opérations de sélection et le décodage final.

Pour comparer attention manuelle et SDPA, répéter la même commande en remplaçant uniquement la valeur de `-attention-backend` par `manual`. SDPA est une interface de PyTorch qui choisit une implémentation compatible. Elle ne garantit pas Flash Attention : matériel, type numérique, formes et version peuvent conduire à un autre chemin, notamment sur CPU.

Le débit affiché est

$$q = \frac{N_{\text{nouveaux tokens}}}{\Delta t}.$$

La mesure affiche aussi l'égalité des deux textes gloutons. Cette égalité est utile, mais ne démontre pas une identité de tous les logits : des différences d'arrondi peuvent rester sans effet sur les maxima.

Exercice. Un protocole fictif mesure vingt tokens en une demi-seconde, puis en quatre dixièmes de seconde. Calculer les débits et le rapport d'accélération. Donner deux raisons de ne pas généraliser immédiatement.

Correction détaillée. Les débits valent 40 et 50 tokens par seconde ; le rapport de vitesse vaut $0,5/0,4 = 1,25$. Ces valeurs sont purement illustratives. La variabilité des mesures et le choix du prompt peuvent changer la conclusion ; notamment, un contexte saturé supprime ici le gain incrémental.

Point de vigilance. Répéter les mesures, conserver le même matériel et le même dtype, et noter l'ordre des essais. Un échauffement d'un token reste modeste. Ne pas attribuer à SDPA un résultat obtenu en changeant simultanément le checkpoint, la précision et la longueur demandée.

Fiche 92 — Comprendre le débit et les compteurs de mémoire

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

train : metrics

Objectif. Associer chaque métrique à son numérateur, son dénominateur et son périmètre. Un nombre correctement calculé peut être mal interprété si son intitulé est raccourci dans un tableau de résultats.

Le compteur `tokens_seen` augmente de `y.numel()` après chaque mise à jour. Il compte donc les positions cibles des lots d'entraînement effectivement traités. Les fenêtres chevauchantes et l'échantillonnage avec remise peuvent compter plusieurs fois la même position source. La validation n'augmente pas ce compteur.

À comprendre. Le temps démarre avant l'affichage des formes et avant les époques, mais après le prétraitement des données et la construction du modèle. Il s'arrête après les validations, avant la création de la figure et la sauvegarde du checkpoint. Le débit est donc celui des tokens d'entraînement avec le temps de validation inclus.

$$q_{\text{train}} = \frac{\text{positions cibles entraînées}}{\text{durée mesurée, validation comprise}}.$$

Les impressions de progression et le passage pédagogique initial contribuent aussi à cette durée. Une expérience de seulement quelques pas peut être fortement influencée par ces coûts fixes ; elle n'est pas forcément représentative d'un entraînement long.

Le nombre de paramètres additionne `numel()` sur les paramètres enregistrés. Leur volume additionne `p.numel() * p.element_size()`. Il ne comprend pas toutes les autres allocations : gradients, moments d'AdamW, activations conservées pour le retour arrière, lots, bibliothèque et interpréteur consomment aussi de la mémoire. Les poids d'entrée et de sortie n'étant pas liés, les deux ensembles participent au nombre affiché.

Sur CUDA, `max_memory_allocated` mesure le pic des allocations de tenseurs suivies par PyTorch depuis la remise à zéro du compteur. Ce n'est ni toute la mémoire réservée par son allocateur ni toute la mémoire occupée par le processus sur la carte. Sur CPU, `peak_cuda_bytes` vaut `None` : aucun pic CPU n'est mesuré.

Exercice. Supposer 2048 positions d'entraînement, deux secondes pour leurs calculs et une seconde supplémentaire de validation et autres opérations chronométrées. Calculer le débit publié et le débit qui ignorerait ce surcoût. Pourquoi les deux ne répondent-ils pas à la même question ?

Correction détaillée. Le débit publié vaut $2048/3 \simeq 682,7$ positions par seconde. Un débit limité aux calculs d'entraînement vaudrait $2048/2 = 1024$. Le premier décrit le coût du parcours mesuré ; le second isolerait une partie du travail. Le script ne mesure pas séparément cette deuxième durée : elle a seulement été supposée pour l'exercice.

Point de vigilance. Conserver dans les données archivées l'intitulé complet `training_tokens_per_second_including_validation`, quitte à l'expliquer en français dans un graphique. La taille des paramètres n'est jamais à présenter comme la mémoire totale requise pour entraîner ou générer.

Fiche 93 — Ce que prouvent réellement les tests d'intégration

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/tests/test_integration.py

IntegrationTests

Objectif. Lire une assertion comme une garantie locale, sans transformer un test technique réussi en évaluation linguistique. Les tests du dépôt utilisent `unittest`, la bibliothèque standard de Python, et de petits modèles CPU.

La commande ciblée ci-dessous sélectionne le fichier d'intégration existant. Elle est proposée au lecteur ; son écriture dans ce manuel ne signifie pas qu'une exécution et ses résultats sont rapportés ici.

```
python -m unittest discover \
    -s /home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/tests \
    -p test_integration.py -v
```

À comprendre. `test_train_reload_character_defaults` entraîne pendant un pas, vérifie notamment $D = 64$, $C = 32$, les variantes par défaut, une entrée de validation et la création de la courbe. Il recharge ensuite le modèle et vérifie une génération de trois tokens depuis `bonjour` .

`test_bpe_training_reload` couvre un entraînement BPE miniature avec RoPE et SDPA, la restauration des identifiants, un aller-retour accentué et des générations avec et sans cache. Si la dépendance `tokenizers` manque, ce test est ignoré : il ne faut pas annoncer cette branche comme validée dans un tel environnement.

`test_legacy_checkpoint` fabrique une ancienne sauvegarde et compare les logits rechargés avec ceux du modèle initial. `test_cache_generation_across_window_boundary` compare les textes gloutons pour positions apprises et RoPE, attention manuelle et SDPA, avec un prompt court puis un prompt plus long que le contexte.

`test_empty_prompt_and_zero_generation` vérifie le refus du prompt vide et le cas de zéro ajout. `test_split_precedes_windows` contrôle la séparation de listes avant fenêtrage dans le helper `create_data loaders`. Cette assertion ne couvre pas à elle seule tout le prétraitement memmap réellement utilisé par `train`.

`test_invalid_options` examine plusieurs options refusées : époques nulles, pas nul, limite négative, préréglage inconnu, précision non prévue et taux non fini. Enfin, `test_notebook_code_syntax` compile chaque cellule de code du notebook sans l'exécuter.

Exercice. Le test caractère obtient une chaîne de longueur onze après trois ajouts au prompt `bonjour` . Prouve-t-il une bonne syntaxe française ? Le test du notebook prouve-t-il que toutes ses cellules s'exécutent correctement ?

Correction détaillée. Non aux deux questions. Le prompt contient huit caractères connus ; trois tokens caractères expliquent la longueur onze, indépendamment de leur qualité linguistique. Compiler une cellule vérifie sa syntaxe Python, mais pas la présence des dépendances, les valeurs des variables ni la réussite de ses calculs lors d'une exécution.

Point de vigilance. Un test réussi ne démontre que les propriétés effectivement assertées dans ses conditions. Ces tests ne mesurent ni une accélération matérielle ni la généralisation du modèle, et leurs exemples finis ne remplacent pas toutes les vérifications de formes et de limites.

Fiche 94 — Comparer des expériences à unités constantes

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/README.md

Checkpoints et comparaisons

Objectif. Construire une comparaison interprétable où chaque changement a une signification identifiable. La reproductibilité commence par les données, les unités et les options, pas seulement par l'ajout d'une graine aléatoire.

Pour comparer deux variantes d'architecture, fixer le corpus, sa séparation, le tokenizer, le contexte, le pas des fenêtres et la validation. Changer plusieurs éléments simultanément rend difficile l'attribution d'un gain à une seule cause. Par exemple, une validation plus facile peut réduire la perte même sans amélioration du modèle.

À comprendre. Une perte moyenne est exprimée par token. Or un token caractère et un token BPE ne couvrent pas la même quantité de texte. Leurs pertes brutes et leurs débits en tokens par seconde ne sont donc pas directement comparables, même lorsque les deux entraînements lisent un même fichier.

La perplexité conserve cette dépendance à l'unité :

$$\text{PPL} = e^{\mathcal{L}}.$$

Transformer la perte en perplexité ne résout donc pas le problème. Une étude entre tokenizers peut examiner les mêmes prompts décodés, la mémoire et le temps pour une quantité comparable de texte. Annoncer comment cette quantité est mesurée, sans réutiliser aveuglément une métrique par token.

Le script appelle `torch.manual_seed` avec la graine de configuration. Cela contrôle des tirages PyTorch, mais ne garantit pas une identité universelle entre matériels, versions, backends et types numériques. La CLI de génération ne possède pas d'option `-seed` et son échantillonnage n'est pas automatiquement fixé par la graine enregistrée dans le checkpoint.

Comparer cache et backend avec un même checkpoint, son tokenizer et des prompts identiques, en sélection gloutonne. Pour les entraînements, séparer les dossiers de sortie, relever les configurations et répéter avec plusieurs graines plutôt que retenir seulement le meilleur cas.

Exercice. Un modèle caractère affiche une perte fictive de 1,5, un modèle BPE une perte de 2,0. Peut-on classer leur qualité ? Puis deux modèles utilisent exactement le même tokenizer et la même validation, avec pertes 1,5 et 1,4 : calculer le rapport de leurs perplexités.

Correction détaillée. La première comparaison ne permet pas un classement direct : les unités diffèrent. Dans la seconde, le rapport est $e^{1,5}/e^{1,4} = e^{0,1} \simeq 1,105$; le second modèle possède une perplexité plus faible sur cette validation. Cela ne garantit pas une meilleure réponse à chaque prompt ni une amélioration hors distribution.

Point de vigilance. Archiver versions, matériel, précision et commandes exactes. Une graine unique n'est pas un intervalle d'incertitude. Les textes produits doivent également être choisis selon un protocole annoncé, et non sélectionnés après coup uniquement parce qu'ils donnent une impression favorable.

Fiche 95 — Diagnostiquer une erreur avant de changer le modèle

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

validations ; 07_generate.py : generate, load_model_and_tokenizer

Objectif. Classer les échecs par étape pour corriger leur cause plutôt que multiplier des modifications au hasard. Un modèle pédagogique peut être techniquement correct et produire malgré tout un texte peu convaincant.

Vérifier chemin du corpus, encodage UTF-8, longueur des portions et valeurs positives pour le contexte, le batch, les couches et le pas des fenêtres. Une portion de N tokens doit contenir une cible après la fenêtre d'entrée :

$$N > C, \quad D \bmod H = 0, \quad \text{avec RoPE : } (D/H) \bmod 2 = 0.$$

La première condition concerne les données ; les autres, l'architecture. Augmenter les époques ne corrige aucune de ces erreurs.

À comprendre. Une erreur de chargement n'est pas une erreur de génération. Vérifier le chemin du checkpoint, son format, la configuration et la présence de la dépendance BPE avant de modifier la température. Un fichier explicitement demandé mais absent, hors chemin par défaut, doit provoquer une erreur plutôt qu'une substitution silencieuse par des poids aléatoires.

Pour un problème de cache, contrôler le mode évaluation, l'absence de cibles, le nombre de paires égal à L , leurs formes, leurs périphériques et leurs types. La somme des tokens passés et nouveaux doit rester dans le contexte. La génération gère le débordement par reconstruction ; un appel direct doit respecter ces contraintes.

Si la perte devient non finie, vérifier les entrées et le taux d'apprentissage, puis revenir à une configuration petite en `float32`. Le code refuse un taux non fini, mais ne garantit pas qu'un taux fini excessif entraîne de façon stable. Il n'applique pas de clipping pour rattraper automatiquement une telle situation.

Une génération répétitive peut refléter un petit corpus, peu d'apprentissage ou un choix glouton. La température change la sélection, pas les connaissances. Le projet n'implémente ni format de dialogue, ni adaptation aux instructions, ni RLHF, ni récupération documentaire, ni mécanisme de vérification factuelle.

Exercice. Diagnostiquer trois cas : un prompt vide avec zéro token demandé ; une configuration RoPE avec $D = 60$, $H = 4$; une génération qui continue après un point final jusqu'à vingt ajouts. Lesquels constituent des erreurs ?

Correction détaillée. Le prompt vide est refusé avant la boucle, même pour zéro ajout. La dimension par tête vaut quinze : elle est impaire, donc RoPE est refusé. La continuation après un point n'est pas une erreur d'arrêt : aucun EOS ni signe de ponctuation ne déclenche l'arrêt dans ce programme. La longueur demandée commande seule le nombre d'ajouts.

Point de vigilance. Ne pas attribuer une compréhension fiable à une phrase plausible. Ce modèle prédit des tokens et peut inventer des affirmations. Ses limites pédagogiques ne sont pas des options cachées qu'une commande de génération permettrait d'activer.

Fiche 96 — Projet final : expliquer, vérifier et conclure avec prudence

/home/runner/work/SHOKOBO/SHOKOBO/mini_gpt/06_train.py

train ; 07_generate.py : generate ; tests/test_integration.py

Objectif. Assembler un compte rendu qui relie les données, les dimensions, l'apprentissage et la génération. Le projet final évalue la solidité du raisonnement autant que la capacité à lancer une commande.

Choisir un corpus local autorisé, suffisamment long. Décrire sa séparation, le tokenizer et les fenêtres. Avec une configuration adaptée au CPU, réaliser un essai court dans un dossier distinct. Conserver commandes absolues, versions et paramètres effectifs ; ce contrôle technique ne prouve pas la qualité.

À comprendre. Le rapport doit suivre une prédiction complète : un identifiant désigne un embedding, les blocs causaux produisent une représentation, la tête fournit V logits, et la perte compare ces scores au token suivant. Les gradients modifient les poids ; la génération réutilise ensuite les derniers logits sans effectuer cet apprentissage.

Comparer les courbes avec leurs limites, recharger le checkpoint et expliquer ses champs. Examiner enfin les continuations sur une petite liste fixée de prompts, avec et sans cache en mode glouton. Une différence exige une enquête ; une égalité ne démontre ni une accélération ni une qualité linguistique. Séparer clairement observations effectuées, calculs théoriques et hypothèses.

Exercice. Évaluation de synthèse, avec nombres fictifs. Choisir $B = 2$, $C = T = 8$, $D = 16$, $H = 2$, $L = 2$, $F = 32$ et RoPE. Deux lots de validation contiennent seize puis huit cibles et ont les pertes deux puis trois. Un prompt de trois tokens reçoit six ajouts avec cache. Vérifier l'architecture, calculer la validation, donner les longueurs des six appels et dire si le checkpoint permet une reprise exacte d'AdamW.

Correction détaillée. La dimension par tête est huit, entière et paire : RoPE est valide. La validation vaut $(16 \times 2 + 8 \times 3)/24 = 7/3$ nats par cible. Les longueurs des appels sont trois, un, un, un, un, un : le sixième atteint un cache de longueur huit. Un septième appel reconstruirait toute la fenêtre. Le checkpoint conserve les poids, mais pas les moments d'AdamW ni tous les états nécessaires à une reprise exacte. Ces réponses n'utilisent aucun résultat d'entraînement supposé.

Pour prolonger l'étude, consulter les sources primaires : la [documentation PyTorch](#) pour `CrossEntropyLoss`, AdamW, autocast et SDPA ; *Attention Is All You Need* pour le Transformer ; *RoFormer* pour les positions rotatives. Lire ces textes comme des références, pas comme une description automatique de chaque choix de ce dépôt.

Point de vigilance. La réussite finale consiste à pouvoir justifier une forme, une équation, une commande et une limite à partir du code réel. Sans test indépendant, sans mesure réalisée ou sans mécanisme implémenté, ne pas annoncer respectivement une généralisation, une accélération ou une capacité d'assistant que le projet n'a pas démontrée.